

People's Democratic Republic of Algeria
الجمهورية الجزائرية الديمقراطية الشعبية
Ministry of Higher Education and Scientific Research
وزارة التعليم العالي و البحث العلمي



المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الإعلام الآلي سابقا)
École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Final Year Project Report

In partial fulfillment of the requirements for the State Engineer Degree in
Computer Science

Option: Intelligent Systems and Data (SD)

Transformer Compression Using Genetic Algorithms

Presented by:

MAHRAZ ABDELRAHMEN
REZIG HAMAZA

Supervised by:

MEZIANI Lila
GHORAB SARA

Host organization : LCSi ESI

Cohort: 2026/2027

Abstract

Compressing Transformer-based language models requires deciding which heads, neurons, layers, and bit-widths to retain. These decisions are interdependent, yet existing methods make them independently, scoring each unit in isolation before the effect of the full set of changes is known. We hypothesize that this local view misses interactions between retained components, leaving the final configuration suboptimally allocated. This work treats the compression configuration as the optimization target and proposes a general genetic algorithm compression framework for jointly assigning structured pruning and quantization decisions under explicit budget constraints.

The search divides the configuration space into regions using geometric clustering, with each region maintained as an independent population that adapts its own search strategy over time. The framework is tested on an encoder-based language model across text classification tasks under matched recovery conditions, and on a decoder-based language model under joint layer-drop and quantization search evaluated on language modeling benchmarks.

Results are competitive with structured compression baselines on both model families. On the encoder model, the method performs comparably to prior structured pruning approaches at multiple compression targets. On the decoder model, the joint search achieves competitive perplexity against other evolutionary methods in fewer generations.

Keywords: structured pruning, quantization, genetic algorithms, transformer compression, knowledge distillation

Résumé

Comprimer les modèles de langage à base de Transformers implique de décider quelles têtes, neurones, couches et largeurs de bits conserver. Ces décisions sont interdépendantes, pourtant les méthodes existantes les traitent de manière indépendante, en évaluant chaque composant isolément avant de connaître l’effet de l’ensemble des changements. Nous formulons l’hypothèse que cette vue locale rate les interactions entre composants conservés, laissant la configuration finale sous-optimalement allouée. Ce travail traite la configuration de compression comme l’objet d’optimisation et propose un cadre général de compression par algorithmes génétiques pour assigner conjointement les décisions d’élagage structuré et de quantification sous contraintes explicites de budget.

La recherche divise l’espace de configuration en régions par regroupement géométrique, chaque région étant maintenue comme une population indépendante qui adapte sa propre stratégie de recherche au fil du temps. Le cadre est évalué sur un modèle de langage à encodeur pour des tâches de classification de texte, ainsi que sur un modèle à décodeur dans le cadre d’une recherche conjointe de suppression de couches et de quantification, mesuré sur des benchmarks de modélisation du langage.

Les résultats sont compétitifs face aux méthodes de compression structurée de référence sur les deux familles de modèles. Sur le modèle à encodeur, la méthode obtient des performances comparables aux approches d’élagage structuré antérieures à plusieurs régimes de compression. Sur le modèle à décodeur, la recherche conjointe atteint une perplexité compétitive face aux autres méthodes évolutionnaires en moins de générations.

Mots-clés : élagage structuré, quantification, algorithmes génétiques, compression de Transformers, distillation de connaissances

ملخص

يقتضي ضغط النماذج اللغوية القائمة على بنية المحوّلات انتقاءً دقيقاً لما يُستبقى من رؤوس الانتباه، والعصبونات، والطبقات، إلى جانب تحديد مستويات التكميم (Quantization) لأرقام الفاصلة العائمة. وتتداخل هذه الخيارات عضوياً ضمن البنية الشاملة للنموذج؛ إذ لا يتجلى الأثر الفعلي لأي قرار بمفرده، بل يتبلور عبر تفاعله مع بقية المكونات في النموذج المُكَمَّم والنهائي. وتنهض هذه الدراسة على فرضية مفادها أن التقييم الموضوعي لكل مكون على حدة يحجب تفاعلات مفصلية بين الأجزاء المستبقاة، مما قد يُفضي إلى توجيهٍ قاصرٍ لموارد الضغط.

وتأسيساً على ذلك، تعتمد هذه الدراسة إلى صياغة تهيئة الضغط بحد ذاتها كمسألة تستوجب التحسين الآلي. ولتحقيق هذه الغاية، نقترح إطاراً عاماً للضغط يستلهم الخوارزميات الجينية للبحث في فضاء تهيئة الضغط، محكوماً بقيود حسابية صارمة. ويعتمد هذا الإطار على تجزئة فضاء التهيئة إلى نطاقات هندسية متميزة؛ بحيث يُدار كل نطاق كمجتمع بحثي مستقل، يطوّر استراتيجيته ذاتياً مع توالي الأجيال.

اختُبر الإطار على نموذج قائم على المُشَفِّر فقط في مهام تصنيف النصوص، ضمن شروط استرداد متطابقة. كما اختُبر على نموذج قائم على فَالِك التشفير فقط في بحث يشمل حذف الطبقات والتكميم، مع قياس الحيرة على معايير نمذجة اللغة.

تدل النتائج على قدرة الإطار على منافسة خطوط الأساس في ضغط نماذج المحوّلات. ففي النموذج القائم على المُشَفِّر فقط، يبلغ أداءُ يضاهي طرائق التقليم الهيكلي السابقة عند أهداف ضغط متعددة. أما في النموذج القائم على فَالِك التشفير فقط، فيبلغ البحث قيمةً تنافسية للحيرة قياساً بطرائق تطويرية منافسة، خلال عدد أقل من الأجيال.

الكلمات المفتاحية: التقليم الهيكلي، التكميم، الخوارزميات الجينية، ضغط المحوّلات، تقطير المعرفة.

Dedication

Acknowledgements

Contents

Abstract	i
Résumé	ii
iii	ملخص
Dedication	iv
Acknowledgements	v
List of Acronyms and Abbreviations	xvi
General Introduction	1
I State of the Art	2
1 Foundations	3
1.1 Introduction	3
1.2 Parameterization of Neural Networks	4
1.2.1 The Neuron as a Linear Model	4
1.2.2 Activation Functions	4
1.2.3 The Multi-Layer Perceptron (MLP)	6
1.3 Optimization and Gradients	7
1.3.1 Empirical Risk Minimization	7
1.3.2 Algorithms for Gradient-Based Optimization	8
1.4 Backpropagation	10
1.5 Specialized Architectures	11
1.5.1 Recurrent Neural Networks (RNNs)	11
1.5.2 Autoencoders (AEs)	12
1.5.3 Transformer Networks	13
1.5.4 Core Architectural Components: Building Blocks of the Transformer	13
1.5.5 Encoder and Decoder Architecture	15
1.5.6 The Computational Bottleneck of Transformers	17
1.5.7 Hardware-Aware Efficiency Metrics	18
1.6 Genetic Algorithms and Multi-Objective Search	20

1.6.1	Genetic Algorithm Fundamentals	20
1.6.2	Compression as a Search Problem	22
1.6.3	Multi-Objective Optimization and Pareto Dominance	22
1.6.4	NSGA-II	23
1.7	Conclusion	24
2	Neural Networks Compression	25
2.1	Introduction	25
2.2	Quantization	25
2.2.1	Mathematical Formulation	26
2.2.2	Quantization Granularity and Schemes	26
2.3	Neural Network Pruning	28
2.3.1	Formal Problem Formulation	29
2.3.2	Structural Forms of Pruning	29
2.4	Low-Rank Factorization	30
2.4.1	Mathematical Formulation	30
2.4.2	Applications	31
2.5	Knowledge Distillation	31
2.5.1	Mathematical Formulation	32
2.5.2	Methodological Taxonomies	32
2.6	Conclusion	34
3	Related Work	35
3.1	Introduction	35
3.2	Related Work on Post-Training Quantization	35
3.2.1	Problem Setting and Taxonomy	36
3.2.2	Rotation-Based Methods	37
3.2.3	Saliency-Based Methods	40
3.2.4	Compensation-Based Methods	42
3.2.5	Optimization-Based Methods	46
3.2.6	Search-Based Methods	48
3.2.7	Empirical Comparison	49
3.3	Related Work on Pruning	52
3.3.1	Problem Setting and Taxonomy	52
3.3.2	Saliency-Based Methods	53
3.3.3	Gradient-Based Methods	54
3.3.4	Second-Order Methods	56
3.3.5	Search-Based Methods	61
3.3.6	Empirical Comparison	65
3.4	Discussion	68
3.4.1	Local scoring and its limits	68
3.4.2	Structured pruning as architecture selection	69
3.4.3	Why global allocation remains unresolved	69
3.4.4	Need for evolutionary search	70

3.5	Conclusion	71
II	Contribution	72
4	Methodology	73
4.1	Candidate Parameterisation	74
4.1.1	Encoder Structured-Pruning Encoding	74
4.1.2	Encoder Structural Counts	74
4.1.3	Encoder Granularity and Candidate Identity	75
4.1.4	Decoder Compression Encodings	76
4.2	Efficiency Budget and Cost Model	77
4.3	The Surrogate Distance Metric	78
4.3.1	Equal-Cost Candidate Ranking	78
4.3.2	Residual Update Matching	79
4.3.3	Aggregation Across Layers	80
4.3.4	Task-Facing Distance After Structural Filtering	81
4.3.5	Two-Step Cohort Ranking	81
4.3.6	Calibration Noise and Resampling	82
4.3.7	Limits of the Surrogate	82
4.4	The Genetic Search Algorithm	83
4.4.1	The Level Database	83
4.4.2	Search Interface and Feasibility Objective	83
4.4.3	The Five-Phase Genetic Procedure	85
4.5	The Detailed Pipeline	88
4.5.1	Proposal Scoring	88
4.5.2	Initialisation: Structure-Aware Sweep	90
4.5.3	CVT Partition and Island Construction	91
4.5.4	Operator Library	94
4.5.5	Bandit and Niche Conditioning	95
4.5.6	The Generation Loop	96
4.5.7	Archives	99
4.5.8	Final Funnel	100
4.5.9	Fine Evaluation and Selection	101
4.6	Recovery Process (Encoder)	101
4.6.1	Intermediate Representation Recovery	102
4.6.2	Prediction-Level Distillation	104
4.7	Technical Architecture of the Solution	105
4.8	Technologies Used	106
4.8.1	Python	106
4.8.2	Transformers	106
4.8.3	PyTorch	107
4.8.4	NumPy	107
4.8.5	Matplotlib	107

4.8.6	Google Drive	108
4.8.7	Git	108
4.8.8	GitHub	109
4.9	Datasets Used	109
4.9.1	Pre-Training Corpora	109
4.9.2	Fine-Tuning and Evaluation Benchmarks	109
4.9.3	Language Model Evaluation Datasets	111
4.10	Model	112
4.10.1	BERT	112
4.10.2	EleutherAI Pythia-1.4B	112
4.11	Experimental Environment	112
4.11.1	Baselines and Reporting	113
4.11.2	Level Database and Token Budgets	113
4.12	Conclusion	114
5	Results	115
5.1	Result Tables and Discussion	115
5.1.1	Encoder Classification Results	116
5.1.2	Decoder Compression Results (Pythia-1.4B)	118
5.2	Conclusion	120
	Conclusion and Perspectives	121
	Appendices	122

List of Figures

1.1	A single artificial neuron with input vector x , weight vector w , bias b , and pre-activation output z . The activation function f is applied to z to produce the final output y	4
1.2	Comparison of common activation functions	6
1.3	Fully connected MLP with 4 input features, two hidden layers (5 and 4 neurons), and 2 output neurons.	7
1.4	Recurrent Neural Network (RNN) unrolled in time. The input x_t at each time step updates the hidden state h_t through the recurrent weights W_{hh} , while W_{xh} maps inputs to hidden states and W_{hy} maps hidden states to outputs y_t	12
1.5	Autoencoder architecture	12
1.6	Complete Encoder-Decoder Transformer architecture. The encoder memory matrix H_{enc} (highlighted in blue) is fed to all decoder layers for cross-attention. Residual connections (curved arrows) enable stable gradient flow throughout the network.	15
1.7	Transformer encoder layer architecture. The input embedding (Emb.) is combined with positional information, processed by self-attention and a Feed-Forward Network (FFN), and stabilized by residual connections with Layer Normalization (LayerNorm).	16
1.8	Transformer decoder layer architecture. The decoder contains masked self-attention for previous target tokens, cross-attention for reading encoder outputs, and a Feed-Forward Network (FFN), each wrapped with residual connections and Layer Normalization (LayerNorm).	17
2.1	Uniform quantization (left) divides the input range into L equal-width bins, assigning each interval $[b_i, b_{i+1})$ to a fixed representative r_i . Non-uniform quantization (right) applies a companding function $g(x)$ to warp the input space, allocating finer resolution at lower magnitudes where values concentrate and coarser bins at higher magnitudes.	28
2.2	A dense weight matrix can be pruned into unstructured sparsity, semi-structured 2:4 sparsity, or structured row, column, and block sparsity. The distinction is important at deployment time because increasingly regular sparsity patterns align more directly with accelerator execution primitives.	30
2.3	Schematic of Low-Rank Factorization. The dense matrix W is decomposed into low-rank components A and B , significantly reducing parameter count when $r \ll m, n$	32

2.4	The Knowledge Distillation framework. The student minimizes a dual objective: the task loss $\mathcal{L}_{\text{task}}$ against hard labels and the distillation loss $\mathcal{L}_{KD}$ against the teacher’s soft probability targets. Inspired from mansourian2025comprehensivesurveyknowledgedistillation	34
3.1	Taxonomy of post-training quantization methods	37
3.2	QuaRot FFN quantization pipeline	38
3.3	QuIP methodology	39
3.4	OWQ overview	41
3.5	GPTQ algorithm	43
3.6	VPTQ methodology	44
3.7	OmniQuant methodology	47
3.8	Taxonomy of the pruning methods considered in this chapter. The classification crosses the pruning unit, sparsity topology, optimization signal, recovery stage, deployment objective, and evaluation regime.	53
3.9	Second-order compensation strategies for unstructured and semi-structured pruning. SparseGPT removes one scalar weight per step and compensates using one column of $\hat{H}^{-1}$, while MRP removes an N:M group jointly and uses the block $(\hat{H}_{MM}^{-1})^{-1}$ to model interactions among simultaneously removed weights.	58
3.10	ZipLM pipeline. The global phase benchmarks attention-head and FFN configurations and searches for layer-wise sparsity profiles satisfying the required speedup. The local phase applies second-order structured pruning through inverse-Hessian scoring, compensation updates, and iterative Hessian refinement.	60
3.11	EvoPress $(1 + \lambda)$ evolutionary search loop for dynamic LLM compression (evopress2024). The method applies budget-preserving level-switch mutation and progressively evaluates candidates with larger calibration budgets.	62
3.12	DarwinLM search loop (darwinlm2025). The method first builds a sparsity-level database with OBS-style structured pruning, then performs training-aware evolutionary selection over stitched candidate models.	64
4.1	Five-phase overview of the genetic search. Phase 1 computes proposal scores from the teacher on the fixed evaluation subset. Phase 2 builds a feasible seed pool by a structure-aware sweep. Phase 3 runs the island GA: a Centroidal Voronoi Tessellation (CVT) partition of $\mathcal{X}_B$ (du1999centroidal) and a T -generation evolutionary loop with archive updates. Phase 4 pools finalists and re-evaluates them at full fidelity. Phase 5 selects $\hat{c}$ from the pooled finalists.	83
4.2	Inputs and output of the search stage. The candidate representation, compute budget, and surrogate metric are fixed before search. The search procedure explores feasible candidates and returns one selected mask $\hat{c}$ for recovery.	84
4.3	The feasibility band $\mathcal{X}_B$ as a populated space. Feasible candidates form clusters of architecturally similar masks; clusters in different regions of the band differ qualitatively.	86

4.4	The four pieces of machinery inside the generational loop. Regions maintain separate populations, operators produce feasible offspring after repair, bandits adapt operator probabilities, and selection feeds both the next population and the archives.	87
4.5	One iteration of the search and the final funnel. Each region’s population produces offspring with operators chosen by its bandit; offspring are projected back into the region, evaluated on the calibration packs, ranked by the cohort gate, and used to update the population and the archives. After T iterations, the archives and the per-region best candidates are re-evaluated at higher fidelity and one mask $\hat{c}$ is returned.	88
4.6	Proposal scoring. For each calibration batch, gradients of the teacher’s per-layer update target with respect to head and neuron gates yield squared sensitivities and a reconstruction MSE; their combination, row-normalised per layer, produces $s_H[\ell, h]$ and $s_N[\ell, n]$. Summing over each layer gives $\text{imp}(\ell)$. Sorting $s_N[\ell, \cdot]$ in descending order and accumulating block sums yields the table $S_\ell[b]$, from which marginal gain and loss queries take $O(1)$	89
4.7	Structure-aware sweep. Each sampled design point $(N_a, \phi_H, \text{mode})$ yields one candidate. The subset mode chooses the active layers from the layer importance scores; the head–neuron split ϕ_H divides the budget; and the importance-weighted per-layer allocation distributes the split into integer head counts and block-aligned neuron counts.	91
4.8	From scored sweep candidates to islands. Sweep candidates are projected into the (z_1, z_3) plane on the left and form a continuous cloud. The centre-based clustering step assigns each descriptor to its nearest centre and updates each centre to the mean of its assigned descriptors. This produces Voronoi cells. Each cell becomes a basin $\mathcal{B}_j$ on the right; the best scored candidates assigned to that basin seed the island $\mathcal{I}_j$, and the island then evolves with its own population, best-so-far candidate, and operator-selection bandit.	94
4.9	One generation of Phase 3 inside a single island. The current population is scored, parents are selected with crowding correction and novelty, offspring are produced and projected back into the basin, the combined cohort is ranked by the two-step cohort rule of Section 4.3.5, and the surviving candidates form the next population. Archive updates use the same ranked cohort and are detailed in Section 4.5.7.	99
4.10	Recovery flow after architecture search. The mask is fixed throughout recovery; only the surviving parameters are trained.	102
4.11	Technical pipeline of the implemented compression framework. A pretrained Transformer and task dataset are loaded first, a baseline setup fixes the training configuration, the search stage selects a compressed architecture under the active budget constraint, recovery fine-tuning adapts the selected candidate when needed, and the final stage evaluates the model and saves checkpoints, metrics, logs, and figures.	105
4.12	Python logo.	106
4.13	Hugging Face Transformers logo.	106

List of Figures

4.14	PyTorch logo.	107
4.15	NumPy logo.	107
4.16	Matplotlib logo.	108
4.17	Google Drive logo.	108
4.18	Git logo.	108
4.19	GitHub logo.	109

List of Tables

1.1	Summary of common neural network loss functions.	8
2.1	Standard numerical formats commonly used in neural network quantization ieee754_2019 ; kalamkar2019bfloat16 ; jacob2018quantization ; krishnamoorthi2018quantizing ; lin2024awq . The integer ranges correspond to signed stored integer values before dequantization; the represented real-valued range depends on the quantization scale and, when used, the zero-point.	28
3.1	PTQ methods by family	50
3.2	Weight-only quantization spectrum	50
3.3	W4A4 perplexity comparison	51
3.4	EvoPress non-uniform quantization results	51
3.5	Preparation cost and evaluation requirements of representative pruning methods. The table separates low-cost post-training scoring, training-dependent structured pruning, curvature-based reconstruction, and population-based search.	64
3.6	Representative unstructured and semi-structured pruning results on WikiText perplexity. PPL denotes perplexity; lower is better.	65
3.7	Representative structured Transformer pruning results across recovery regimes. GLUE denotes the General Language Understanding Evaluation benchmark, SQuADv1 denotes Stanford Question Answering Dataset version 1.1, acc. denotes accuracy, and F1 denotes question-answering F1 score.	66
3.8	Representative search-based pruning results. Wiki2 and C4 are perplexity values where lower is better; task average is accuracy where higher is better.	67
3.9	Aggregated interpretation of the empirical pruning evidence by regime.	68
4.1	Layer statuses induced by the structural masks.	75
4.2	Candidate encodings and budget constraints across the five compression families. The table records what a candidate stores before any search operator, evaluation rule, or recovery step is applied.	77
4.3	Six encoder descriptor coordinates used to partition the feasible sweep pool.	92
4.4	Input formats and label spaces for the selected evaluation tasks.	110
5.1	Selected headline results. Accuracy is higher-is-better for GLUE; perplexity is lower-is-better for Pythia-1.4B.	115

5.2	Development accuracy under matched recovery for three search strategies. Random search and beam search use the same teacher, budget band, and recovery procedure as the proposed method.	116
5.3	Comparison against structured pruning baselines and the uncompressed task baseline. Scores are development accuracy.	117
5.4	Perplexity ($\downarrow$) on Pythia-1.4B at 25% sparsity. EvoPress results are reported at generation 72; ours at generation 11.	118
5.5	Perplexity ($\downarrow$) on Pythia-1.4B at 50% sparsity. EvoPress results are reported at generation 90; ours at generation 30.	118
5.6	Additional 50% layer-sparsity result under a lower calibration budget for both methods. Perplexity is lower-is-better.	119
5.7	Perplexity ($\downarrow$) on Pythia-1.4B under low-rank factorization. EvoPress is the baseline.	119
5.8	Perplexity ($\downarrow$) on Pythia-1.4B under low-rank factorization, after recovery fine-tuning.	119
5.9	Perplexity ($\downarrow$) on Pythia-1.4B at 2.25 bits average quantization.	120
5.10	Perplexity ($\downarrow$) on Pythia-1.4B at 2.5 bits average quantization.	120

List of Acronyms and Abbreviations

Adam	Adaptive Moment Estimation
AdaLoRA	Adaptive Budget Allocation via Low-Rank Adaptation
AE	Autoencoder
ALBERT	A Lite BERT
BERT	Bidirectional Encoder Representations from Transformers
BF16	Brain Float 16
CE	Cross-Entropy Loss
CNN	Convolutional Neural Network
CoFi	Coarse-to-Fine Structured Pruning via Learned Masks
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
CVT	Centroidal Voronoi Tessellation
DarwinLM	Training-Aware Evolutionary Structured Pruning
DoRA	Weight-Decomposed Low-Rank Adaptation
DyLoRA	Dynamic Low-Rank Adaptation with Flexible Rank Selection
ELU	Exponential Linear Unit
EMA	Exponential Moving Average
EvoPress	Evolutionary Search over Layer-Wise Compression Profiles
EXP3	Exponential-weight algorithm for Exploration and Exploitation
FFN	Feed-Forward Network
FLOPs	Floating Point Operations

List of Acronyms and Abbreviations

FP16	16-bit Floating Point
FP32	32-bit Floating Point
FT	Fine-Tuning
GA	Genetic Algorithm
GaLore	Gradient Low-Rank Projection
GELU	Gaussian Error Linear Unit
GLUE	General Language Understanding Evaluation
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
GSM8K	Grade School Math 8K
INT4	4-bit Integer
INT8	8-bit Integer
KD	Knowledge Distillation
KL	Kullback-Leibler Divergence
LLaMA	Large Language Model Meta AI
LLM	Large Language Model
LDLQ	LDL-Transpose Quantization
LLM-Pruner	Dependency-Aware Gradient-Based Structured Pruning
LoRA	Low-Rank Adaptation
MACs	Multiply-Accumulate Operations
MAE	Mean Absolute Error
MAP-Elites	Multi-dimensional Archive of Phenotypic Elites
MHA	Multi-Head Attention
MLP	Multi-Layer Perceptron
MNLI	Multi-Genre Natural Language Inference
MOP	Multi-Objective Optimization Problem
MRP	Multi-Weight Removal Process

List of Acronyms and Abbreviations

MSE	Mean Squared Error
N:M	Semi-Structured Sparsity Pattern
NAS	Neural Architecture Search
NLP	Natural Language Processing
NSGA-II	Non-Dominated Sorting Genetic Algorithm II
OBS	Optimal Brain Surgeon
OPT	Open Pre-trained Transformer
PEFT	Parameter-Efficient Fine-Tuning
PiSSA	Principal Singular Values and Vectors Adaptation
PPL	Perplexity
QNLI	Question Natural Language Inference
ReLU	Rectified Linear Unit
RMSNorm	Root Mean Square Normalization
RNN	Recurrent Neural Network
rsLoRA	Rank-Stabilized LoRA
SGD	Stochastic Gradient Descent
SoRA	Sparse Low-Rank Adaptation
SparseGPT	Second-Order Unstructured Pruning via Hessian Compensation
SQuAD	Stanford Question Answering Dataset
SVD	Singular Value Decomposition
Tanh	Hyperbolic Tangent
VeRA	Vector-Based Random Matrix Adaptation
Wanda	Weight and Activation-Based Pruning Criterion
ZipLM	Runtime-Aware Second-Order Structured Pruning

General Introduction

Current neural networks achieve high accuracy at the expense of performing numerous computations with massive numbers of weights. This poses a deployment problem as we are faced with memory, latency, and power constraints in the environment where the model needs to run. These trade-offs between performance and cost are particularly transparent in Transformer models due to multiple repetitions of attention projections, feed-forward layers, residual connections, and normalization steps. In this report we will explore pruning as one approach to lower cost by pruning weights and entire Transformer components from a trained network.

In order to pose that problem in an accurate way, the report first defines some of the foundations of neural networks, Transformer architecture and the cost on hardware. This part presents the basic vocabulary which is used to express what can be discarded, what is modified in the remaining network and where the cost arises during execution. From that definition, pruning is presented as a constrained modification on a trained model, described by the concepts of density, sparsity, topology, and recovery.

With all that we've discussed in the previous chapters, we arrive at pruning, where we isolate the questions that a method needs to answer. What structure can be removed, when importance is given to that structure by a criterion, at what stage removal is carried out, and under which practical regime the method falls, are defined in the chapter, and then mapped back to the context of Transformers in the shape of attention heads, feed-forward channels and layers.

The report then reviews the main pruning state of the art methods through this structure and the empirical comparison reads their results under the same lens. The point is not to list methods one by one, together, they show the recurring issue behind these methods: pruning decisions depend on local importance and on the way the compression budget is allocated across the model.

Part I

State of the Art

Chapter 1

Foundations

1.1 Introduction

The purpose of pruning is to keep the behavior of a trained neural network while modifying its size by removing some of its parameters or biggest parts. In order to study what is eligible to be removed and how the network will be effected by removing these parts it is first needed to explain what are the objects that pruning can work on: Neurons, weights, activations, layers, feed-forward neural network. The vocabulary is not given here as general knowledge.

We begin this chapter by giving a description of a single artificial neuron. This is necessary for pruning because weights and biases are the smallest parameters which are available for training and activation determines where the information should propagate to after weights and biases have been updated. It naturally leads us to multi-layer perceptrons, loss functions, gradient based methods and backpropagation because we measure the performance of a pruning technique by the amount to which the output behavior, or the loss, changes when removing some part of a networkGoodfellow2016; Rumelhart1986.

1.2 Parameterization of Neural Networks

An artificial neuron is a computing unit that takes an input vector, it applies a linear transformation and finally inputs this result to a non-linear activation function. This form of an artificial neuron enables the artificial neural network to learn non-linear mappings. The linear part is also called an affine transformation, which computes a weighted sum of the input vector and then adds a bias. The non-linear activation function is important to enable the artificial neural network to learn complex patterns, since it is responsible for approximate the non-linearities in the data. Examples for common activation functions are ReLU, sigmoid and tanh.

1.2.1 The Neuron as a Linear Model

The artificial neuron, formally introduced by **McCulloch1943**, serves as the fundamental building block of neural networks. Mathematically, it performs an **affine transformation** on its inputs followed by a non-linear activation f which is represented in Figure 1.1 and expressed in Eq.(1.1).

$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b \quad (1.1)$$

Given an input $\mathbf{x} \in \mathbb{R}^n$, weights $\mathbf{w} \in \mathbb{R}^n$ scale the importance of each feature, while the bias $b \in \mathbb{R}$ acts as an offset. Figure 1.1 shows this affine unit before the activation is applied: the weighted inputs are summed with the bias to produce the pre-activation response z .

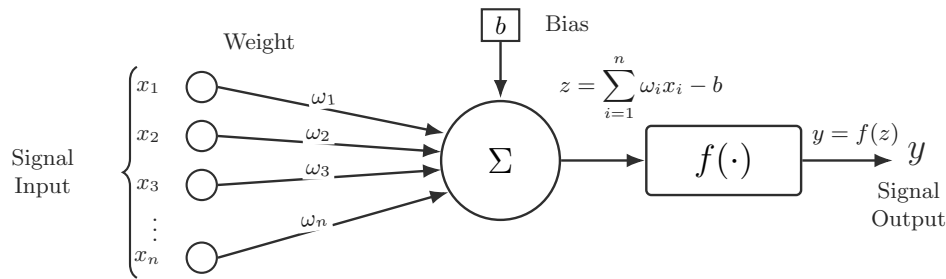


Figure 1.1: A single artificial neuron with input vector $\mathbf{x}$, weight vector $\mathbf{w}$, bias b , and pre-activation output z . The activation function f is applied to z to produce the final output y .

The term $\mathbf{w}^T \mathbf{x}$ measures how strongly the input matches the learned weights. This affine computation is the basic operation repeated across layers in larger neural networks.

1.2.2 Activation Functions

The neural network's ability to model complex patterns depends on the non-linear activation functions applied after each affine transformation. If we were to stack multiple layers without any non-linearity, the entire network would collapse into a single affine transformation.

Therefore, depth alone is not enough: neural networks require non-linear **activation functions** to model complex patterns **Goodfellow2016**.

As shown in Figure 1.1, the activation function f takes the pre-activation output z and produces the final output $y = f(z)$. The choice of activation function can significantly affect the network's performance, convergence, and ability to learn from data.

Classical Saturating Activations

Sigmoid (Eq.(1.2)) and hyperbolic tangent (Eq.(1.3)) functions were often used in early networks. While these functions take real-valued inputs and output bounded values, which could be useful in the case of probabilities or normalized hidden states, they have very small derivatives when the input is very large or very small. This saturation causes learning to be very slow in deep networks **Glorot2010**; **Goodfellow2016**.

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma : \mathbb{R} \rightarrow (0, 1). \quad (1.2)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \tanh : \mathbb{R} \rightarrow (-1, 1). \quad (1.3)$$

Rectified Activations

Rectified Linear Units (ReLU) became common in deep learning because they are simple, computationally cheap, and reduce saturation for positive inputs **Nair2010**. ReLU is defined as showing in Eq.(1.4):

$$\text{ReLU}(x) = \max(0, x), \quad \text{ReLU} : \mathbb{R} \rightarrow [0, \infty). \quad (1.4)$$

Leaky ReLU keeps a small non-zero slope for negative inputs, reducing the risk that a unit becomes permanently inactive **Maas2013**, as shown in Eq.(1.5):

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases} \quad \alpha \in (0, 1), \quad \text{LeakyReLU} : \mathbb{R} \rightarrow \mathbb{R}. \quad (1.5)$$

The Exponential Linear Unit (ELU) uses a smooth negative branch and a linear positive branch **Clevert2016**, which can lead to faster convergence and improved performance in some cases, as shown in Eq.(1.6):

$$\text{ELU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha(e^x - 1), & x < 0, \end{cases} \quad \alpha > 0, \quad \text{ELU} : \mathbb{R} \rightarrow (-\alpha, \infty). \quad (1.6)$$

Modern Transformer architectures often use the Gaussian Error Linear Unit (GELU), a smooth activation that weights inputs by their value under the standard Gaussian cumulative

distribution **Hendrycks2016**, as shown in Eq.(1.7):

$$\text{GELU}(x) = x\Phi(x) \approx \frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3) \right) \right). \quad (1.7)$$

Figure 1.2 compares the shapes of these activation functions on the same input range. The plot makes the difference between saturating functions and rectified functions visible.

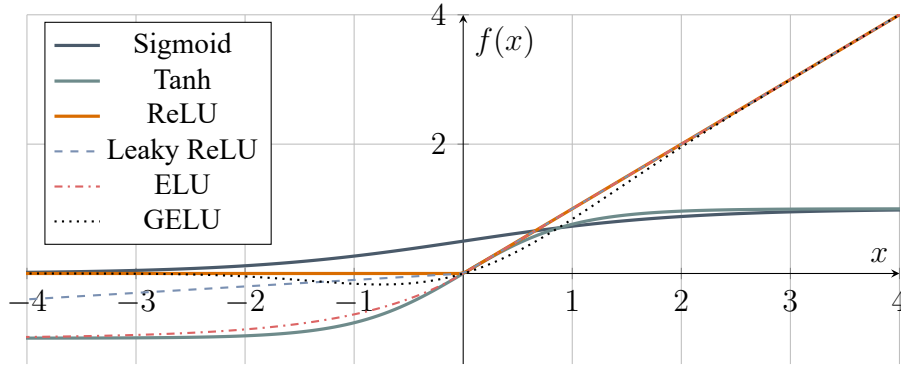


Figure 1.2: Comparison of common activation functions

1.2.3 The Multi-Layer Perceptron (MLP)

Although a single active neuron can map an input to a basic response, any practical model would require many neurons to co-operate together. In a Multi-Layer Perceptron, neurons are stacked into layers where one layer transforms the output of the preceding one.

Going from a single neuron to an MLP increases expressiveness by composing affine transformations with non-linear activations across layers **Rumelhart1986**. MLPs are *feedforward* and typically *fully connected*, meaning each neuron connects to all neurons in the next layer.

Eq. (1.8) formalizes the computation performed by each layer of an MLP, to transform the raw input x into the final output (prediction) y .

$$\begin{aligned} \mathbf{h}^{(0)} &= \mathbf{x}, \\ \mathbf{h}^{(\ell)} &= f(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}) \quad \text{for } \ell = 1, 2, \dots, L-1, \\ \mathbf{y} &= f_{\text{out}}(\mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}), \end{aligned} \quad (1.8)$$

Figure 1.3 illustrates a typical MLP with one input layer, two hidden layers, and one output layer.

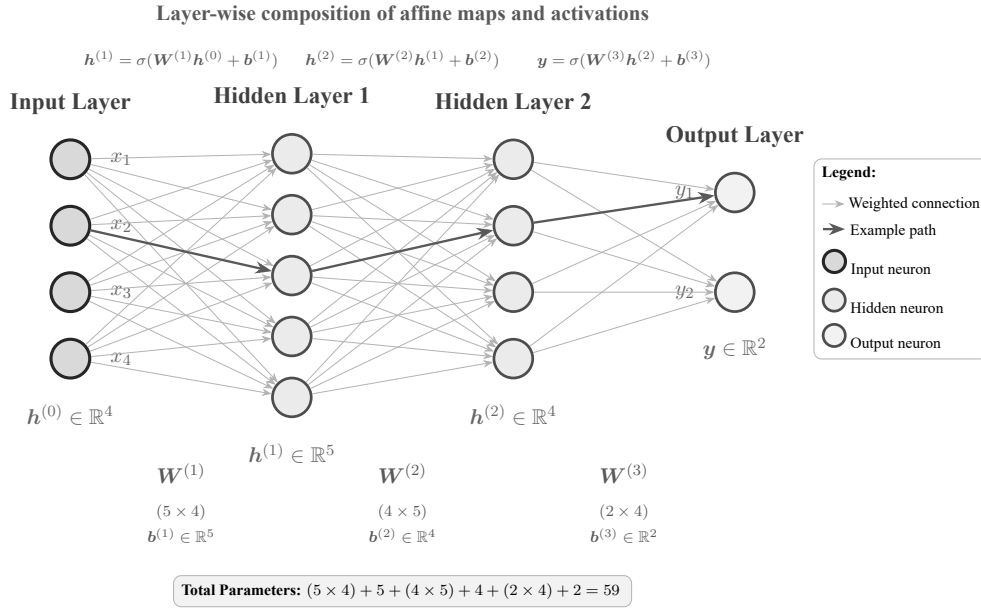


Figure 1.3: Fully connected MLP with 4 input features, two hidden layers (5 and 4 neurons), and 2 output neurons.

1.3 Optimization and Gradients

We outlined how an MLP generates a prediction which is by a composition of transformations applied at each layer. With forward computation, learning can be thought of as figuring out what the weights and biases should be given the data. Network construction then becomes an optimization problem: select the parameters that make the output as close as possible to the target on a training set.

Hence, a quantitative measure must be provided that can actually be minimized; this provides the rationale for empirical risk minimization.

1.3.1 Empirical Risk Minimization

Given a dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, learning consists of finding parameters θ that minimize the discrepancy between the model predictions $f_{\theta}(\mathbf{x})$ and the target values. This is commonly formulated as empirical risk minimization **Vapnik1998**:

$$\theta^* = \arg \min_{\theta} \hat{R}(\theta) = \arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m \mathcal{L}(f_{\theta}(\mathbf{x}^{(i)}), y^{(i)}). \quad (1.9)$$

where m is the number of training samples; $\mathbf{x}^{(i)}$ and $y^{(i)}$ are the input and target of the i -th sample; θ are the model parameters; $\hat{R}(\theta)$ is the empirical risk, i.e. the average loss over the training set; and $\mathcal{L}(\cdot, \cdot)$ is the per-sample loss function. For a K -class classification problem the prediction is a probability vector $\hat{\mathbf{y}} \in \Delta^K$ with entries $\hat{y}_k$, the target $\mathbf{y}$ has entries y_k , and c denotes the index of the true class; the Kullback–Leibler divergence **cover2006** $D_{\text{KL}}(\mathbf{y} \parallel \hat{\mathbf{y}})$ measures the excess cost of using $\hat{\mathbf{y}}$ in place of $\mathbf{y}$.

This empirical-risk objective in Eq. (1.9) gives the general learning template: choose parameters by minimizing the average loss over the training set. It does not specify the loss itself. That choice is determined by the problem and the structure of the output.

Loss Functions

Now we have an optimization problem. But what are we minimizing? A loss function accounts for the discrepancy between predictions and targets and makes explicit assumptions about the problem and the output space. Table 1.1 summarizes the most common loss functions used in neural network training, along with their formulas, target types, and properties. The choice of loss function depends on the task at hand (classification vs regression) and the nature of the targets (hard one-hot labels, soft probability distributions, or continuous real values.).

Table 1.1: Summary of common neural network loss functions.

Loss	Task	Formula	Target type	properties
Cross-Entropy	Classification	$\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{y}_k$	Soft or one-hot	Measures total encoding cost of $\mathbf{y}$ under code optimal for $\hat{\mathbf{y}}$; reduces to $-\log \hat{y}_c$ for hard labels.
KL Divergence	Classification	$D_{\text{KL}}(\mathbf{y} \parallel \hat{\mathbf{y}}) = \sum_{k=1}^K y_k \log \frac{y_k}{\hat{y}_k}$	Soft distribution	Measures <i>excess</i> encoding cost; $D_{\text{KL}} = \mathcal{L}_{\text{CE}} - H(\mathbf{y})$. Preferred over CE when targets are soft.
MSE (L2)	Regression	$\mathcal{L}_{\text{MSE}}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$	Continuous	Penalises errors quadratically; equivalent to maximum likelihood estimation (MLE) under Gaussian noise. Sensitive to outliers.
MAE (L1)	Regression	$\mathcal{L}_{\text{MAE}}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m \hat{y}^{(i)} - y^{(i)} $	Continuous	Penalises errors linearly; robust to heavy-tailed distributions and outliers.

Notes. K : number of classes; $\hat{\mathbf{y}} \in \Delta^K$: predicted probability vector (softmax output); $\mathbf{y}$: target distribution (one-hot or soft); c : index of the true class; $H(\mathbf{y}) = - \sum_k y_k \log y_k$: entropy of the target; m : number of samples; $\hat{y}^{(i)}, y^{(i)}$: predicted and true scalar outputs for sample i . When $\mathbf{y}$ is one-hot, $H(\mathbf{y}) = 0$ and $D_{\text{KL}} = \mathcal{L}_{\text{CE}}$.

1.3.2 Algorithms for Gradient-Based Optimization

With the loss function established, the remaining question is how the resulting error signal propagates back through the network to update the parameters.

Training neural networks is usually a problem of finding a parameter vector $\theta \in \mathbb{R}^d$ that minimizes the scalar objective $J(\theta)$ where J is the empirical risk defined by the loss function and the training data. Formally, we want to solve (1.10):

$$\theta^* = \arg \min_{\theta} J(\theta). \quad (1.10)$$

The parameter space in most contemporary neural networks is high-dimensional and the objective is largely non-convex so a solution in closed form for optimum parameters rarely

exists. Training is hence iterative: the parameters are updated according to directions suggested by the gradient of the loss **Bottou2018**.

The gradient $\nabla J(\theta)$ indicates the direction of steepest local increase of the objective, and optimizers will use it in reverse to adjust the parameters to yield a lower loss. This concept forms the basis of the gradient descent algorithm as well as of all the adaptive optimization schemes currently used in training neural networks.

Gradient Descent

Gradient descent updates parameters in the negative-gradient direction. For iteration t and learning rate $\eta > 0$, the update is formulated in Eq.(1.11):

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t). \quad (1.11)$$

The learning rate controls the step size. If it is too small, optimization may progress slowly; if it is too large, the updates can overshoot useful regions of the objective surface. In large scale learning, the exact gradient over the full dataset is often replaced by a stochastic estimate computed on a mini-batch. This gives stochastic gradient descent, which is cheaper per update and is widely used for neural-network training **Bottou2018**.

Adam

Adam is an adaptive gradient-based optimizer that combines momentum with per-parameter learning-rate adaptation **kingma2015**. Instead of using only the current gradient, Adam keeps exponential moving averages of the first and second moments of the gradient, which are updated as shown in Eq.(1.12):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (1.12)$$

where $g_t = \nabla J(\theta_t)$ is the gradient estimate at iteration t , m_t tracks the average direction of recent gradients, and v_t tracks their squared magnitude.

Because both moving averages are initialized at zero, Adam applies bias correction computed as follows Eq.(1.13):

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (1.13)$$

The parameter update is then given by (1.14):

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (1.14)$$

Adam is often utilized due to its robustness in relation to scaling differences among gradients for different parameters and typically requires less fine-tuning of learning rates as opposed

to simple gradient descent. Subsequent versions of Adam include AdamW, which removes the weight decay from the adaptive learning component and is applicable to a variety of deep learning applications **Loshchilov2019**.

Both gradient descent and Adam describe how the parameters should be updated, but they both require that the gradients be available. These computations can become expensive in deep, multi-layered networks and thus require an alternative method, termed backpropagation.

1.4 Backpropagation

As established in Section 1.3.2, gradient descent and Adam require gradients of the loss with respect to the parameters. Backpropagation is the mechanism that connects the loss at the output of the network to the parameters inside each layer by applying the chain rule backward through the network **Rumelhart1986**.

During the forward pass, each layer transforms the representation produced by the previous one. If the input to layer l is $a^{(l-1)}$, then the layer first forms

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad (1.15)$$

$$a^{(l)} = \phi(z^{(l)}), \quad (1.16)$$

where $W^{(l)}$ is the weight matrix, $b^{(l)}$ is the bias vector, $z^{(l)}$ is the pre-activation, $a^{(l)}$ is the activated output, and ϕ is the activation function. The loss $\mathcal{L}(a^{(L)}, y)$ is only observed at the final output, but every earlier layer contributed to that output through the sequence of transformations that followed it.

The central idea is therefore to move the effect of the loss backward through the network. At each layer, backpropagation keeps track of how sensitive the loss is to the pre-activation value of that layer:

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}}. \quad (1.17)$$

For the last layer, this quantity is obtained directly from the derivative of the loss and the derivative of the activation:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \odot \phi'(z^{(L)}), \quad (1.18)$$

where $\odot$ denotes element-wise multiplication. For an earlier layer, the signal is obtained from the layer after it:

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \phi'(z^{(l)}). \quad (1.19)$$

This is the chain rule applied repeatedly along the computational path from the loss back to the parameters.

Once this backward signal is known, the gradients of the layer parameters follow from the same local quantities used in the forward pass:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T, \quad (1.20)$$

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}. \quad (1.21)$$

The update of a weight is therefore tied to the activation that entered the layer and to the error signal that returned to it. Backpropagation avoids deriving each parameter derivative separately; it reuses the stored forward activations and propagates one backward signal from layer to layer.

Since prediction is done by forward propagation and learning is done by backpropagation, we have established the core mechanism of training a neural network. This mechanism does not depend on a specific architecture: if the network defines a differentiable function from inputs to outputs, its parameters can be trained by this method. What differs between network families is the set of assumptions they make about the input data.

This is the idea behind the following section, which breaks away from the general MLP setup and looks at specific architectures. The rationale for specific architectures is that some inputs cannot be treated as abstract, unordered vectors, as is the case with time series because of temporal order, compressed representations because of latent structure, and language because of word dependencies.

1.5 Specialized Architectures

While Multi-Layer Perceptrons offer a generic learning model over vector inputs, they do not inherently leverage any underlying structure of the data, and thus can only indirectly learn the temporal dependencies of sequences, the latent structure of data compressed in autoencoders, or the relational structure of positions within a context in token-based models. This fundamental limitation gives rise to architectures tailored to the structure of the data.

1.5.1 Recurrent Neural Networks (RNNs)

RNNs are designed for sequential data (e.g., time series, text). Unlike feedforward networks, RNNs utilize cyclic connections to maintain a *hidden state* that captures historical information. The network dynamics are described by:

$$h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

where h_t is the hidden state, x_t the input, and y_t the output at time t . Figure 1.4 represents this recurrence by unrolling the same recurrent computation across time steps, so that the dependence of h_t on h_{t-1} becomes visible.

RNNs focus on preserving information across time. Another important architectural idea is to preserve only the most essential information by compressing the input into a lower-dimensional representation.

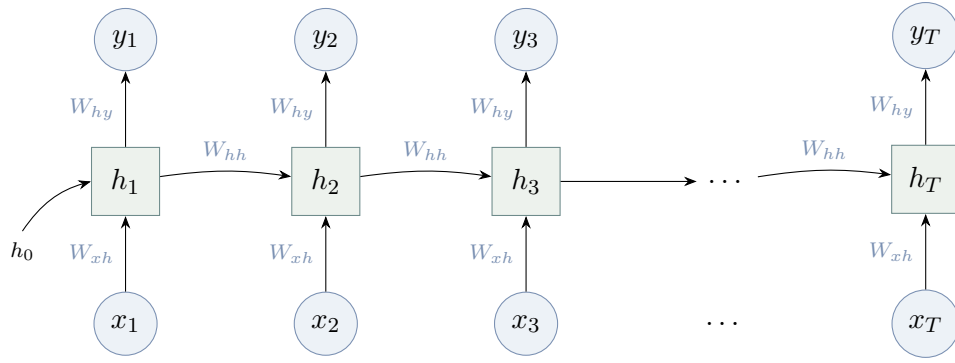


Figure 1.4: Recurrent Neural Network (RNN) unrolled in time. The input x_t at each time step updates the hidden state h_t through the recurrent weights W_{hh} , while W_{xh} maps inputs to hidden states and W_{hy} maps hidden states to outputs y_t .

1.5.2 Autoencoders (AEs)

Autoencoders are neural networks designed for unsupervised learning of efficient data codings. They act as an identity function that learns to compress and then reconstruct data. Figure 1.5 shows this structure as an encoder that maps the input to a latent space and a decoder that reconstructs the output from that compressed representation.

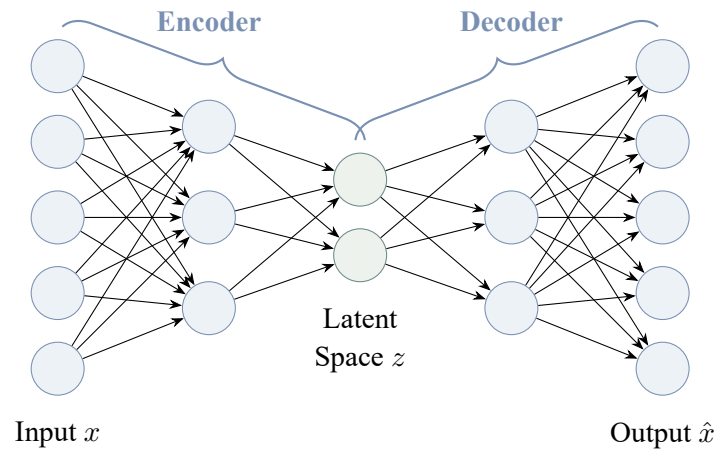


Figure 1.5: Autoencoder architecture. The encoder compresses the input x into a lower-dimensional latent space z , also called the bottleneck, and the decoder reconstructs the output $\hat{x}$ from that latent representation.

- **Encoder:** Maps the high-dimensional input x to a lower-dimensional latent representation z .
- **Decoder:** Reconstructs the output $\hat{x}$ from the latent representation z .

The network is trained to minimize the reconstruction error (e.g., Mean Squared Error) between the input and the output:

$$L(x, \hat{x}) = ||x - \hat{x}||^2$$

By forcing data through a bottleneck, the network learns to capture the most salient features of the data distribution.

Autoencoders show how architectural structure can impose a useful compression constraint. For the language and sequence models considered later in this report, the most important specialized architecture is the Transformer.

1.5.3 Transformer Networks

Transformer networks, introduced by **vaswani2017attention**, changed sequence modeling by replacing recurrent processing with attention-based computation. Instead of processing tokens one after another, a Transformer can compare tokens across the whole sequence in parallel. This makes it highly effective for language modelling, translation, summarization, and other sequence tasks. The original architecture was an encoder-decoder model for machine translation, while later systems often use encoder-only variants such as BERT **devlin2019bert** or decoder-only variants such as LLaMA **touvron2023llama2**.

1.5.4 Core Architectural Components: Building Blocks of the Transformer

Before separating the model into encoder and decoder stacks, it is useful to define the components repeated inside those stacks. Transformer layers are built from token embeddings, positional information, attention projections, feed-forward channels, residual connections, and normalization layers. These components are also the main targets for compression, because they determine both parameter count and runtime cost.

Input Representation: Embeddings and Positional Encoding

Every token is mapped to a continuous embedding vector. Since self-attention does not inherently encode token order, positional information is added to the token representations. The original Transformer uses sinusoidal positional encodings **vaswani2017attention**:

$$PE_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right). \quad (1.22)$$

Self-Attention Mechanism

Self-attention allows each token representation to aggregate information from the rest of the sequence. The mechanism uses three learned projections of each input: queries, keys, and values **vaswani2017attention**. Queries represent what a token is looking for, keys represent what each token offers for comparison, and values contain the information that will be aggregated.

For an input sequence representation $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$, the projections are:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V. \quad (1.23)$$

Scaled dot-product attention is then computed as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (1.24)$$

The factor $\sqrt{d_k}$ prevents dot products from becoming too large, which would push the softmax into saturated regions with weak gradients.

Multi-Head Attention

Multi-head attention applies several attention mechanisms in parallel, allowing the model to attend to different representation subspaces and token relationships at the same time [vaswani2017attention](#). For h heads:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_i^Q, \mathbf{X}\mathbf{W}_i^K, \mathbf{X}\mathbf{W}_i^V), \quad i = 1, \dots, h. \quad (1.25)$$

The head outputs are concatenated and projected back to the model dimension:

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O. \quad (1.26)$$

Here, $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ is the output projection. Attention heads and their projection matrices are important pruning targets because entire heads can sometimes be removed with limited accuracy loss.

Feed-Forward Network

After attention mixes information across tokens, a position-wise Feed-Forward Network (FFN) processes each token representation independently:

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2. \quad (1.27)$$

The FFN usually expands the hidden representation to a larger intermediate dimension d_{ff} and then projects it back to d_{model} . In the original Transformer, d_{ff} is typically much larger than d_{model} , making FFN channels a major source of parameters and a natural target for structured pruning.

Residual Connections and Layer Normalization

Each attention or FFN sub-layer is wrapped by a residual path and layer normalization. Residual connections preserve a direct path for information and gradients, while normalization stabilizes the scale of hidden representations. This design is essential for training deep Transformer stacks and is also important when later compression methods remove heads, channels, or blocks [vaswani2017attention](#); [He2016](#); [Ba2016](#).

1.5.5 Encoder and Decoder Architecture

With the core components defined, the encoder and decoder can be described cleanly. The full encoder-decoder Transformer contains two main stacks:

- **Encoder:** Processes the complete input sequence in parallel and produces contextual source representations.
- **Decoder:** Generates the output sequence autoregressively while attending to both previous target tokens and the encoder output.

The encoder converts the source sequence into a contextual memory representation H_{enc} . The decoder uses this memory through cross-attention while also using masked self-attention to prevent access to future target tokens. The complete architecture is shown in Figure 1.6.

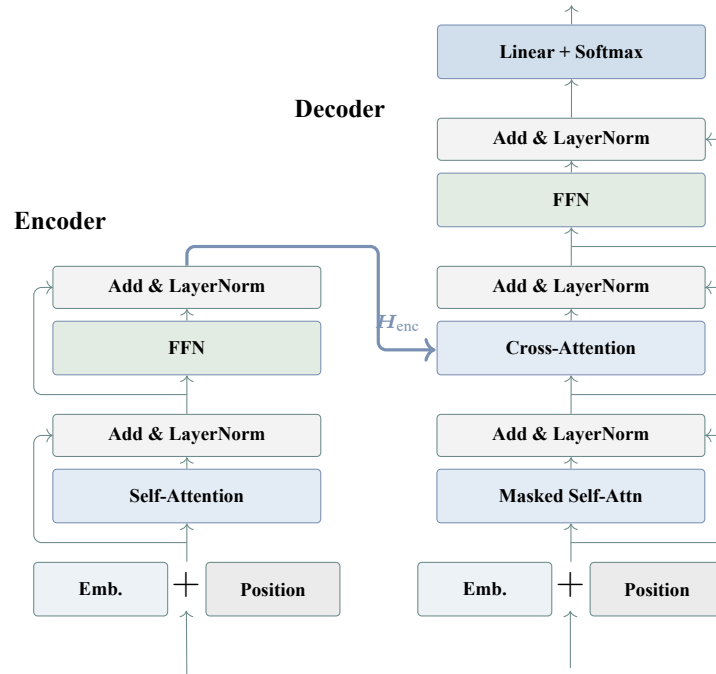


Figure 1.6: Complete Encoder-Decoder Transformer architecture. The encoder memory matrix H_{enc} (highlighted in blue) is fed to all decoder layers for cross-attention. Residual connections (curved arrows) enable stable gradient flow throughout the network.

The Encoder Stack

An encoder layer contains multi-head self-attention followed by a feed-forward network, with residual connections and normalization around each sub-layer. In encoder self-attention, the queries, keys, and values all come from the same source sequence. This makes the encoder bidirectional: every input token can attend to every other input token. Figure 1.7 shows the internal order of these operations before the resulting representation is passed to the decoder.

After N encoder layers, the output is the memory matrix $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{n \times d_{\text{model}}}$, which summarizes the source sequence for the decoder.

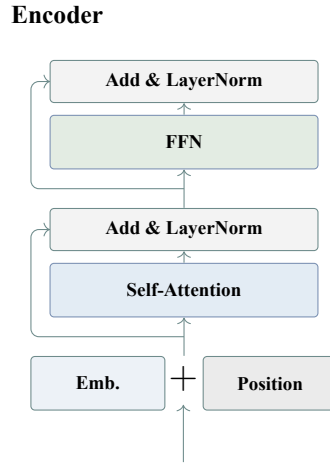


Figure 1.7: Transformer encoder layer architecture. The input embedding (Emb.) is combined with positional information, processed by self-attention and a Feed-Forward Network (FFN), and stabilized by residual connections with Layer Normalization (LayerNorm).

The Decoder Stack

A decoder layer adds masked self-attention and cross-attention before the feed-forward network, enabling autoregressive generation while attending to encoder outputs. Masked self-attention restricts token t to positions $\leq t$, preventing information leakage from future target tokens. Cross-attention then links the decoder to the encoder: decoder states act as queries, while encoder states provide keys and values. Figure 1.8 shows this additional cross-attention block, which is the main structural difference between the decoder layer and the encoder layer.

Encoder-Decoder Information Flow

The complete sequence-to-sequence computation proceeds as follows:

1. **Encoding phase.** Source tokens $\mathbf{x} = (x_1, \dots, x_n)$ are embedded, combined with positional information, and passed through N encoder layers. The output is $\mathbf{H}_{\text{enc}}$.
2. **Decoding phase.** Target tokens $\mathbf{y} = (y_1, \dots, y_t)$ are embedded and processed autoregressively. Each decoder layer applies masked self-attention, cross-attention over $\mathbf{H}_{\text{enc}}$, and a feed-forward transformation.
3. **Output phase.** The final decoder representation is projected to vocabulary logits. A softmax layer converts logits into token probabilities, and the selected token is fed back into the decoder at the next step.

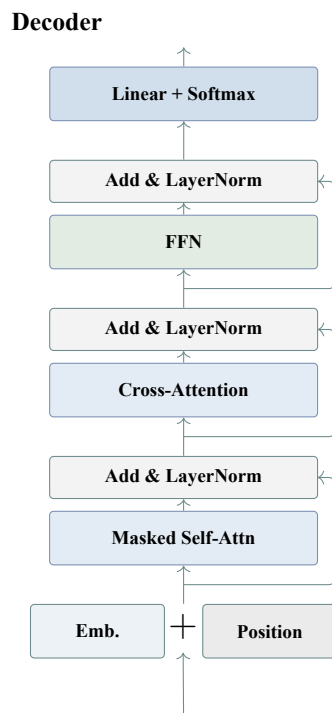


Figure 1.8: Transformer decoder layer architecture. The decoder contains masked self-attention for previous target tokens, cross-attention for reading encoder outputs, and a Feed-Forward Network (FFN), each wrapped with residual connections and Layer Normalization (LayerNorm).

At this point in the report, we can view the Transformer as an arrangement of embedding layers, attention layers, feed-forward functions, and encoder–decoder connections. This architecture accounts for why it performs well on sequence modelling tasks, since each position can refine its representation using information from the rest of the sequence. This expressive structure, however, is computationally demanding. The next point examines where this cost arises inside the Transformer architecture.

1.5.6 The Computational Bottleneck of Transformers

While powerful, the Transformer’s success comes at a significant computational cost, which forms a central motivation for the compression techniques explored in this report. The standard Transformer architecture replaces recurrence with self-attention, enabling strong parallelism and high performance across sequence modelling tasks [vaswani2017attention](#). However, this advantage comes with a major bottleneck: in full self-attention, each token attends to every other token in the sequence. For a sequence of length n , this requires an attention score matrix of size $n \times n$, making the attention component scale quadratically with sequence length in both computation and memory [beltagy2020longformer](#); [tay2020efficient](#).

This quadratic behaviour has direct practical implications. Doubling the sequence length from 512 to 1024 multiplies the attention matrix size by four, since $1024^2/512^2 = 4$. As a result, processing long documents or long-context inputs becomes substantially more expensive, which has motivated a large body of work on efficient Transformer variants and sparse or linear attention mechanisms [beltagy2020longformer](#); [tay2020efficient](#).

At the same time, modern language models benefit strongly from scaling model size, training data, and compute [kaplan2020scaling](#). Later scaling-law work further showed that model size and the number of training tokens should be scaled together under a fixed compute budget, highlighting the continuing importance of large-scale training [hoffmann2022training](#). This creates a tension between scale and deployment: larger models often achieve better performance, but practical inference is constrained by latency, memory, energy, and hardware availability. This tension motivates compression methods such as pruning, quantization, and knowledge distillation [han2015deep](#); [sanh2019distilbert](#).

The computational bottleneck defined above can be considered in several different ways. A Transformer may be expensive because of the size of its parameters, the memory required for activations, the number of arithmetic operations it performs, or the time required to run it on a specific hardware platform. Before discussing model efficiency in more detail, it is therefore useful to introduce the most common measures used to assess computational cost.

1.5.7 Hardware-Aware Efficiency Metrics

Compression is evaluated through several distinct efficiency metrics. Parameter count, memory footprint, arithmetic cost, latency, and throughput capture different aspects of model execution. Hardware-aware evaluation therefore requires separate treatment of storage cost, arithmetic cost, and runtime behavior [Sze2017EfficientDNN](#).

Parameter Count and Model Size

Let $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^L$ denote the parameters of a network. The total parameter count is

$$P = \sum_{l=1}^L (|W^{(l)}| + |b^{(l)}|). \quad (1.28)$$

If the parameters are stored with precision b_w bits, the static model footprint is

$$M_{\text{model}} = \frac{P b_w}{8} \text{ bytes}. \quad (1.29)$$

This follows from counting the stored elements and multiplying by the number of bytes used by each element `PyTorchNumel`; `PyTorchElementSize`. This quantity determines whether a model fits within device memory and how much parameter traffic must be sustained during inference. Quantization reduces b_w , pruning reduces the number of active parameters, and low-rank factorization reduces the parameterization itself.

For sparse models, storage depends on both the remaining weights and the associated meta-data. If P_{nz} denotes the number of non-zero weights and M_{meta} the storage needed for indices or masks, then

$$M_{\text{sparse}} = \frac{P_{\text{nz}} b_w}{8} + M_{\text{meta}}. \quad (1.30)$$

MACs, FLOPs, and Arithmetic Intensity

For a dense matrix multiplication $Y = AB$ with $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, the arithmetic cost is

$$\text{MACs}(A, B) = mkn, \quad \text{FLOPs}(A, B) \approx 2mkn. \quad (1.31)$$

The distinction is conventional: one multiply-accumulate corresponds to one MAC, while FLOP counting often treats the multiply and the addition as two floating-point operations **NVIDIAMatrixMultiplication**. In Transformer layers, the feed-forward subnetwork contributes large dense matrix products, while self-attention adds the sequence-dependent $O(n^2 d_{\text{model}})$ cost of score computation and value aggregation.

Arithmetic cost alone is insufficient for hardware analysis. A useful complementary quantity is the arithmetic intensity

$$I = \frac{\text{FLOPs}}{\text{Bytes Moved}}, \quad (1.32)$$

which measures how much computation is performed per byte transferred **Williams2009Roofline**; **NVIDIAMatrixMultiplication**. Low arithmetic intensity indicates a memory-bound workload, whereas high arithmetic intensity indicates a compute-bound workload.

Bandwidth, Latency, and Throughput

Memory bandwidth measures the rate at which data can be transferred between memory and compute units. If B_{move} bytes are transferred during an operation that takes time T , the effective bandwidth demand is

$$\text{BW}_{\text{eff}} = \frac{B_{\text{move}}}{T}. \quad (1.33)$$

This follows the standard effective-bandwidth definition used for GPU kernels **CUDABestPractices**. When parameter streaming and activation movement dominate execution, bandwidth becomes the primary runtime constraint.

Latency and throughput characterize runtime from complementary viewpoints. Latency is the time required to complete one inference request or one decoding step **Sze2017EfficientDNN**:

$$T_{\text{latency}} = T_{\text{compute}} + T_{\text{memory}} + T_{\text{overhead}}. \quad (1.34)$$

Throughput measures completed work per unit time, for example requests per second or generated tokens per second **Sze2017EfficientDNN**:

$$\text{Throughput} = \frac{N_{\text{outputs}}}{T}. \quad (1.35)$$

In large autoregressive models, latency and throughput often follow different scaling trends. Batching can increase throughput while single-request latency remains high. Parameter compression can reduce memory transfer while leaving runtime gains limited when execution is poorly aligned with the hardware schedule.

1.6 Genetic Algorithms and Multi-Objective Search

With the hardware-aware metrics introduced above, it becomes possible to compare model variants concretely. In particular, different designs may be compared with regard to prediction accuracy, parameter count, memory footprint, computational cost, memory bandwidth usage, and latency/throughput. In compression, these variants may correspond to different pruning masks, retained heads, feed-forward widths, removed layers, or layer-wise numerical precision.

Taken together, these choices form a large design space. Since enumerating all possible candidates is usually prohibitive, a search procedure is needed to explore this space and identify candidates that satisfy efficiency requirements while preserving prediction accuracy.

To manage this search, one can maintain a set of candidate designs instead of tracking a single design at a time. Each candidate corresponds to a possible compressed model. The candidates are evaluated, the more relevant ones are kept with higher probability, and new candidates are generated from existing ones through small changes or recombination. By repeating this process, the candidate set can move toward desirable regions of the design space.

A population-based search algorithm of this kind is known as a genetic algorithm. In this terminology, a candidate compressed model is an individual, the set of candidates is a population, and the evaluation measure is the fitness function. The following section introduces these notions together with encoding, selection, crossover, and mutation, then links them to compression design choices, multi-objective optimization, and NSGA-II [holland1975adaptation](#); [goldberg1989genetic](#); [deb2001multi](#).

1.6.1 Genetic Algorithm Fundamentals

A Genetic Algorithm (GA) is a population-based search method inspired by biological evolution. It maintains candidate solutions, represents them in a form that can be modified, evaluates their quality, and repeatedly generates new candidates through selection, crossover, and mutation [holland1975adaptation](#); [goldberg1989genetic](#); [michalewicz1996genetic](#).

Components of a Genetic Algorithm

The basic object manipulated by a GA is an individual, also called a chromosome. Each individual encodes one candidate solution to the problem. A population is then a set of such individuals evaluated during the same generation. The fitness function assigns a quality score to each candidate, and the genetic operators define how new candidates are produced from existing ones [goldberg1989genetic](#); [michalewicz1996genetic](#). These definitions establish what

the algorithm manipulates; the next issue is how a candidate is written as a chromosome.

In model compression, an individual may represent a compressed architecture. For example, consider the chromosome

$$\mathbf{c} = [1, 0, 1, 1 \mid 2048, 1024 \mid 8, 4, 4, 8], \quad (1.36)$$

where the first block indicates which attention heads are retained, the second block gives selected feed-forward widths, and the final block assigns layer-wise bit-widths. The fitness value then measures how good this candidate is with respect to the desired objective, such as validation accuracy, parameter count, memory footprint, or latency.

Encoding Methods for Individuals

The encoding determines how a candidate solution is written as a chromosome and therefore what kind of changes the algorithm can apply [michalewicz1996genetic](#). A **binary encoding** represents each gene with one bit, so every decision has two possible states. In compression, a bit value of 1 may indicate that a neuron, head, or channel is retained, while 0 indicates that it is removed. An **integer encoding** represents each gene with an integer selected from a finite set of choices. This is useful when a layer must choose one option among several possible bit-widths, such as $\{2, 4, 8, 16\}$.

The choice of encoding is important because it determines what the genetic operators can change. A poor encoding may create invalid candidates or make useful solutions hard to reach. A good encoding reflects the structure of the compression problem while keeping the search space manageable. Once this representation is fixed, the algorithm can operate on a population across generations.

Operation of Genetic Algorithms

A GA begins by initializing a population, either randomly or from existing candidate designs. Each individual is evaluated using the fitness function. The algorithm then selects promising candidates, recombines their encoded decisions through crossover, and applies mutation to preserve exploration [goldberg1989genetic](#). The resulting candidates form the next population, and this cycle is repeated until a stopping criterion is reached, such as a maximum number of generations, convergence of the population, or an evaluation budget. The three operators below define how this update takes place.

Selection Operator. Selection chooses which individuals are allowed to reproduce. Better candidates should have a higher chance of being selected, but weaker candidates are not always discarded immediately because they may contain useful partial structures. Common strategies include roulette-wheel selection, rank-based selection, and tournament selection [baker1985adaptive](#); [miller1995genetic](#). The selected candidates then serve as parents for crossover.

Crossover Operator. Crossover combines information from two or more parent individuals to produce offspring. In binary or integer encodings, this may exchange segments of chromosomes between parents. In uniform crossover, each gene can be inherited independently from either parent **syswerda1989uniform**. For compression, crossover can combine useful pruning patterns or precision assignments discovered in different candidates. Since crossover reuses information already present in the population, mutation is used to introduce local changes.

Mutation Operator. Mutation introduces small random changes into an individual. It prevents the population from collapsing too early around one region of the search space and allows new structures to appear **back1993optimal**. In a compression chromosome, mutation may flip a keep-or-remove bit, change the bit-width of a layer, or adjust a pruning threshold.

With these mechanics defined, the compression problem can now be written as a genetic search problem.

1.6.2 Compression as a Search Problem

With the GA terminology in place, compression can be described as a search over encoded model variants. Let $\mathbf{x}$ denote a compression configuration, such as a pruning mask, a vector of layer-wise bit-widths, or a set of structural choices. Evaluating $\mathbf{x}$ produces a compressed model whose quality depends on both predictive performance and deployment cost.

A single scalar score is sometimes used, for example by combining accuracy loss and model size into one weighted objective. However, this requires choosing the trade-off weights before the search begins. In practice, compression often requires a set of alternatives: one model may preserve more accuracy, another may use less memory, and another may reduce latency more aggressively. This need for several valid trade-offs leads to multi-objective optimization and Pareto dominance.

1.6.3 Multi-Objective Optimization and Pareto Dominance

Problem Formulation

Multi-objective optimization formalizes the comparison introduced above by replacing one scalar objective with an objective vector. A Multi-Objective Optimization Problem (MOP) minimizes a vector of objective functions **deb2001multi**:

$$\begin{aligned} & \underset{\mathbf{x}}{\text{minimize}} && \mathbf{f}(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x})]^T \\ & \text{subject to} && g_j(\mathbf{x}) \leq 0, \quad j = 1, \dots, J \\ & && x_i^{(L)} \leq x_i \leq x_i^{(U)}, \quad i = 1, \dots, n. \end{aligned} \tag{1.37}$$

Here, $\mathbf{x} \in \mathbb{R}^n$ is the decision vector and $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^M$ maps each candidate to the objective space. In compression, the objectives may include validation error, number of parameters, model footprint, FLOPs, latency, or energy consumption.

Since each candidate is represented by several objective values, candidates cannot always be ranked by a single numerical score. The comparison is therefore based on whether one candidate improves at least one objective without worsening the others. This idea is formalized by Pareto dominance.

Pareto Dominance

Definition (Pareto Dominance). A vector $\mathbf{u}$ dominates $\mathbf{v}$, denoted $\mathbf{u} \prec \mathbf{v}$, if and only if:

$$\forall i : u_i \leq v_i \quad \wedge \quad \exists j : u_j < v_j. \quad (1.38)$$

Here, $\mathbf{u}$ and $\mathbf{v}$ are objective vectors, and u_i and v_i are their values on objective i . Thus, $\mathbf{u}$ is no worse than $\mathbf{v}$ in every objective and strictly better in at least one objective.

Definition (Pareto Optimal Set). The Pareto optimal set in decision space is:

$$PS^* = \{\mathbf{x} \in \Omega \mid \nexists \mathbf{y} \in \Omega : \mathbf{f}(\mathbf{y}) \prec \mathbf{f}(\mathbf{x})\}. \quad (1.39)$$

Here, Ω is the feasible search space, $\mathbf{x}$ and $\mathbf{y}$ are candidate solutions, and $\mathbf{f}(\mathbf{y}) \prec \mathbf{f}(\mathbf{x})$ means that candidate $\mathbf{y}$ dominates candidate $\mathbf{x}$ in objective space. Therefore, PS^* contains the feasible candidates that are not dominated by any other feasible candidate.

The corresponding Pareto front in objective space is:

$$PF^* = \{\mathbf{f}(\mathbf{x}) \mid \mathbf{x} \in PS^*\}. \quad (1.40)$$

Here, PF^* is the image of the Pareto optimal set under the objective function $\mathbf{f}$. For model compression, the Pareto front gives a set of non-dominated compressed models. A user can then choose the model that best matches the deployment constraint, rather than relying on a single fixed trade-off. Pareto dominance provides the ranking principle, and NSGA-II combines this ranking with the population-based mechanics of genetic algorithms.

1.6.4 NSGA-II

NSGA-II, the Non-dominated Sorting Genetic Algorithm II, is a widely used multi-objective genetic algorithm that connects GA search with Pareto-based ranking **deb2002fast**. It keeps the population-based structure of a GA, but replaces scalar fitness ranking with Pareto-based selection. This makes it suitable for compression problems where accuracy, memory, and runtime objectives must be considered together.

The first key mechanism is non-dominated sorting. Candidates are grouped into fronts: the first front contains candidates that are not dominated by any other candidate, the second front

contains candidates dominated only by members of the first front, and so on. This gives priority to candidates closer to the Pareto front.

The second mechanism is crowding distance. Within the same front, NSGA-II favors candidates located in less crowded regions of the objective space. This preserves diversity and prevents the search from returning many nearly identical compression configurations.

The third mechanism is elitist preservation. Parents and offspring are merged, ranked by non-dominated sorting, and filtered using crowding distance when a front cannot fit entirely into the next generation. As a result, strong candidates are preserved while the population continues exploring new trade-offs.

In the context of this report, NSGA-II provides the bridge between genetic search and hardware-aware compression. The chromosome defines the compression decision, the objectives measure predictive and deployment cost, and the algorithm searches for a Pareto set of compressed Transformer variants.

This completes the foundations needed for the proposed contribution: the model can be measured through hardware-aware metrics, encoded as a candidate design, and searched under several objectives at once.

1.7 Conclusion

This chapter established the mathematical and architectural foundations required for the study of compression methods in the following chapters. It introduced the neuron model, activation functions, multilayer perceptrons, learning objectives, gradient-based optimization, and back-propagation, then surveyed representative neural architectures, including recurrent, autoencoding, and Transformer-based models.

The discussion of hardware-aware efficiency formalized the principal deployment metrics that later compression methods seek to improve, including parameter count, memory footprint, arithmetic cost, bandwidth demand, latency, and throughput. The chapter then introduced multi-objective genetic algorithms as a search framework for problems involving competing objectives and discrete design variables. Together, these foundations motivate the compression formulations developed later in the report, where predictive quality must be balanced against memory, runtime, and hardware constraints.

Within this conceptual frame, pruning emerges as a natural next step. Once neural computation is expressed through parameterized operators, decomposed into identifiable architectural units, and evaluated through resource-sensitive metrics, the central pruning question becomes precise: which parts of the network may be removed with limited functional loss, and under what deployment constraints does that removal become meaningful.

Chapter 2

Neural Networks Compression

2.1 Introduction

Model compression encompasses a family of techniques aimed at reducing the computational and memory demands of neural networks while preserving, as much as possible, their predictive performance. These methods are typically applied at the level of layers, submodules, or parameter groups and introduce explicit trade-offs between efficiency metrics and model accuracy. Key objectives include reducing model size, lowering inference latency, decreasing memory bandwidth usage, and accelerating both training and deployment, particularly in resource-constrained environments.

A notable characteristic of modern deep learning models is their heavy overparameterization. In such regimes, compression does not merely remove redundancy but can also act as an implicit form of regularization, mitigating overfitting and occasionally improving generalization performance. This phenomenon has been observed across a variety of architectures, especially in large-scale transformer-based models.

Among the most widely adopted compression strategies are *pruning*, *quantization*, *low-rank factorization*, and *knowledge distillation*. These techniques are not mutually exclusive and are frequently combined in practice. Hybrid compression pipelines leverage their complementary strengths to achieve higher compression ratios and better efficiency–accuracy trade-offs than any single method alone.

2.2 Quantization

Quantization can be defined as the function of mapping continuous values from a high-precision space to a finite low-precision representation. When it come to neural networks, this means the act of reducing the bit-width of weights, activations, or, in training settings, gradients [gholami2022survey](#). Since training requires better stability than inference, it is common to use high-precision floating-point representations like FP32 or mixed-precision formats [krishnamoorthi2018quantizing](#). During inference, however, lower bits can still express

the required precision to keep accuracy high enough. For this reason, it is effective to use quantization as a way to decrease model size and memory movement [han2016ie](#).

2.2.1 Mathematical Formulation

The aforementioned high-precision to low-precision reduction can be described using a quantization operator. Given an input $x \in \mathbb{R}$, quantization maps x to a finite set of values defined by the bit-width k , yielding 2^k possible integer levels.

In affine quantization, the real-valued range of interest is first restricted to a clipping interval $[\alpha, \beta]$, so that values outside this interval are clipped to one of the bounds, while values inside are represented using an integer code x_q . For an unsigned k -bit integer domain, the representable integer range is

$$q_{\min} = 0, \quad q_{\max} = 2^k - 1.$$

The spacing between two adjacent quantization levels is controlled by the scale s , defined as

$$s = \frac{\beta - \alpha}{q_{\max} - q_{\min}} = \frac{\beta - \alpha}{2^k - 1}. \quad (2.1)$$

Since the real interval does not have to begin at zero, an integer zero-point z is chosen such that real zero corresponds to an integer code. In the unsigned case,

$$z = \text{round}\left(q_{\min} - \frac{\alpha}{s}\right) = \text{round}\left(-\frac{\alpha}{s}\right). \quad (2.2)$$

Given these parameters, the real value x is mapped to an integer by

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right), \quad (2.3)$$

where clipping ensures that the resulting integer remains within the available k -bit range. The approximate real value is then recovered through dequantization:

$$\hat{x} = s(x_q - z). \quad (2.4)$$

The difference between the original value and its reconstructed approximation,

$$e_q = x - \hat{x}, \quad (2.5)$$

is the quantization error. Quantizing a neural network introduces this error into both the weights and activations during the forward pass, so the network behavior depends not only on the bit-width used but also on the chosen clipping range, scale, and zero-point.

2.2.2 Quantization Granularity and Schemes

Quantization strategies are categorized by how the scaling factors are computed and applied (granularity) and how the quantization parameters are learned (regime).

Symmetric vs. Asymmetric Quantization

- **Asymmetric Quantization:** Asymmetric quantization permits the quantization range to match the data distribution. That implies that no longer should the quantization range be symmetric about zero and therefore we can more efficiently encode skewed or one-sided data ranges (useful for activations and after ReLU type functions when values are primarily non-negative). The main disadvantage is the need for a zero-point, adding a slight correction term in computation.
- **Symmetric Quantization:** With symmetric quantization, the range is symmetric around zero. Both negative and positive sides are treated equally, and the zero-point is typically zero. This yields a simpler representation which can be implemented efficiently especially when it comes with weights. But the problem is that if the data range is not zero-centric, some of the quantization levels can be unused.

Methodologies: PTQ and QAT

- **Post-Training Quantization (PTQ):** The model is quantized after training is complete without updating the weights via backpropagation. Calibration data is used to compute optimal clipping thresholds (α, β) to minimize information loss (e.g., minimizing KL-divergence between FP32 and INT8 distributions) [liu-et al-2023-llm](#).
- **Quantization-Aware Training (QAT):** The quantization error is simulated during the training process (typically the fine-tuning phase). Since the rounding operation is non-differentiable, QAT employs "Straight-Through Estimators" (STE) to approximate gradients during backpropagation, allowing the network to adapt its weights to the loss of precision [jacob2018quantization](#).

Quantization schemes also differ in how bin widths are distributed across the input range. In **uniform quantization**, the input domain is divided into L equal-width intervals of size $\Delta x = 1/L$, assigning each input $x \in [b_i, b_{i+1})$ to the representative value $r_i = (i + 1/2)/L$. This uniform spacing simplifies hardware implementation but allocates the same resolution to all magnitudes regardless of where values actually concentrate. In **non-uniform quantization**, bin widths vary across the range through a companding function $g(x)$: narrower bins are placed where values are denser (typically near zero) and wider bins where they are sparse, reducing overall quantization error for skewed distributions. Figure 2.1 illustrates both schemes side by side, contrasting the constant step size of the uniform case against the variable spacing produced by $g(x) = \ln(1 + \mu x) / \ln(1 + \mu)$ with $\mu = 6$. Table 2.1 summarizes the standard numerical formats encountered in neural network quantization.

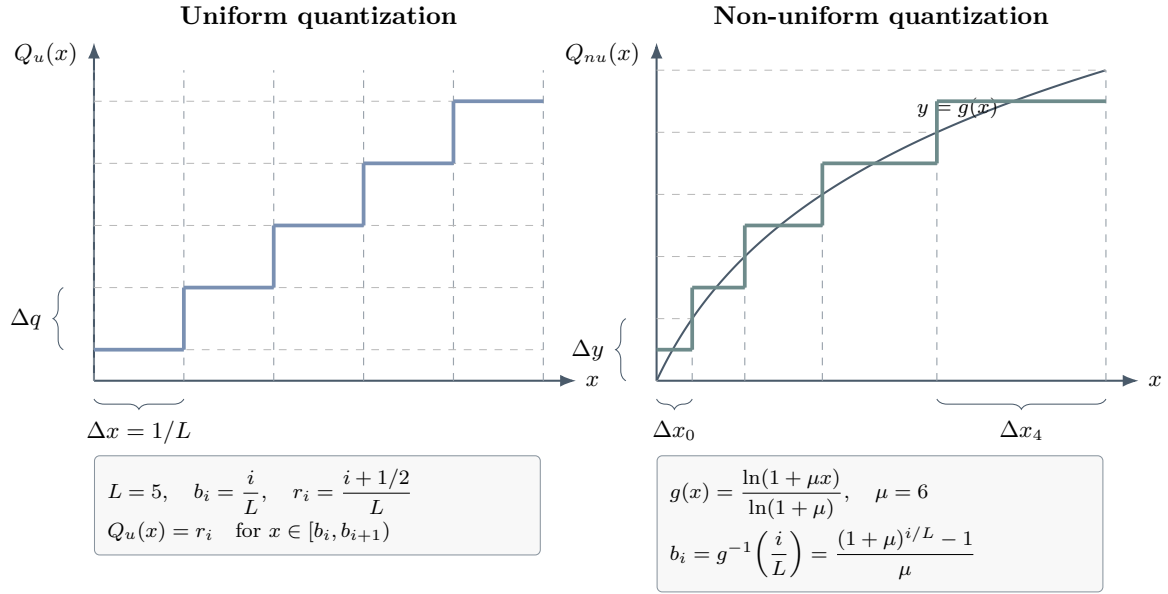


Figure 2.1: Uniform quantization (left) divides the input range into L equal-width bins, assigning each interval $[b_i, b_{i+1})$ to a fixed representative r_i . Non-uniform quantization (right) applies a companding function $g(x)$ to warp the input space, allocating finer resolution at lower magnitudes where values concentrate and coarser bins at higher magnitudes.

Table 2.1: Standard numerical formats commonly used in neural network quantization [ieee754_2019](#); [kalamkar2019bfloat16](#); [jacob2018quantization](#); [krishnamoorthi2018quantizing](#); [lin2024awq](#). The integer ranges correspond to signed stored integer values before dequantization; the represented real-valued range depends on the quantization scale and, when used, the zero-point.

Format	Bit-width	Type	Range	Typical application
FP32	32	Floating point	$\sim \pm 10^{38}$	Full-precision training baseline
BF16	16	Floating point	$\sim \pm 10^{38}$	Mixed-precision training
INT8	8	Integer	$[-128, 127]$	Standard quantized inference
INT4	4	Integer	$[-8, 7]$	Aggressive low-bit compression

2.3 Neural Network Pruning

Pruning is the process of reducing the size of a neural network by eliminating either connections or components that are responsible for outputting only a small proportion of the overall model prediction. The principle behind pruning relies on the observation that overparameterized neural networks (especially deep ones) generally do not require all the units and connections in order to achieve a good performance. Pruning can therefore reduce the storage space and computational cost to produce a less resource-intensive network.

2.3.1 Formal Problem Formulation

Given a parameterized network $f(\cdot; \Theta)$ whose P parameters are stored in $\Theta \in \mathbb{R}^P$, a binary mask $M \in \{0, 1\}^P$ is chosen such that those parameters selected to be kept (whose mask value is 1) can be retrieved and those not (mask value 0) excluded from further calculation by computing $f(x; M \odot \Theta)$, which zeros out parameters for which $M_i = 0$ by elementwise product.

The goal is then to find the mask M and the remaining weights Θ that keep the loss $\mathcal{L}$ as low as possible while meeting a budget on how many parameters are retained:

$$\min_{\Theta, M} \mathcal{L}(M \odot \Theta) \quad \text{subject to} \quad \frac{\|M\|_0}{P} \leq \kappa, \quad (2.6)$$

where $\|M\|_0$ counts the nonzero entries of M and $\kappa \in (0, 1]$ is the target density. In practice this hard constraint is often replaced by a regularised objective that avoids having to fix κ in advance:

$$\min_{\Theta, M} \mathcal{L}(M \odot \Theta) + \lambda \Omega(M), \quad (2.7)$$

where λ controls how aggressively compression is penalised and $\Omega(M)$ can capture parameter count, memory footprint, or latency depending on the deployment target. This regularised form folds the budget into the objective instead of enforcing it as a hard constraint (Eq. (2.7)).

Finding an optimal pruning solution is generally NP-hard, since the search space grows exponentially with the number of parameters [dong2017learning](#). Practical methods thus resort to approximate measures which evaluate how pruning a parameter or structure affects performance without considering all possible masks. Such measures can be obtained by approximating the change in training loss due to pruning, commonly through a Taylor expansion [molchanov2017variational](#).

2.3.2 Structural Forms of Pruning

The binary mask in Eq. (2.6) indicates which elements in the network need to be removed, but it does not indicate the pattern of removal. The zeroes in M may show up as discrete individual parameters, as repetitive local structures, or as entire computational blocks. This leads to the three typical types of pruning: unstructured, semi-structured and structured pruning.

- **Unstructured pruning** uses the mask at the level of individual weights. Each weight can be kept or removed independently, so the tensor dimensions remain unchanged while some entries are set to zero.
- **Semi-structured pruning** partitions a weight tensor into small blocks and applies the same rule to each block. In an $N : M$ pattern, every block of M weights keeps N entries and removes the remaining $M - N$. For example, in a $2 : 4$ pattern, a group of four weights (w_1, w_2, w_3, w_4) may receive a mask such as $(1, 0, 1, 0)$, meaning that w_1 and w_3 are kept while w_2 and w_4 are removed. The same counting rule is then repeated across the tensor, block after block.

- **Structured pruning** applies the mask to complete computational units. These units may be neurons, groups of neurons, hidden dimensions, or full layers. Removing such units also removes the parameters attached to them, which naturally changes the dimensions of the affected parts of the network.

Thus, after the pruning objective has been written in terms of a mask, the next distinction is the form taken by that mask inside the network: isolated entries, repeated local groups, or complete units, as illustrated in Figure 2.2.

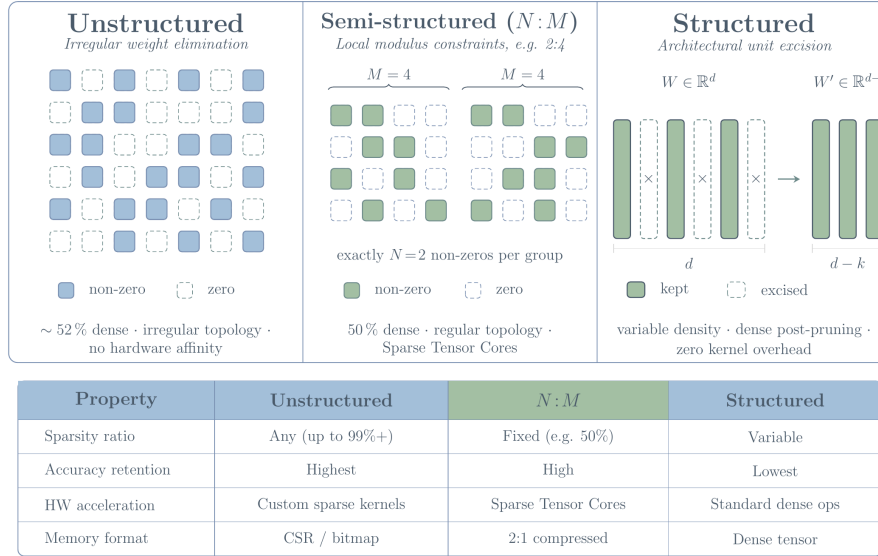


Figure 2.2: A dense weight matrix can be pruned into unstructured sparsity, semi-structured 2:4 sparsity, or structured row, column, and block sparsity. The distinction is important at deployment time because increasingly regular sparsity patterns align more directly with accelerator execution primitives.

2.4 Low-Rank Factorization

Low-rank factorization is a method for compressing high dimensional weight matrices by factorizing them into two or more lower dimensional matrices. This leverages the fact that a lot of weight matrices found in learned neural networks tend to be of a low rank, implying that the encoded information is residing on a lower dimensional manifold within the high dimensional parameter space **li2018measuring**. By explicitly factorizing these matrices, the parameter space and the cost of matrix multiplications can be lowered.

2.4.1 Mathematical Formulation

Consider a weight matrix $W \in \mathbb{R}^{m \times n}$ of a pre-trained layer. We seek to approximate W using two factor matrices $A \in \mathbb{R}^{m \times r}$ and $B \in \mathbb{R}^{r \times n}$, where r is the rank and $r \ll \min(m, n)$. The approximation is given by:

$$W \approx AB. \quad (2.8)$$

The optimization objective is to minimize the reconstruction error, typically measured by the Frobenius norm:

$$\min_{A,B} \|W - AB\|_F^2. \quad (2.9)$$

Ideally, the optimal solution is provided by the truncated Singular Value Decomposition (SVD). If $W = U\Sigma V^T$, the best rank- r approximation is $U_r \Sigma_r V_r^T$, where Σ_r contains the top r singular values. In practice, A and B can be learned directly via gradient descent, or initialized via SVD and then fine-tuned.

The compression ratio achieved is defined as:

$$C_{\text{ratio}} = \frac{mn}{r(m+n)}, \quad (2.10)$$

where mn is the parameter count of the original matrix and $r(m+n)$ is the total count of the two factor matrices. For square matrices where $m = n$, the complexity reduces from $O(n^2)$ to $O(rn)$, offering linear scaling with respect to dimensionality when r is constant.

2.4.2 Applications

As described in the formulation above, substituting a network's weight matrices with their low-rank approximations (Eq. 2.8) directly yields a compressed model **eckart1936approximation**; **xue2013restructuring**; **sainath2013lowrank**. The approach works best in layers where the weight matrix is already close to low-rank in practice, meaning a small number of components is enough to reconstruct it accurately **eckart1936approximation**; **xue2013restructuring**.

Moreover, one of the most prominent modern uses of low-rank factorization is Parameter-Efficient Fine-Tuning (PEFT). In methods such as Low-Rank Adaptation (LoRA) **hu2021lora**, the pre-trained weights W_0 are frozen, and a trainable low-rank update ΔW is injected into the forward pass. For an input x , the layer output h becomes:

$$h = W_0 x + \Delta W x = W_0 x + ABx. \quad (2.11)$$

Here, A is typically initialized from a Gaussian distribution and B is initialized to zero, ensuring the training starts with the original model behavior. Figure 2.3 illustrates the decomposition geometrically. This allows for the adaptation of massive models by optimizing only a fraction of the parameters.

2.5 Knowledge Distillation

Knowledge Distillation (KD) is a model compression technique in which a lightweight student model is trained to imitate a larger, higher-performing teacher model. Rather than relying only on hard ground-truth labels, KD transfers knowledge through the teacher's soft probability distributions **hinton2015distilling**. These probabilities can encode useful structure in the data manifold, since similar incorrect-class probabilities may reflect semantic or visual relationships, providing the student with a richer supervisory signal.

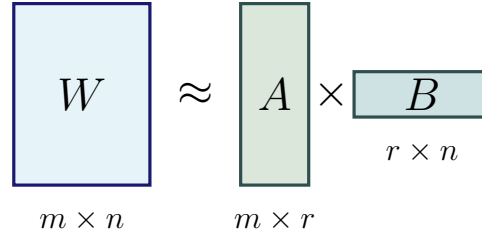


Figure 2.3: Schematic of Low-Rank Factorization. The dense matrix W is decomposed into low-rank components A and B , significantly reducing parameter count when $r \ll m, n$.

2.5.1 Mathematical Formulation

The standard distillation framework modifies the softmax function by introducing a temperature hyperparameter T . For a given input, let z_t and z_s be the logits of the teacher and student, respectively. The “softened” probability for class i , q_i , is computed as:

$$q_i(z; T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \quad (2.12)$$

A higher T produces a smoother probability distribution, emphasizing the relationships between non-target classes.

The training objective for the student is a linear combination of the standard task loss (typically Cross-Entropy against ground truth hard labels, y) and the distillation loss (typically Kullback-Leibler divergence between the student’s and teacher’s soft distributions). The total loss $\mathcal{L}_{\text{total}}$ is defined as:

$$\mathcal{L}_{\text{total}} = \alpha T^2 \mathcal{L}_{\text{KD}}(q(z_t; T), q(z_s; T)) + (1 - \alpha) \mathcal{L}_{\text{CE}}(y, q(z_s; 1)), \quad (2.13)$$

where α is a balancing coefficient and the T^2 term scales the gradients to account for the magnitude change caused by the temperature scaling.

2.5.2 Methodological Taxonomies

Distillation methodologies are categorized by the source of knowledge transferred, the training scheme employed, and the algorithmic mechanism used to align representations.

Knowledge Sources

The source determines what information is extracted from the teacher.

- **Logit-Based Distillation:** The student mimics the teacher’s final prediction distribution [hinton2015distilling](#). The loss L_{logit} is the KL divergence between softened logits. While simple, it offers no insight into intermediate reasoning.

- **Feature-Based Distillation:** The student aligns its intermediate activations with the teacher's [Romero2014](#). The loss L_{feature} minimizes the distance (e.g., L2) between transformed feature maps. This guides the student through the teacher's step-by-step reasoning.
- **Similarity-Based Distillation:** The student learns the *relationships* between data samples. The loss $L_{\text{similarity}}$ matches the angles or distances between pairs of features in the teacher's space to those in the student's space, capturing structural topology rather than absolute values.

Training Schemes

The scheme defines the interaction between teacher and student during optimization.

- **Offline Distillation:** A two-stage process where a high-capacity teacher is pre-trained and frozen. The student is trained subsequently to mimic the static teacher. This is effective for utilizing proprietary models but incurs high pre-training costs.
- **Online Distillation:** Teacher and student are trained simultaneously from scratch, learning collaboratively from each other and the ground truth. This eliminates pre-training but can be unstable as the teacher is not fully converged.
- **Self-Distillation:** The network acts as its own teacher. Knowledge is transferred from deeper layers to shallower layers, or from previous epochs to the current one, serving as a powerful regularization technique without external models.

Advanced Algorithms

Advanced mechanisms refine how the knowledge transfer is executed.

- **Attention-Based Distillation:** This method transfers spatial attention maps instead of full feature maps [Zagoruyko2016](#). It teaches the student where to focus and emphasizes saliency rather than raw activation values.
- **Contrastive Distillation:** Inspired by self-supervised learning, this pulls the student's representation of an input closer to the teacher's while pushing it away from negative samples (via InfoNCE loss). This improves structural alignment in the feature space.
- **Adaptive Distillation:** These methods dynamically adjust distillation parameters (such as loss weights or sample importance) during training. By focusing on harder examples or altering the teacher's influence over time, they optimize the transfer efficiency.

KD's major problems consist of what knowledge needs to be transferred, which algorithm to choose, and designing student and teacher architectures [mansourian2025comprehensivesurveyknowledgedistillation](#). Figure 2.4 presents a typical diagram of the teacher-student KD framework.

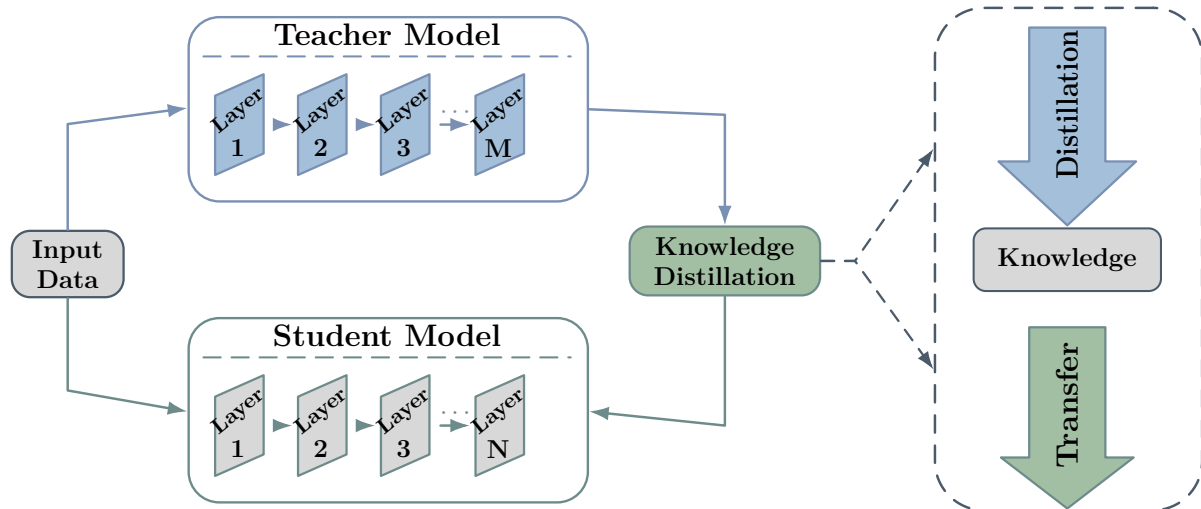


Figure 2.4: The Knowledge Distillation framework. The student minimizes a dual objective: the task loss $\mathcal{L}_{\text{task}}$ against hard labels and the distillation loss $\mathcal{L}_{KD}$ against the teacher’s soft probability targets. Inspired from mansourian2025comprehensiveurveyknowledgedistillation.

2.6 Conclusion

This chapter discussed theoretical basis of model compression. We have presented the mathematical formulations and the structural taxons of Quantization, Pruning, Low-Rank Factorization and Distillation. All these compression techniques make use of redundancies in neural networks in order to achieve reduction in memory usage and computational costs. This evidence also shows that over-parameterized models can be reduced in size without losing predictiveness.

While the fundamental principles remain constant, practical deployment necessitates adaptation to specific architectures. In our case, the massive scale and unique attention mechanisms of the Transformer present distinct challenges. Consequently, the next chapter, *Related Work*, investigates the empirical application of these concepts. It reviews the evolution of compression techniques specifically tailored for Large Language Models and Transformers and their optimization.

Chapter 3

Related Work

3.1 Introduction

Chapter 2 established the formal goal of quantization and pruning: to reduce bit-width, parameter counts, or computational costs, with bounded loss on the model’s outputs. What remains is to decide how those goals are achieved in the context of large Transformers, particularly given the computational bottleneck described in Section 1.5.6, where size, structure, and deployment constraints determine which techniques are feasible. This chapter covers the main approaches taken.

Post-training quantization is reviewed from the perspective of precision reduction under activation outliers: the systematic high-magnitude activations that appear in large language models make uniform low-bit quantization lossy, and the methods reviewed here differ in how they handle that constraint, through weight rotation [ashkboos2024quarot](#), salience-aware protection [lin2024awq](#), or second-order weight adjustment [frantar2022gptq](#).

Pruning is reviewed along two paired dimensions: the topology of the sparsity produced, and the signal used to determine which units to drop. Both dimensions bear on deployment, since methods with similar compression ratios can correspond to very different runtimes and recovery costs.

3.2 Related Work on Post-Training Quantization

Post-training quantization has received considerable attention for large language models, where memory bandwidth is the dominant deployment cost and reducing bit-width directly lowers that cost without retraining. The discussion here is restricted to post-training methods — those that determine the quantized representation under limited post-training resources, without an additional training or fine-tuning phase. Quantization-aware training operates in a fundamentally different regime: it requires simulating quantization throughout a training or fine-tuning process, and its computational cost makes it unsuitable for LLMs at scale. OmniQuant notes that many LLM quantization methods specifically prioritize training-free PTQ for this reason

shao2024omniquant. Computationally efficient QAT variants still report 41 hours on a single A100-80GB for a 2-bit LLaMA-2-70B run **tang2024efficientqat**, and standard QAT for 7B models requires more than 70GB of GPU memory **bondarenko2024lrqat**. LLM-QAT makes the distinction concrete: quantization-aware training for LLMs is not a quantization formula but a full data-free distillation procedure over the pretrained model **liu2023llm**, placing it in the same regime as recovery-based compression rather than post-training compression.

The methods reviewed here differ in how they control quantization error within the PTQ setting. The related work is organized by the dominant mechanism: rotation-based methods that redistribute activation energy before discretization, salience-based methods that identify and protect sensitive weights or channels, compensation-based methods that use second-order curvature to correct for quantization decisions locally, and optimization-based methods that tune quantization parameters directly on calibration data.

3.2.1 Problem Setting and Taxonomy

Post-train quantization (PTQ) reduces numerical precision after the full-precision phase. Conversion to lower bit-widths such as INT4 and INT8 lowers memory traffic and permits efficient integer arithmetic during inference. This makes PTQ particularly important for large language models, where deployment cost is often dominated by memory movement rather than arithmetic alone. Accuracy preservation, however, remains difficult. Moderate regimes typically focus on controlling layer-wise reconstruction error, whereas extreme regimes at 2–3 bits must additionally address severe sensitivity to activation outliers and discrete approximation error **frantar2022gptq**; **chee2024quip**; **ashkboos2024quarot**.

The taxonomy organizes methods along five axes, summarized in Figure 3.1. The **method family** axis identifies the dominant error-control principle: rotation-based, salience-based, compensation-based, optimization-based, or search-based; each family is elaborated in the subsections that follow. The **quantized object** axis distinguishes weight-only, joint weight-and-activation, KV-cache, and vector-group compression regimes. The **error-control** axis captures the mechanism used to bound approximation error, from redistribution and protection to learned transforms and end-to-end search. The **calibration signal** records the informational basis used to set quantization parameters: activation statistics, Hessian curvature, reconstruction loss, gradient feedback, or output distribution. The **deployment target** axis links each formulation to its intended hardware outcome, whether storage reduction, bandwidth savings, full low-bit execution, or budget-constrained mixed-precision inference.

Activation Outliers in Large Language Models

In the context of Large Language Models, quantization is fundamentally limited by the presence of *outliers*, namely values with magnitudes significantly larger (often 100×) than the median distribution. Unlike standard statistical noise, outliers in Transformers exhibit a **systematic** structure: they appear persistently in specific feature channels across all tokens, a phenomenon that emerges as model scale increases (typically > 6B parameters) **dettmers2022llmint8**.

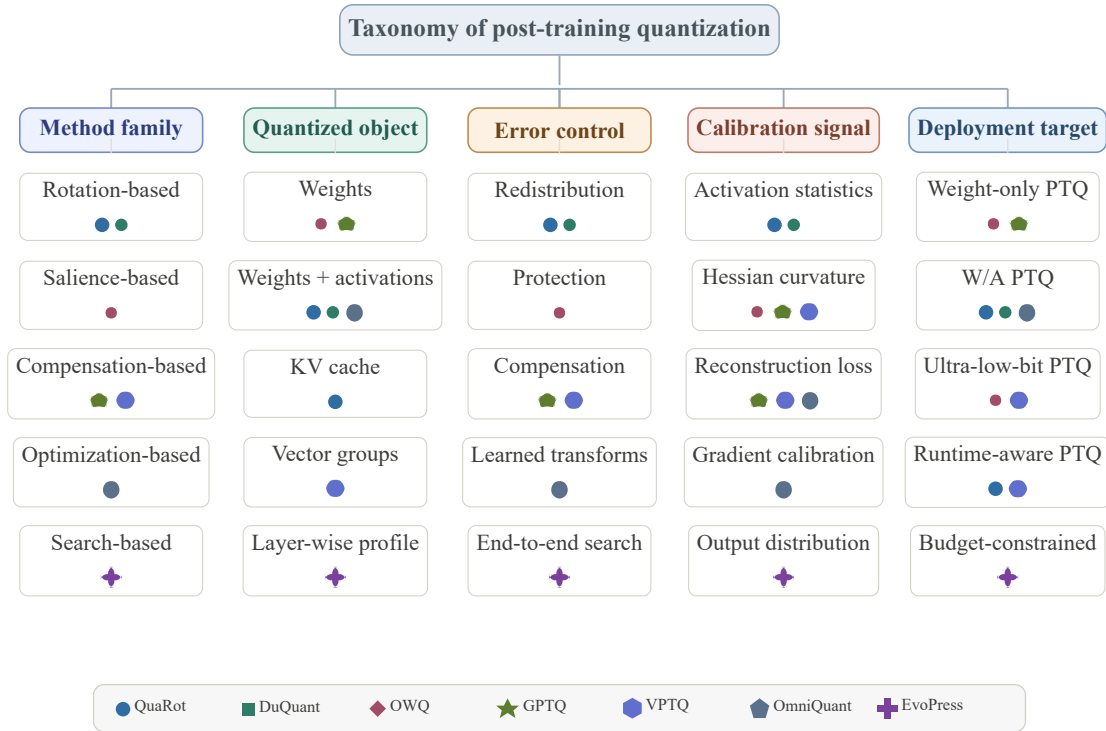


Figure 3.1: Taxonomy of post-training quantization methods reviewed in this section.

These outliers pose a critical challenge for uniform quantization. The scale s in the affine mapping of Section 2.2 (Eq. 2.1) is fixed by the range endpoints, so under symmetric MinMax it reduces to $s = \max(|X|)/(2^k - 1)$. A single massive outlier forces this range to stretch. Consequently, the vast majority of “normal” values are compressed into a minimal subset of available integers, causing catastrophic quantization error and performance collapse. The method families below differ mainly in how they prevent this single failure mode.

3.2.2 Rotation-Based Methods

The core challenge addressed by rotation-based methods is the presence of structured activation outliers in Transformer models. Prior work shows that Transformer activations can have high dynamic ranges and structured outliers, especially in residual streams, making them difficult to represent with low-bit fixed-point formats **bondarenko2021understanding**. Subsequent studies show that these outliers are often concentrated in specific channels and may be asymmetric across channels, so a small number of activation dimensions can dominate the quantization range **wei2023outlier+**. This behavior is also connected to architectural mechanisms inside Transformers: the LayerNorm scaling parameter has been identified as an amplifier of activation outliers, while attention and residual mechanisms have also been linked to the formation of strong outlier features **wei2022outlier**; **bondarenko2023quantizable**. When activations are quantized naively, these large-magnitude channels determine the clipping or scaling range, leaving most non-outlier values compressed into a much smaller portion of the available quantization levels.

Rotation-based approaches address this issue using structured orthogonal transformations. Since an orthogonal matrix preserves the ℓ_2 norm, rotating an activation vector does not change its total energy, but it redistributes that energy across coordinates. Methods such as QuaRot and SpinQuant exploit this property by inserting function-preserving rotations into Transformer blocks, reducing hidden-state outliers and improving low-bit quantization accuracy [ashkboos2024quarot](#); [liu2025spinquant](#). In this sense, rotation does not remove information; it changes the coordinate system so that extreme channel-wise concentration becomes less harmful for uniform quantization.

QuaRot [ashkboos2024quarot](#)

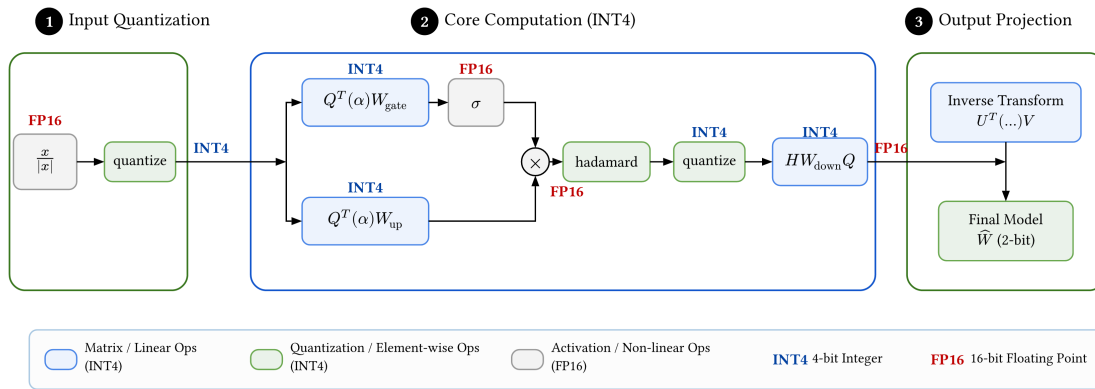


Figure 3.2: QuaRot applied to a LLaMa-style FFN. RMSNorm scaling (α) is absorbed into the weights.

Hidden state X is rotated by Q , canceled by $Q^\top$ in the first two weights. Weights and activations are INT4; matmul yields INT32, cast and scaled to FP16. While in FP16, a Hadamard transform is applied before quantizing and computing a modified down-proj, producing rotated output YQ .

[ashkboos2024quarot](#)

QuaRot introduces a quantization scheme based on randomized Hadamard rotations that eliminates activation outliers, enabling end-to-end 4-bit quantization of weights, activations, and the KV cache without performance degradation. Full 4-bit integer arithmetic is achieved by removing the root cause of quantization difficulty rather than working around it with mixed precision.

Methodology. The method relies on *computational invariance* grounded in a basic property of orthogonal matrices: for any orthogonal Q , $QQ^\top = I$, so inserting the pair $Q^\top Q$ between any two operations is an identity transformation. QuaRot exploits this to rotate activations by Q while multiplying $Q^\top$ into the next weight matrix, leaving the computation unchanged but spreading outlier energy uniformly across channels before quantization. RMSNorm (Root Mean Square Normalization), which normalizes each token by the root mean square of its features, is compatible with this transformation: $\text{RMSNorm}(X) = \text{RMSNorm}(XQ^\top)Q$. The RMSNorm

scaling parameters α are first multiplied into the adjacent weights, then a randomized Hadamard matrix Q is multiplied into all input projections:

$$W_{in} \leftarrow Q^\top \text{diag}(\alpha) W_{in}, \quad X \leftarrow XQ, \quad (3.1)$$

where X is the input activation matrix, W_{in} the weight matrices of input projections (query, key, value, and gate), and $\text{diag}(\alpha)$ a diagonal matrix formed from the RMSNorm channel scales. For the KV cache, per-head Hadamard matrices H_{d_h} are multiplied into the value and output projections to remove head-level outliers:

$$W_v \leftarrow W_v(I \otimes H_{d_h}), \quad W_{out} \leftarrow (I \otimes H_{d_h})W_{out}, \quad (3.2)$$

where I is the identity matrix, $\otimes$ denotes the Kronecker product that replicates H_{d_h} block-diagonally across all heads, and d_h is the per-head dimension. Within the FFN, a Hadamard H is applied during inference between the up- and down-projections and merged into the down-projection as $W_{down} \leftarrow HW_{down}$. With outliers redistributed, weights are quantized to INT4 via GPTQ or RTN, activations are quantized per-token symmetrically during inference, and the KV cache uses asymmetric group-wise quantization.

Results. QuaRot yields up to $3.33\times$ prefill speedup and $3.89\times$ decoding memory savings on Llama-2-70B, with the 4-bit model retaining 99% zero-shot performance and a perplexity increase of only 0.47 on WikiText-2.

QuIP chee2024quip

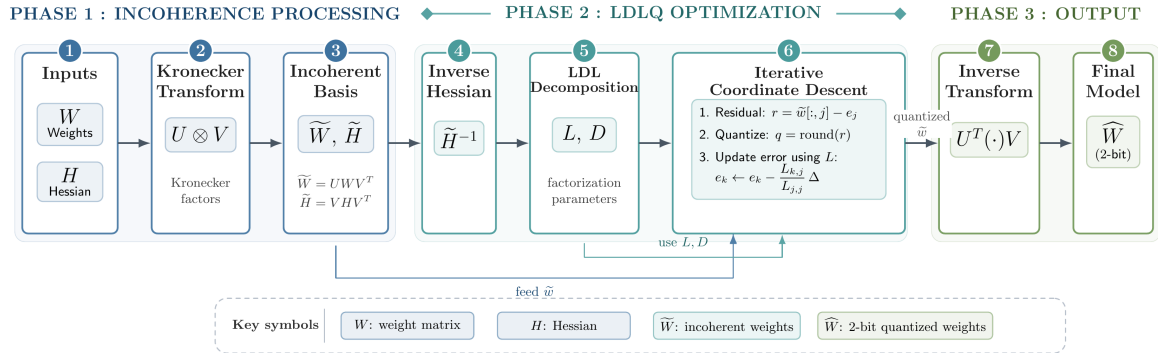


Figure 3.3: QuIP Methodology. The approach tackles the “proxy gap” in 2-bit quantization by transforming the Hessian and weights into an incoherent basis using Kronecker-factored orthogonal matrices (U and V), then employing LDLQ, a coordinate-descent algorithm using the LDL^T decomposition of the inverse Hessian, to quantize weights while correcting errors in remaining full-precision weights. **chee2024quip**

QuIP targets the “proxy gap” at 2-bit precision: the layer-wise quadratic proxy minimized by GPTQ correlates well with task loss at 4 bits but breaks down at 2 bits. The cause is that the LLM

Hessian is coherent, with a few large eigenvalues dominating, so greedy quantization aligns error vectors with those principal directions and produces $O(d)$ worst-case error. QuIP addresses this by rotating weights and Hessian into an incoherent basis before quantization.

Methodology. Random orthogonal matrices $U \in \mathbb{R}^{d_{\text{out}} \times d_{\text{out}}}$ and $V \in \mathbb{R}^{d_{\text{in}} \times d_{\text{in}}}$ are applied as:

$$\tilde{W} = UWV^\top, \quad \tilde{H} = VHV^\top, \quad (3.3)$$

where V spreads the Hessian eigenvalue spectrum so that no single direction dominates the quantization loss, U flattens weight magnitude distributions, and both matrices are Kronecker products of Hadamard matrices, keeping the transform cost $O(d \log d)$ rather than $O(d^2)$.

Quantization of the transformed weights $\tilde{W}$ then proceeds via LDLQ (LDL[⊤] Quantization), a coordinate-descent algorithm built on the $LDL^\top$ factorization of $\tilde{H}^{-1}$, where L is lower-triangular and D is diagonal. After rounding weight j and collecting error $\delta_j = \tilde{w}_j - \hat{\tilde{w}}_j$, all future weights $k > j$ are corrected:

$$\tilde{w}_k \leftarrow \tilde{w}_k - \frac{L_{kj}}{L_{jj}} \delta_j, \quad (3.4)$$

where L_{kj} and L_{jj} are entries of L and δ_j is the scalar rounding error at column j . Operating on $\tilde{H}^{-1}$ rather than H prevents outlier columns from propagating large corrections to later weights. Incoherence guarantees the proxy loss decreases at rate $O(1/n)$ in block size n , so error stays bounded rather than diverging as n grows.

Algorithm Overview

QuIP quantizes each weight matrix in three steps and does not retrain the model. First, a pre-processing step rotates both the weight matrix and its Hessian into an incoherent basis by multiplying them with the random orthogonal matrices U and V (Eq. 3.3); in this basis the weight magnitudes are flattened and no single Hessian direction dominates, which is what makes rounding survive at 2 bits. Second, the rotated weights are quantized column by column with LDLQ: each column is rounded to the grid, and its rounding error is passed to the columns not yet quantized through the triangular factor L of the inverse Hessian, the same error-passing idea used by GPTQ (Eq. 3.4). Third, a post-processing step multiplies the quantized weights by $U^\top$ and V to rotate them back into the original basis, so the stored low-bit matrix can be used directly at inference. Because U and V are products of small Hadamard matrices generated from a fixed random seed, they are cheap to apply and need not be stored.

Results. QuIP was the first method to produce non-degenerate output on LLaMA-2-70B at 2-bit precision, where GPTQ collapses entirely, and outperforms all prior 2-bit baselines on WikiText-2 perplexity across the LLaMA-1 and LLaMA-2 families.

3.2.3 Saliency-Based Methods

Rotation-based methods reshape the activations so that one uniform grid fits every channel. The next family leaves the grid untouched and asks a different question: does every weight really

deserve the same number of bits? The fundamental challenge addressed by salience-based methods is the heterogeneous importance of weights within neural networks. Not all parameters contribute equally to model output. Empirical analysis reveals that approximately 1% of weights in Large Language Models are disproportionately critical for performance preservation, while the vast majority exhibit high redundancy. Uniform quantization schemes that apply identical bit-widths across all parameters fail to exploit this structure. They either misallocate precious bits on unimportant weights or catastrophically degrade salient features.

Salience-based approaches address this challenge through precision allocation according to importance. The core principle involves the preservation of high-impact weights with higher precision alongside aggressive quantization of low-impact parameters to minimal bit-widths. The critical technical question becomes: how to accurately measure “importance” in a post-train setting without access to gradients or optimization dynamics? Different methods answer this question differently; some rely on activation magnitude analysis, while others utilize Hessian-based sensitivity measures. However, all share the objective to align bit allocation with functional impact.

OWQ lee2024owq

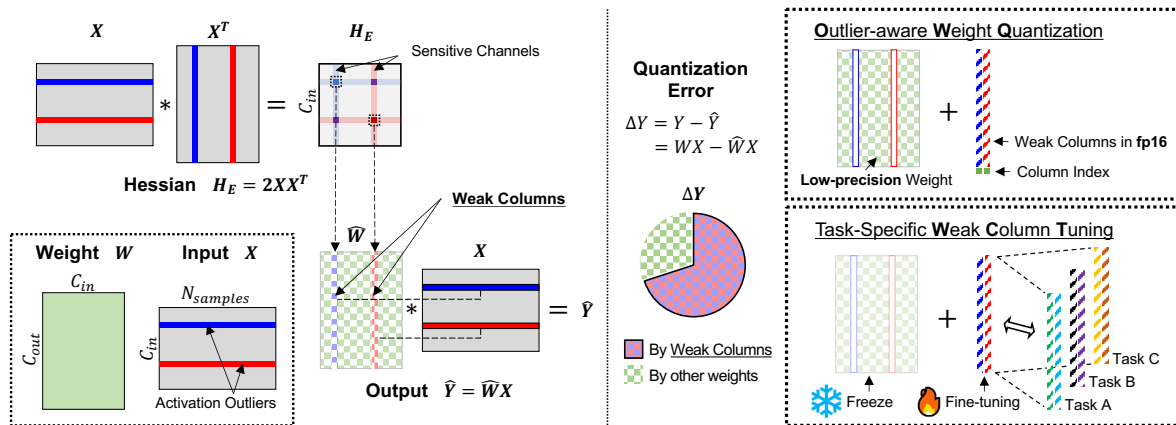


Figure 3.4: Overview of OWQ (Outlier-Aware Weight Quantization). The method identifies “weak columns” in weight matrix W corresponding to outlier activation channels. These columns are segregated into sparse FP16 matrix W_{weak} , while remaining “strong columns” are quantized to low precision using GPTQ. lee2024owq

OWQ addresses the amplification problem inherent in quantizing weights that interact with outlier activations. Since output error is defined by $\Delta Y = \Delta W X$, when X contains the outlier channels described in Section 3.2.1 (magnitudes around $100\times$ larger than surrounding features), even small weight quantization errors ΔW in the corresponding columns are amplified dramatically. Standard methods either clip activations (permanently destroying information needed for difficult reasoning) or apply uniform quantization (causing performance collapse). OWQ provides a principled mixed-precision solution based on Hessian sensitivity analysis.

Methodology. OWQ formulates weight importance through the expected output error variance:

$$\min_{\hat{\mathbf{W}}} \mathbb{E} \left[\|(\mathbf{W} - \hat{\mathbf{W}})\mathbf{X}\|_2^2 \right] = \text{Tr} \left((\mathbf{W} - \hat{\mathbf{W}})^\top (\mathbf{W} - \hat{\mathbf{W}}) \mathbb{E}[\mathbf{X}\mathbf{X}^\top] \right) \quad (3.5)$$

The term $\mathbb{E}[\mathbf{X}\mathbf{X}^\top]$ forms the empirical Hessian $\mathbf{H}$. Error contribution of the i -th weight column is proportional to $\mathbf{H}_{ii}$ (the aggregate power of the i -th input feature):

- **Weak Columns:** Large $\mathbf{H}_{ii}$ indicates weights multiplying outlier activations; quantizing them causes high error
- **Strong Columns:** Small $\mathbf{H}_{ii}$ allows aggressive quantization with minimal impact

Algorithm Overview

OWQ first decides which weight columns are too sensitive to quantize. It ranks the columns by their Hessian diagonal $\mathbf{H}_{ii}$, which measures how strongly the layer output reacts to error in column i , and marks the top few (about 1%, set by the memory budget) as weak columns. The matrix is then split into two parts that sum to the original, $\mathbf{W} = \mathbf{W}_{\text{quant}} + \mathbf{W}_{\text{weak}}$: the weak columns are kept at full FP16 precision in $\mathbf{W}_{\text{weak}}$ (zero elsewhere), and every remaining column is quantized to the target low bit-width, for example 3-bit, with GPTQ. At inference the two parts are computed separately and summed, so the handful of sensitive columns never pass through the low-bit grid:

$$\mathbf{Y} = \text{dequant}(\hat{\mathbf{W}}_{\text{quant}})\mathbf{X} + \mathbf{W}_{\text{weak}}[\cdot, S_{\text{weak}}]\mathbf{X}[S_{\text{weak}}, \cdot] \quad (3.6)$$

Results. A 3.01-bit OWQ model (3-bit base with 1% FP16 weak columns) outperforms standard 4-bit GPTQ across multiple benchmarks, demonstrating that low-bit performance collapse is driven almost entirely by the small fraction of weight columns that interact with outlier activations. OWQ requires only a small calibration set and provides Pareto-efficient trade-offs between model size and perplexity.

3.2.4 Compensation-Based Methods

Salience-based methods choose where to spend extra bits, but each weight is still rounded on its own. The next family asks what to do about the error that this independent rounding leaves behind. Compensation-based methods address quantization error through corrective updates to unquantized weights. These approaches utilize second-order curvature information to minimize output perturbations. The fundamental challenge stems from greedy quantization, which rounds each weight to the nearest discrete level. This independent operation neglects the interdependencies between parameters. Upon quantization of a weight, the resultant error propagates through the layer computation and influences the optimal values for all subsequent weights. Failure to account for these cross-weight dependencies results in error accumulation that scales poorly with model size, particularly within low-bit regimes.

The critical insight behind compensation-based approaches posits that the *Hessian* (second-order derivative) of the layer-wise reconstruction loss reveals the impact of quantization errors on the optimal values of other weights. Through decomposition of the Hessian (or its inverse) and application of this structure to propagate corrections, these methods transform naive greedy quantization into a globally-aware optimization process. The resultant algorithms achieve superior accuracy-compression trade-offs compared to simple round operations, albeit at the cost of increased computational complexity during the quantization process itself.

GPTQ frantar2022gptq

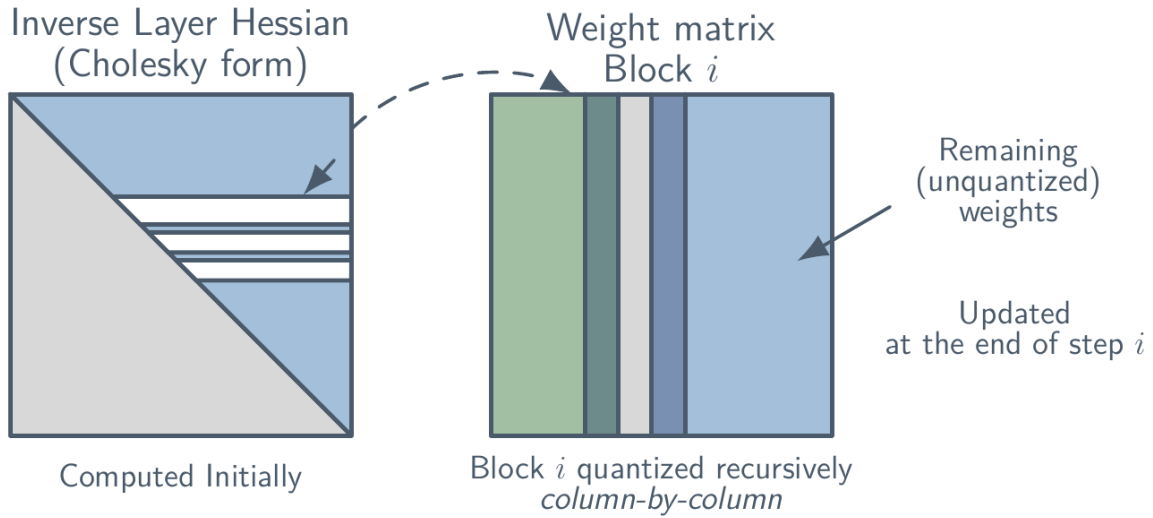


Figure 3.5: The GPTQ Algorithm. To avoid computationally prohibitive row-wise updates of OBQ, GPTQ employs a Lazy Batch-Update strategy, quantizing weights in blocks. frantar2022gptq

GPTQ scales second-order quantization to billion-parameter models by building upon Optimal Brain Quantization (OBQ) while introducing three critical algorithmic innovations that reduce complexity from months to hours. While OBQ provides high-accuracy quantization through Hessian-based error correction, its $O(d_{\text{row}} \cdot d_{\text{col}}^3)$ complexity renders it infeasible for LLMs. Standard Round-to-Nearest (RTN) executes quickly but destroys accuracy. GPTQ bridges this gap by approximating second-order optimization with runtime comparable to a single forward pass.

Methodology. GPTQ solves the layer-wise reconstruction problem:

$$\hat{\mathbf{W}} = \arg \min_{\tilde{\mathbf{W}}} \|\mathbf{W}\mathbf{X} - \tilde{\mathbf{W}}\mathbf{X}\|_F^2 \quad (3.7)$$

Using trace identities, this becomes:

$$\|\mathbf{W}\mathbf{X} - \tilde{\mathbf{W}}\mathbf{X}\|_F^2 = \text{Tr}((\mathbf{W} - \tilde{\mathbf{W}})\mathbf{X}\mathbf{X}^\top(\mathbf{W} - \tilde{\mathbf{W}})^\top) \quad (3.8)$$

For a single row w , the Hessian is $H = 2XX^\top$, yielding quadratic form:

$$\ell(\tilde{w}) = \frac{1}{2} (w - \tilde{w})^\top H (w - \tilde{w}) \quad (3.9)$$

Algorithm Overview

GPTQ rounds the columns of W one at a time, from left to right. Before it begins, it builds the inverse Hessian $H^{-1} = (2XX^\top + \lambda I)^{-1}$ from the calibration inputs X , where λ is a small constant added to the diagonal for stability; this matrix tells it how to adjust the columns it has not yet touched whenever it rounds one. After rounding column j to the low-bit grid, GPTQ measures the error this caused and pushes a matching correction onto the remaining columns:

$$e_j = \frac{W_{:,j} - \text{quant}(W_{:,j})}{[H^{-1}]_{jj}}, \quad W_{:,j+1:} \leftarrow W_{:,j+1:} - e_j H_{j,j+1:}^{-1}, \quad (3.10)$$

where e_j is the rounding error of column j divided by the diagonal entry $[H^{-1}]_{jj}$, and $H_{j,j+1:}^{-1}$ is the part of row j that sets how strongly each later column is corrected. To keep the GPU busy, these corrections are confined to a block of B columns (typically $B = 128$) while the block is processed, and only once the block is finished is its accumulated error applied to all columns that follow. The inverse Hessian is prepared once through a Cholesky factorization, which keeps the repeated corrections numerically stable on large matrices.

Key Properties and Results. GPTQ reduces OBQ runtime by orders of magnitude through shared column ordering, block updates, and Cholesky precomputation. Complexity decreases from $O(d_{\text{row}} \cdot d_{\text{col}}^3)$ to $O(\max\{d_{\text{row}} \cdot d_{\text{col}}^2, d_{\text{col}}^3\})$ in practice. The method quantizes OPT-175B to 3–4 bits with negligible perplexity degradation in ≈ 4 GPU hours (single A100) using 128 calibration samples. It supports flexible bit-widths (4/3/2-bit, with 2-bit typically requiring smaller blocks) and operates as pure post-training quantization requiring minimal calibration data.

VPTQ liu2024vptq

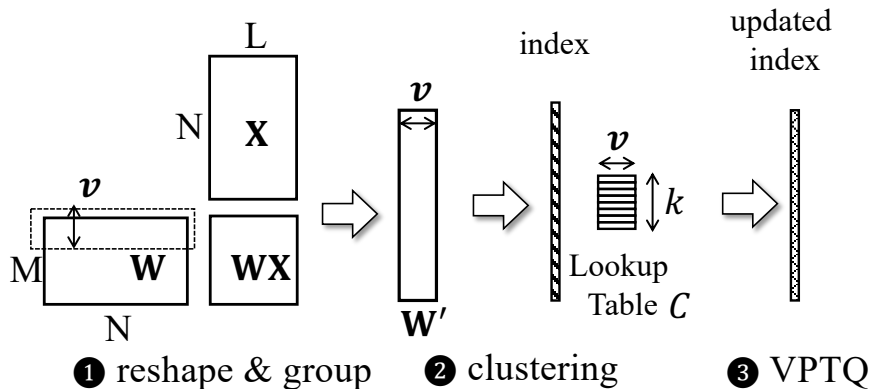


Figure 3.6: VPTQ partitions weights into short vectors represented by centroid indices and codebooks.

Quantization uses column-wise second-order updates, and inference reconstructs weights through lookup-table reads. liu2024vptq

VPTQ (Vector Post-Training Quantization) targets the regime where scalar weight-only PTQ becomes unstable near 2 effective bits. Instead of rounding each scalar independently, it partitions each weight column into short vectors $\mathbf{v}_{j,t} \in \mathbb{R}^\ell$ and represents each vector by an index into a learned codebook $\mathcal{C} = \{\mathbf{c}_1, \dots, \mathbf{c}_K\}$, enabling fractional effective bit-widths and capturing correlations among neighboring weights.

Methodology. The quantization objective is the standard second-order PTQ proxy:

$$\min_{\hat{\mathbf{W}} \in \mathcal{Q}} \text{Tr}((\mathbf{W} - \hat{\mathbf{W}}) \mathbf{H} (\mathbf{W} - \hat{\mathbf{W}})^\top), \quad (3.11)$$

where $\mathbf{H}$ is the layer Hessian estimated from calibration data and $\mathcal{Q}$ is the set of vector-quantized matrices. Columns are processed one at a time: after assigning column j , the error $\delta_j = \mathbf{w}_j - \hat{\mathbf{w}}_j$ propagates to the remaining unquantized columns F via

$$\mathbf{W}_{:,F} \leftarrow \mathbf{W}_{:,F} - \frac{\delta_j}{[\mathbf{H}^{-1}]_{jj}} [\mathbf{H}^{-1}]_{jF}, \quad (3.12)$$

the same GPTQ-style correction that prevents error accumulation across columns. The codebook is initialized by solving a Hessian-weighted k -means over the weight vectors $\{\mathbf{v}_{j,t}\}$:

$$\min_{\mathcal{C}, \mathcal{I}} \sum_{j,t} h_{j,t} \|\mathbf{v}_{j,t} - \mathbf{c}_{I_{j,t}}\|_2^2, \quad (3.13)$$

where $h_{j,t}$ is the local Hessian weight for vector (j, t) ; vectors in high-curvature directions of the reconstruction loss pull centroids harder, seeding the codebook toward the weight patterns that matter most for output fidelity. A second codebook is then fitted to the residuals $\mathbf{r}_{j,t} = \mathbf{v}_{j,t} - \mathbf{c}_{I_{j,t}}^{(1)}$, so each vector is represented by two indices rather than one, recovering precision without widening the primary codebook. Outlier-dominated tiles are assigned a separate partition. At inference, weights are reconstructed entirely by lookup-table reads on the stored indices and codebooks.

Algorithm Overview

VPTQ proceeds layer by layer. (1) Compute the layer Hessian $\mathbf{H}$ from calibration activations and partition each weight column into vectors $\{\mathbf{v}_{j,t}\}$ of length ℓ . (2) Initialize the primary codebook $\mathcal{C}^{(1)}$ via Hessian-weighted k -means over $\{\mathbf{v}_{j,t}\}$, using $h_{j,t}$ as per-vector weights. (3) Quantize columns left to right: assign each $\mathbf{v}_{j,t}$ to its nearest centroid, accumulate the column error δ_j , and propagate it to all remaining columns via the inverse-Hessian update. (4) Fit a residual codebook $\mathcal{C}^{(2)}$ to the first-stage residuals $\{\mathbf{r}_{j,t}\}$; assign outlier-dominated tiles to a dedicated partition. (5) Store the layer as centroid index pairs plus codebooks; inference reconstructs weights by lookup rather than dequantization arithmetic.

Results. VPTQ reaches ≈ 2.02 effective bits on LLaMA-2-7B (WikiText-2 PPL 6.13, AvgQA 58.2) and LLaMA-3-70B (PPL 5.60, AvgQA 70.9) [liu2024vptq](#), with a $1.6\text{--}1.8\times$ throughput gain over prior ultra-low-bit baselines attributed to lookup-based decoding and vector lengths aligned with cache-line granularity.

3.2.5 Optimization-Based Methods

Compensation-based methods repair rounding error with a fixed rule read off the curvature. The next family is less committed: instead of fixing the error afterwards, it treats the quantizer’s own settings as quantities to be learned. Optimization-based post-training quantization casts the design of quantizer as a calibration problem. The default approach in PTQ to determine the quantized value for a weight or activation involves the simple mapping of translating the range of real values (determined using calibration data) into a scale factor and then rounding all values to the nearest integer. These choices are quick, but localized and static; one large activation can significantly inflate the quantization range and the same rounding mechanism may be used when rounding up versus rounding down produce differing impacts on the output, and the bit precision allocated to a layer may not directly reflect the optimization of reconstruction error.

Therefore, optimization-based methods started to appear as substitutes for these hand-crafted choices by treating them as variables that are tuned on a small calibration set. While the full precision model is fixed, the quantization setup is changed in order to minimize the discrepancy between the full-precision and the quantized model. Depending on the strategy, optimized variables may include rounding, clipping ranges, scaling factors, equivalent transformations, layer-wise precision selection. Since quantization involves discrete operations, particularly rounding, these strategies mostly used continuous relaxation or straight-through gradient estimations to allow the optimization by backpropagation [nagel2020adaround](#); [li2021brecq](#); [esser2020learned](#); [shao2024omniquant](#).

OmniQuant [shao2024omniquant](#)

Methodology. OmniQuant freezes the original full-precision weights and optimizes a small set of learnable quantization parameters through block-wise gradient descent on a calibration set. Processing one block at a time keeps the optimization tractable without retraining the full model.

The first component, Learnable Weight Clipping (LWC), replaces fixed range estimation with learnable upper and lower clipping strengths $\gamma, \beta \in [0, 1]$, parameterized via sigmoid. Given a weight matrix $\mathbf{W}$, a target bit-width N , and a zero-point z that aligns real zero to an integer code, the step size is $h = (\gamma \max(\mathbf{W}) - \beta \min(\mathbf{W})) / (2^N - 1)$, where $\max(\mathbf{W})$ and $\min(\mathbf{W})$ are the maximum and minimum values of the weight matrix. The quantized weight is then:

$$\mathbf{W}_q = \text{clamp} \left(\left\lfloor \frac{\mathbf{W}}{h} \right\rfloor + z, 0, 2^N - 1 \right) \quad (3.14)$$

When $\gamma = \beta = 1$, h reduces to the standard MinMax step size — the range divided uniformly into $2^N - 1$ levels — so LWC strictly generalizes that baseline.

The second component, Learnable Equivalent Transformation (LET), addresses activation outliers by learning a channel-wise scaling vector $\mathbf{s} \in \mathbb{R}^{C_{in}}$ and shifting vector $\delta \in \mathbb{R}^{C_{in}}$, where C_{in} is the number of input channels. These produce a mathematically equivalent rearrangement

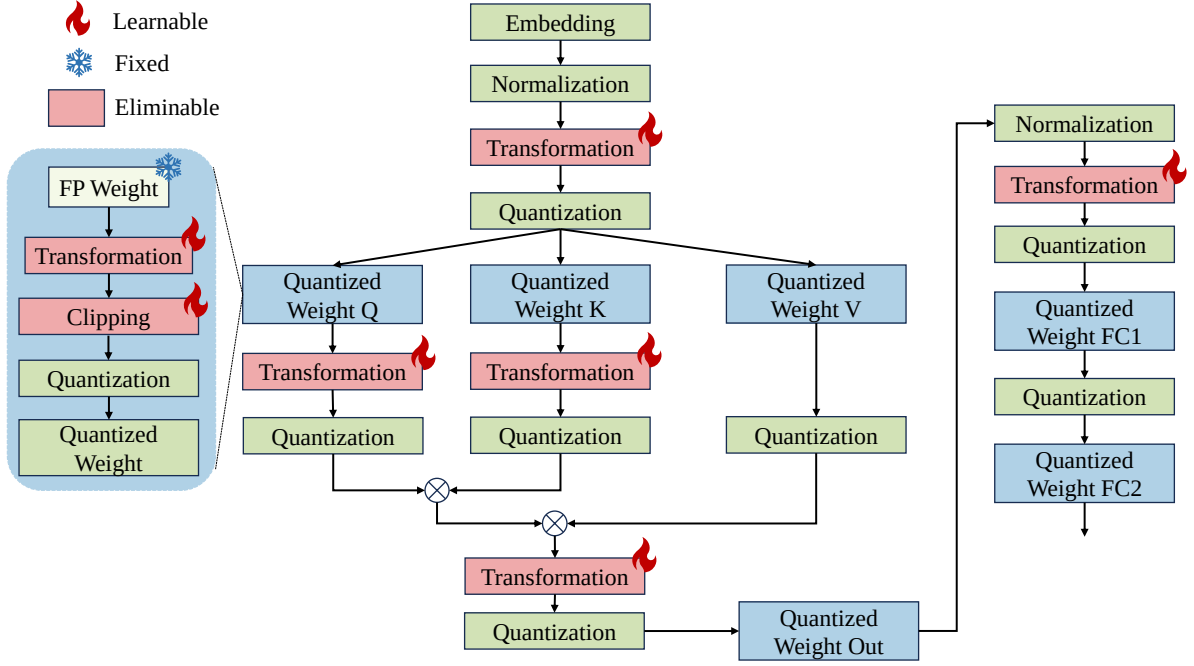


Figure 3.7: OmniQuant freezes the original full-precision weights and learns quantization parameters via gradient descent, simultaneously optimizing Learnable Weight Clipping (LWC) and Learnable Equivalent Transformation (LET). All learnable parameters are absorbed into the quantized weights after calibration, introducing no runtime overhead.

of the linear layer $\mathbf{Y} = \mathbf{X}\mathbf{W} + \mathbf{B}$:

$$\mathbf{Y} = \underbrace{[(\mathbf{X} - \delta) \odot \mathbf{s}]}_{\tilde{\mathbf{X}}} \cdot \underbrace{[\mathbf{s} \odot \mathbf{W}]}_{\tilde{\mathbf{W}}} + \underbrace{[\mathbf{B} + \delta\mathbf{W}]}_{\tilde{\mathbf{B}}} \quad (3.15)$$

where $\odot$ and $\odot$ denote elementwise division and multiplication respectively. The transformed activation $\tilde{\mathbf{X}}$ is quantization-friendly since the outlier magnitude has been shifted into $\tilde{\mathbf{W}}$, where LWC handles it. After calibration, $\mathbf{s}$ and δ are absorbed into adjacent weights, adding no runtime overhead.

Both components are optimized jointly by minimizing the squared output mismatch between the original and quantized block:

$$\arg \min_{\Theta_1, \Theta_2} \|\mathcal{F}(\mathbf{W}, \mathbf{X}) - \mathcal{F}(Q_w(\mathbf{W}; \Theta_1, \Theta_2), Q_a(\mathbf{X}; \Theta_2))\| \quad (3.16)$$

where $\mathcal{F}$ is the forward mapping of one transformer block, Q_w and Q_a are the weight and activation quantizers, and $\Theta_1 = \{\gamma, \beta\}$, $\Theta_2 = \{\mathbf{s}, \delta\}$ are the LWC and LET parameters. SGD with straight-through estimation handles the non-differentiable rounding.

Algorithm Overview

OmniQuant calibrates one transformer block at a time and never changes the original weights, which keeps the procedure light enough to run on a single GPU. For each block it starts the clipping strengths γ, β and the scaling and shifting vectors $\mathbf{s}, \delta$ at their neutral values, then repeats

the following for a few epochs over the calibration set: it applies the current LET transform and the current LWC quantizer to the block, sends the calibration inputs through both the original block and this quantized version, and takes a gradient step on $\gamma, \beta, \mathbf{s}, \delta$ to shrink the squared difference between the two outputs (Eq. 3.16). Since rounding has no usable gradient, a straight-through estimator passes the gradient through it unchanged. Once the block is calibrated, the learned $\mathbf{s}$ and δ are merged into the neighbouring weights and the block is quantized for good; its quantized output then serves as the input for calibrating the next block.

Results. On weight-only quantization, the gains are most pronounced at low bit-widths. At W2A16 on LLaMA-1-13B, OmniQuant achieves a WikiText-2 perplexity of 13.21 while GPTQ collapses to 3832. At W2A16 on LLaMA-1-7B, OmniQuant reaches 15.47 against RTN at 1.1×10^5 . At higher bit-widths the margins narrow: at W3A16 on LLaMA-1-7B OmniQuant reaches 6.49 and at W4A16 it reaches 5.86, consistently ahead of prior methods across the full LLaMA-1 and LLaMA-2 families.

On weight-activation quantization at W4A4, OmniQuant achieves 52.65% average zero-shot accuracy on LLaMA-1-7B across six tasks, against 48.43% for the best prior PTQ method and 46.43% for LLM-QAT+SQ, which requires quantization-aware training with 100k samples. The LLaMA-2 family from 7B to 70B is processed on a single A100-40G GPU with 128 calibration samples in 1 to 16 hours.

3.2.6 Search-Based Methods

The methods reviewed so far assign bit-widths layer by layer using local importance scores or learned parameters, implicitly assuming that a lower sum of per-layer quantization errors produces a better model overall. A different perspective treats the per-layer bit-width assignment as a combinatorial optimization problem and searches over the space of valid configurations directly. This avoids the error-monotonicity assumption entirely: rather than scoring layers independently and minimizing a linear sum, search-based methods evaluate complete compressed models and select the configuration that minimizes end-to-end output degradation subject to a compression budget.

EvoPress evopress2024

Methodology. For quantization, EvoPress first builds a level database by compressing each linear layer independently to each available bit-width (2, 3, 4, 5, and 6 bits) using GPTQ with group size 128. A compressed model is then a vector $v \in \{2, 3, 4, 5, 6\}^n$ assigning one bit-width to each of the n layers. The search objective is:

$$\hat{M}^* = \arg \min_v D_{\text{KL}}(P_M \parallel Q_{\hat{M}_v}) \quad \text{subject to} \quad \text{size}(\hat{M}_v) \leq C, \quad (3.17)$$

where P_M and $Q_{\hat{M}_v}$ are the output distributions of the original and compressed model on calibration tokens, and C is the target model size. KL divergence is used as the fitness signal rather than perplexity or layer-wise MSE, as it is more robust under limited calibration data.

The search follows a $(1 + \lambda)$ -evolutionary algorithm with level-switch mutation: one layer’s bit-width is raised by one step while a second layer of the same size has its bit-width lowered by one step, preserving the compression budget exactly. Selection uses aggressive multi-step evaluation: all λ offspring are first scored on a fraction of a calibration batch, the weakest candidates are eliminated, and the survivors are re-evaluated on progressively larger batches, reaching a full minibatch only in the final stage.

Algorithm Overview

(1) Build the level database: compress each layer to each available bit-width with GPTQ (group size 128). (2) Initialize with the uniform assignment at the target average bit-width. (3) Each generation: generate λ offspring via level-switch mutation (raise one layer by one bit, lower a same-size layer by one bit). (4) Evaluate offspring in multiple stages with increasing token budgets; retain only the fittest at each stage. (5) Repeat until convergence; the survivor defines the per-layer bit-width profile applied at inference.

Results. At 3-bit average quantization on Llama-3-8B, EvoPress achieves WikiText-2 PPL 7.49 against uniform GPTQ PPL 12.19 and DP-based search PPL 29.00, with zero-shot average accuracy 64.3 versus 60.2 for uniform. The collapse of DP to PPL 29.00 on the same model illustrates precisely why error monotonicity fails at extreme compression: minimizing a linear sum of layer-wise errors selects a configuration that is locally cheap but globally destructive. On Mistral-7B-v0.3 the gap is more modest (PPL 5.21 vs 5.54) but consistent across all tested models. EvoPress also supports hardware-aware constrained search: under vLLM’s restriction to 4-bit and 8-bit only with shared bit-widths inside each block, it finds mixed-precision profiles that preserve accuracy at the same average bit-width.

3.2.7 Empirical Comparison

The comparison is organized around three compression scenarios. The first is weight-only quantization across the 4-, 3-, and 2-bit spectrum, where the dominant variable is the quantization family. The second is joint weight-and-activation quantization at 4 bits (W4A4), which additionally requires controlling dynamic activation distributions at inference time. The third is non-uniform bit-width search, where the allocation of bits across layers is itself the optimization target. Perplexity (PPL) on WikiText-2 is the primary metric ($\downarrow$ better); zero-shot average accuracy on reasoning benchmarks is used as a secondary metric ($\uparrow$ better) where reported.

Table 3.1 summarizes the methods reviewed in this section by the dominant mechanism used to control quantization error.

GPTQ, QuIP, and VPTQ cover the weight-only spectrum from 4 bits down to 2 bits; OmniQuant spans the same range through a learned transformation rather than second-order correction. Table 3.2 reports WikiText-2 perplexity on LLaMA-3-8B and LLaMA-3-70B for the three compensation- and rotation-based methods. OmniQuant results, evaluated by the original authors on LLaMA-2-7B and LLaMA-2-70B (comparable scale, one generation earlier), are included with a footnote to indicate the architecture difference.

Table 3.1: Methods covered in this section, organized by error-control family.

Method	Family	Quantizes	Key mechanism
GPTQ	Compensation	Weights	Second-order layer-wise reconstruction via OBS
QuIP	Rotation + Comp.	Weights	Incoherence via orthogonal transforms; LDLQ coordinate descent
VPTQ	Compensation	Weights	Hessian-weighted vector quantization with residual codebook
QuaRot	Rotation	W + Act	Hadamard rotation fused into the model for outlier removal
OWQ	Saliency	Weights	Mixed-precision via activation-outlier column detection
OmniQuant	Optimization	Weights	Learnable equivalent transformation optimized on calibration data
EvoPress	Search	Weights	Evolutionary search over per-layer bit-width profiles under a size budget

Table 3.2: WikiText-2 perplexity ($\downarrow$) for weight-only quantization across bit-width regimes. GPTQ at 2 bits without grouping produces perplexity in the thousands and is omitted from that block.

Method	Bits	8B model	70B model
FP16	16	6.14	2.90
<i>4-bit weight-only</i>			
GPTQ	4	6.50	3.30
QuIP	4	6.50	3.40
VPTQ	≈ 4	6.42	3.15
OmniQuant [†]	4	5.74	3.47
<i>3-bit weight-only</i>			
GPTQ	3	8.20	5.20
QuIP	3	7.50	4.70
VPTQ	≈ 3	6.97	3.81
OmniQuant [†]	3	6.58	3.92
<i>2-bit weight-only</i>			
VPTQ	≈ 2	5.64	5.66
OmniQuant [†]	2 (g128)	11.06	6.55

[†]OmniQuant evaluated on LLaMA-2-7B / LLaMA-2-70B; GPTQ, QuIP, VPTQ on LLaMA-3-8B / LLaMA-3-70B.

Joint quantization of both weights and activations to 4 bits (W4A4) is a stricter regime: activation values vary at inference time and cannot be pre-compensated by a static calibration pass. Table 3.3 compares SmoothQuant (the channel-rescaling baseline), OmniQuant (optimization-based), and QuaRot (rotation-based preprocessing) on WikiText-2. AffineQ, QLLM, and Atom are not covered in this section and are omitted.

Table 3.3: WikiText-2 perplexity ($\downarrow$) under W4A4 joint quantization on LLaMA-1 and LLaMA-2. QuaRot results use the GPTQ rounding variant (QR-GPTQ); dashes indicate the model was not evaluated in that work.

Method	LLaMA-1		LLaMA-2	
	7B	13B	7B	13B
FP16	5.68	5.09	5.47	4.88
SmoothQuant	25.2	40.0	83.1	35.8
OmniQuant	11.2	10.8	14.2	12.3
QuaRot	—	—	6.10	5.40

Non-uniform bit-width search assigns a distinct compression level to each layer rather than applying a single bit-width uniformly. This allows layers with higher sensitivity to retain more bits at the expense of less-sensitive layers, without increasing the total model size. Table 3.4 shows results for EvoPress against two baselines at a target average of 3 bits: uniform GPTQ (all layers at the same bit-width) and a dynamic-programming search that greedily minimizes the sum of per-layer GPTQ reconstruction errors.

Table 3.4: WikiText-2 PPL ($\downarrow$) and zero-shot average accuracy ($\uparrow$) at ~ 3 -bit average weight quantization. DP minimizes the sum of layer-wise GPTQ errors; EvoPress minimizes end-to-end KL divergence from the original model. A dash indicates the result was not reported for that model.

Model	WikiText-2 PPL $\downarrow$			Zero-shot Avg. $\uparrow$		
	Uniform	DP	EvoPress	Uniform	DP	EvoPress
Mistral-7B	5.54	—	5.21	—	—	—
LLaMA-3-8B	12.19	29.00	7.49	60.2	—	64.3

At 4-bit weight-only quantization, second-order compensation (GPTQ) already recovers most of the full-precision accuracy gap, and vector quantization (VPTQ) extends this advantage to 3 bits and remains functional at 2 bits where scalar methods require very small group sizes to avoid collapse. Enabling activation quantization (W4A4) introduces a qualitatively distinct challenge: SmoothQuant’s channel-wise rescaling leaves large perplexity gaps, OmniQuant’s learned transformation improves upon it, but rotation-based preprocessing (QuaRot) is the only family that approaches near-lossless results in this regime. Non-uniform search (EvoPress) provides an orthogonal gain: at the same average bit-width, it outperforms both uniform assignment and DP-based layer scoring, with the advantage widening dramatically at extreme compression where the error-monotonicity assumption underlying DP fails catastrophically.

»¿

3.3 Related Work on Pruning

Pruning is the most widely studied form of model compression, and it has been applied to Transformers in many ways. To compare these methods coherently, this section organizes them along the same axes used in the taxonomy below: what unit is removed, what signal drives the removal decision, what recovery the method requires, and what deployment benefit it targets. These axes are not independent. Two methods can report the same nominal compression ratio yet differ in execution regime, preprocessing cost, and realized speedup, so keeping them explicit avoids reducing the comparison to a single accuracy-versus-compression curve.

3.3.1 Problem Setting and Taxonomy

Building on the compression framework introduced earlier in the report, this subsection makes the classification scheme explicit before the individual methods are reviewed.

Taxonomy of Transformer Pruning Methods

The main sparsity topologies were introduced formally in Section 2.3.2 (see Figure 2.2). From a deployment standpoint, they differ in whether pruning yields compressed storage alone or native dense-shape acceleration after model surgery.

The taxonomy adopted here follows four coupled axes. The *removable unit* determines whether the method acts on scalar weights or executable structures such as attention heads, FFN channels, and layers **michel2022not**; **kurtic2022optimal**. The *optimization signal* ranges from magnitude and activation statistics to differentiable mask learning, first-order Taylor criteria, second-order curvature surrogates, or search-based objectives **sun2023simple**; **xia2022structured**; **ma2023llmpruner**; **frantar2023sparsegpt**; **kwon2022fast**. The *recovery stage* distinguishes direct post-training pruning from methods requiring LoRA recovery, gradual fine-tuning, or distillation. The *deployment objective* determines whether the reported benefit is compressed storage, dense-shape acceleration, or measured speedup **kurtic2023ziplm**; **evopress2024**; **darwinlm2025**.

Each family corresponds to a different deployment objective. Unstructured methods such as Wanda and SparseGPT primarily optimize weight-level saliency **sun2023simple**; **frantar2023sparsegpt**. Structured methods span differentiable mask-learning (CoFi), dependency-aware gradient methods (LLM-Pruner), Fisher-based post-training frameworks, and runtime-aware second-order methods (ZipLM) **xia2022structured**; **ma2023llmpruner**; **kwon2022fast**; **kurtic2023ziplm**. Search-based methods such as EvoPress and DarwinLM treat pruning as configuration optimization under explicit budget or hardware constraints **evopress2024**; **darwinlm2025**. Figure 3.8 summarizes this organization.

Figure 3.8 presents a taxonomy of transformer pruning methods across six dimensions: pruning unit, sparsity topology, optimization signal, recovery stage, deployment target, and evaluation regime. Each marker corresponds to a representative pruning method, illustrating how existing approaches differ not only in what they remove from the model, but also in how pruning

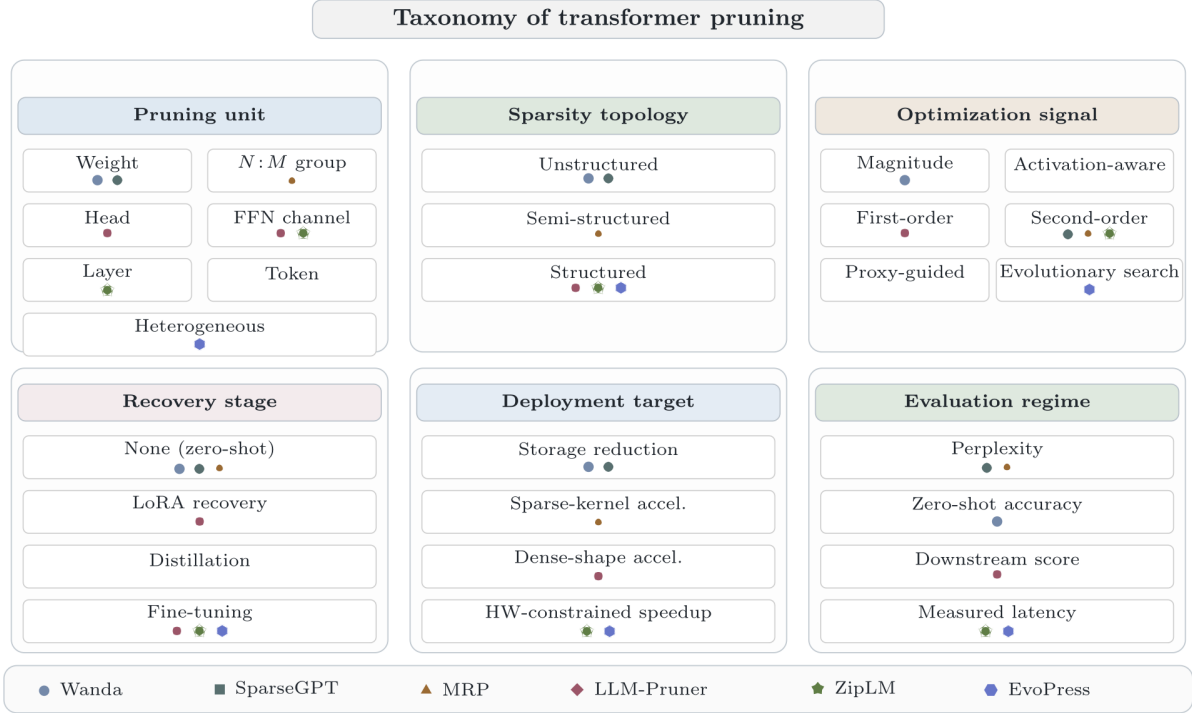


Figure 3.8: Taxonomy of the pruning methods considered in this chapter. The classification crosses the pruning unit, sparsity topology, optimization signal, recovery stage, deployment objective, and evaluation regime.

decisions are made, how the model is recovered, and how the resulting compression is evaluated.

The following subsections use this organization to review the selected pruning families in sequence: saliency-based methods, gradient-based structured methods, second-order methods, and search-based methods.

3.3.2 Saliency-Based Methods

Starting from the Taylor expansion of the loss change induced by pruning a weight θ_i , the first-order term is proportional to the gradient and the second-order term captures curvature. Saliency-based methods discard both terms and instead use the weight magnitude or a proxy derived from activation statistics as a surrogate for the true loss change. This zeroth-order approximation requires no gradient computation and no calibration beyond a simple forward pass, making these methods the cheapest option in the taxonomy.

Magnitude and Activation-Aware Baselines

Weight-Magnitude Pruning. Magnitude pruning uses absolute weight values as the saliency proxy:

$$S_{i,j}^{\text{mag}} = |W_{i,j}|. \quad (3.18)$$

Its core assumption is that small weights contribute less to the output and can be removed with limited degradation. The method requires no gradient-based recovery and remains the simplest baseline against which more sophisticated algorithms are evaluated.

Wanda (Weights $\times$ Activations). Wanda [sun2023simple](#) refines this baseline by incorporating activation information from a small calibration set. The saliency score is

$$\mathcal{S}_{i,j}^{\text{wanda}} = |W_{i,j}| \cdot \|x_j\|_2, \quad (3.19)$$

where x_j is the j -th input activation channel and $\|x_j\|_2$ is its RMS norm estimated over N calibration samples. The key insight is that importance is jointly encoded in weight magnitude and the energy of the coupled input channel.

Algorithm Overview

Wanda processes each layer independently: it collects input activations on a small calibration set (typically 128 samples), computes per-weight saliency scores as the product of weight magnitude and activation norm, then prunes the lowest-scoring weights for each output neuron until the target sparsity p is reached. No weight update or recovery step is applied. Magnitude pruning is a pure linear scan $O(|W|)$ with no calibration; Wanda adds only an activation-collection pass, keeping the overall cost near-linear.

These saliency methods are useful because they are cheap and easy to apply, but their decisions remain local. Once pruning moves from individual weights to heads, FFN channels, or layers, the method must account for structural dependencies between parameters. This motivates gradient-based structured pruning methods, where the score is computed over groups that can be removed consistently.

3.3.3 Gradient-Based Methods

Retaining the first-order term of the Taylor expansion gives a score proportional to the product of the gradient and the weight value. This makes the criterion sensitive to the direction of the loss landscape rather than just the weight magnitude, and it naturally extends to groups of weights by summing or aggregating the per-element scores. The methods in this section use exactly this first-order signal, applied to structured units such as heads, channels, and blocks rather than individual scalars.

Structured pruning replaces individual-weight saliency with group-level importance, because the natural units in a Transformer, namely attention heads, FFN channels, and entire layers, are coupled through residual pathways and shared projection matrices. Removing one head, for example, requires deleting consistent slices from W_Q , W_K , W_V , and W_O together; removing an FFN channel removes coordinated rows and columns from the two projection matrices. Any pruning criterion applied at this level must therefore identify groups that can be removed as a unit without breaking the forward pass [michel2022not](#); [kurtic2022optimal](#).

LLM-Pruner. LLM-Pruner [ma2023llmpruner](#) addresses the dependency problem explicitly by constructing a graph over the neurons of the network. For neurons N_i and N_j with input set $\text{In}(N)$ and output set $\text{Out}(N)$, the graph encodes two rules:

$$N_j \in \text{Out}(N_i) \wedge \deg^-(N_j) = 1 \implies N_j \text{ depends on } N_i, \quad (3.20)$$

$$N_i \in \text{In}(N_j) \wedge \deg^+(N_i) = 1 \implies N_i \text{ depends on } N_j. \quad (3.21)$$

Here $\deg^-(N_j)$ is the in-degree of N_j (number of neurons feeding into it) and $\deg^+(N_i)$ is the out-degree of N_i . The transitive closure of these rules groups all coupled parameters into a single dependency group $\mathcal{G} = \{W_i\}_{i=1}^M$, which is the minimal unit that can be consistently removed.

Once groups are defined, each is scored using a first-order Taylor approximation of the loss increase over a small calibration set $\mathcal{D}$. The importance of a structure tensor W_i is estimated as

$$I_{W_i} \approx \left| \frac{\partial \mathcal{L}^\top(\mathcal{D})}{\partial W_i} W_i - \frac{1}{2} W_i^\top H W_i \right|, \quad (3.22)$$

where H is the local Hessian. The first-order gradient is retained alongside the Hessian term because the calibration set is only a proxy for the pretraining distribution; discarding it would introduce bias. An element-wise variant replaces H with a Fisher-diagonal approximation. Individual tensor scores within a group are aggregated by sum, product, max, or last-only operators, and groups with the lowest aggregate score are removed first.

After pruning, LoRA adapters [hu2021lora](#) are inserted into all linear modules and fine-tuned for two epochs over roughly 50K samples. The low-rank factors are then merged back into the weight matrices, leaving no adapter overhead at inference time. The full scoring stage requires only 10 short calibration sequences, making the preprocessing cost modest relative to the recovery phase.

CoFi. CoFi [xia2022structured](#) formulates structured pruning as mask learning. It places binary masks on attention blocks, heads, FFN blocks, intermediate channels, and the shared hidden dimension, then optimises these masks jointly with the task loss:

$$\text{MHA}(X) = z_{\text{MHA}} \sum_{i=1}^{N_h} z_{\text{head}}^{(i)} \text{Att}\left(W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}, W_O^{(i)}, X\right), \quad (3.23)$$

$$\text{FFN}(X) = z_{\text{FFN}} \text{gelu}(X W_U) \text{diag}(z_{\text{int}}) W_D, \quad (3.24)$$

where N_h is the number of attention heads, $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}, W_O^{(i)}$ are the query, key, value, and output projection matrices of head i , and W_U, W_D are the up- and down-projection matrices of the FFN. Each mask variable z is relaxed to a continuous value during training using the hard-concrete distribution for L_0 regularisation, which allows gradients to flow through the mask. The training objective combines task loss with a distillation term and a Lagrangian sparsity constraint:

$$\mathcal{L}_{\text{prune}} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{distil}} + \lambda_1(\hat{s} - s_0) + \lambda_2(\hat{s} - s_0)^2, \quad (3.25)$$

where s_0 is the sparsity target and $\hat{s}$ is the expected sparsity under the current mask distribution. Training proceeds through a warm-up phase, joint mask-and-weight optimisation with a progressively increasing sparsity target, and a final fine-tuning pass on the thresholded subnetwork. Because the masks at different granularities interact, CoFi can discover structured subnetworks that a simple per-group ranking would miss.

Algorithm Overview

CoFi trains in three stages. A warm-up phase first stabilizes the model weights before mask learning begins, with the sparsity constraint applied softly. The main phase then jointly optimises masks and weights under the Lagrangian objective, progressively tightening the sparsity target toward s_0 over training. Once the target is reached, each continuous mask variable is thresholded against zero to produce a binary decision, and a final fine-tuning pass is run on the surviving subnetwork. Because masks at all granularities are trained simultaneously, the discovered subnetwork reflects cross-level interactions rather than independent per-unit decisions.

Gradient-based structured pruning improves the consistency of the removed units, but the score still depends on a local approximation of how removal affects the loss. When pruning becomes more aggressive, interactions among remaining and removed weights become more important. This leads to second-order methods, which use curvature information or local reconstruction objectives to compensate for the error introduced by pruning.

3.3.4 Second-Order Methods

The second-order term of the Taylor expansion captures the curvature of the loss around the current weights. Retaining this term, rather than discarding it as saliency and gradient methods do, allows the pruning criterion to account for how much a removal actually shifts the loss given the local geometry of the loss landscape. This leads to the OBS framework, where the optimal weight correction after a removal is derived from the inverse Hessian. The methods in this section apply that same principle at the scale of modern LLMs, recovering tractability by restricting the Hessian computation to individual layers or column blocks.

Optimal Brain Compression Formulation

For a layer with weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and calibration activations $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$, the layer-wise sparse reconstruction problem is

$$\min_{\mathbf{W}^{\text{sparse}}} \|\mathbf{W}\mathbf{X} - \mathbf{W}^{\text{sparse}}\mathbf{X}\|_F^2 \quad \text{s.t.} \quad \|\mathbf{W}^{\text{sparse}}\|_0 \leq (1 - p) d_{\text{out}} d_{\text{in}}, \quad (3.26)$$

where $\|\cdot\|_F$ is the Frobenius norm, $\|\cdot\|_0$ counts the number of nonzero entries, and $p \in (0, 1)$ is the target sparsity ratio. Using the empirical Hessian proxy $\mathbf{H} = \mathbf{X}\mathbf{X}^T/n$, this becomes a curvature-aware reconstruction problem whose solution depends on $\mathbf{H}^{-1}$ to redistribute pruning error across remaining weights. The methods that follow differ in how they instantiate this objective: SparseGPT removes one scalar weight at a time, MRP removes an entire N:M group

jointly, Fisher-based pruning applies curvature scoring to structured masks, and ZipLM extends the same principle to executable Transformer structures under runtime constraints.

Single-Weight Iterative Removal (SparseGPT)

SparseGPT [frantar2023sparsegpt](#) greedily prunes one weight at a time while applying compensation updates to remaining weights in the current block. Its local objective at iteration t is

$$\min_{\delta w} \frac{1}{2} \delta w^T H \delta w \quad \text{s.t.} \quad (w + \delta w) \cdot e_t = 0, \quad (3.27)$$

where e_t is the one-hot selector for the pruned weight.

Algorithm Overview

SparseGPT processes the weight matrix in column blocks of size B . For each block, it first constructs the regularized inverse Hessian $(XX^T + \lambda I)^{-1}$. Within each block, it iteratively identifies the weight with minimum local second-order loss, sets it to zero, then updates the remaining weights in that block using the corresponding column of $\mathbf{H}^{-1}$. After processing a block, lazy batch updates are applied to adjacent blocks. The method scales as $O(d_{\text{in}}^2 d_{\text{out}} + d_{\text{in}}^3 / B)$ per layer, with a calibration budget of approximately 128 samples.

It is more instructive to understand SparseGPT as a weight-level, Hessian-compensated method for pruning: each decision to prune weights is made locally, and weights are updated to make the layer output’s deviation as minimal as possible [frantar2023sparsegpt](#). It is through this same lens that SparseGPT’s modification for semi-structured N:M sparsity can be understood, where the compensation method is kept and mask selection is constrained to local groups. If the block size is set to $B_s = M$, and from each group of M weights the N weights with the smallest OBS reconstruction score are selected for removal, the compensation becomes more localized to a block rather than the whole layer. Within that block, however, weights are still selected and compensated one at a time, so the compensation at each step does not account for the interactions among the other weights being simultaneously removed from the same group, which is the gap MRP directly addresses.

Joint Optimization for Multi-Weight Removal (MRP)

The MRP framework [zhao2024pruning](#) generalizes second-order pruning to N:M groups. For a vectorized weight block w and binary pruning mask M , where $M_i = 1$ indicates that weight i is pruned, the objective minimizes the second-order loss increase for the full group:

$$\min_{\delta w} \frac{1}{2} \delta w^T H \delta w + g^T \delta w \quad \text{s.t.} \quad (w + \delta w) \odot M = 0, \quad (3.28)$$

where g is the gradient of the loss with respect to w , and $\odot$ denotes elementwise multiplication. Solving the group jointly yields a compensation update for the retained weights:

$$\delta w^* = -H_{\bar{M}\bar{M}}^{-1} (g_{\bar{M}} + H_{\bar{M}M} w_M), \quad (3.29)$$

where M denotes pruned indices, $\bar{M}$ retained indices, and w_M the weight values at the pruned coordinates. This expression makes the role of the block Hessian explicit: in N:M pruning, the removed weights are not independent decisions, so compensation must be computed for the group rather than for a single scalar.

Algorithm Overview

MRP operates layer by layer. For each layer, the block Hessian is estimated from calibration activations. The weight tensor is partitioned into non-overlapping groups of M consecutive weights. For each group, the joint second-order problem is solved: the mask selecting the $M - N$ weights with the highest reconstruction cost is identified, and the compensation update δw^* is applied to the retained N weights before the next group is processed. Because the group is treated as a unit rather than a sequence of scalar removals, the compensation correctly captures interactions among simultaneously pruned coordinates within the block.

Figure 3.9 illustrates this difference. SparseGPT compensates after removing one scalar weight, whereas MRP compensates after removing a constrained local group. The figure is therefore used to distinguish scalar second-order correction from group-level second-order correction.

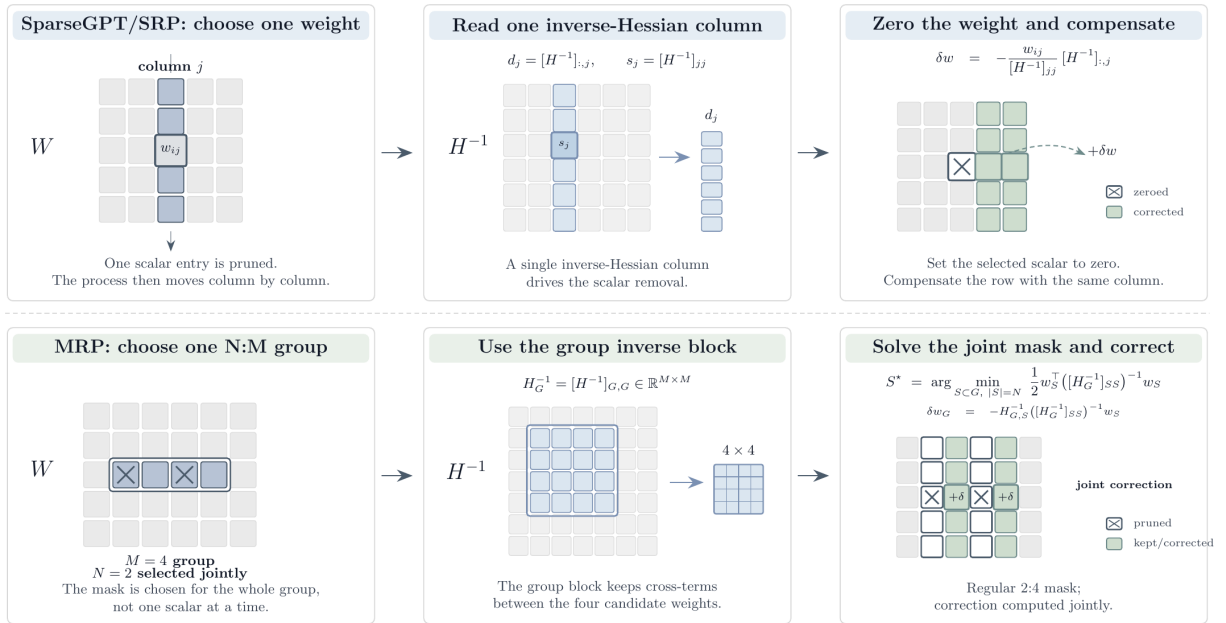


Figure 3.9: Second-order compensation strategies for unstructured and semi-structured pruning. SparseGPT removes one scalar weight per step and compensates using one column of $\hat{H}^{-1}$, while MRP removes an N:M group jointly and uses the block $(\hat{H}_{MM}^{-1})^{-1}$ to model interactions among simultaneously removed weights.

Fisher-Based Post-Training Structured Pruning

The post-training framework of Kwon et al. **kwon2022fast** applies second-order structured pruning without end-to-end post-pruning weight optimization. Binary masks over attention heads and FFN filters are found by solving

$$\arg \min_{\mathbf{m}} \mathcal{L}(\mathbf{m}) \quad \text{s.t.} \quad \text{Cost}(\mathbf{m}) \leq C, \quad (3.30)$$

where Cost is either a FLOPs or measured latency budget. Expanding around the dense mask and applying a diagonal Fisher approximation reduces the search to selecting units with the smallest diagonal Fisher scores subject to the cost constraint. For latency-constrained pruning, a piecewise-linear lookup table replaces the analytical cost model to incorporate real device behavior:

$$\text{LAT}(\mathbf{m}_\ell) = \begin{cases} 0, & \|\mathbf{m}_\ell\|_0 = 0, \\ c, & 0 < \|\mathbf{m}_\ell\|_0 \leq T, \\ a(\|\mathbf{m}_\ell\|_0 - T) + c, & \|\mathbf{m}_\ell\|_0 > T, \end{cases} \quad (3.31)$$

where c is the base latency incurred by any non-empty layer configuration, T is the structural threshold below which latency is approximately constant, and a is the slope capturing how measured latency grows beyond that threshold. All three constants are read from device benchmarks rather than derived analytically.

Algorithm Overview

The method proceeds in four steps: (1) diagonal Fisher scores are computed for head and FFN-filter masks on a small calibration set (2K training examples); (2) a FLOPs- or latency-constrained mask search is solved under the diagonal Fisher objective; (3) a block-diagonal Fisher refinement is applied layer by layer to recover interactions ignored by the diagonal approximation; (4) surviving mask values are tuned by layer-wise linear least-squares reconstruction. Because only mask values are updated, the method avoids end-to-end weight retraining. The paper reports pruning times of 39 seconds on GLUE and 135 seconds on SQuAD, compared to 33 hours for CoFi on MNLI.

Inference-Aware Structured Second-Order Pruning (ZipLM)

ZipLM **kurtic2023zipm** applies an OBS-style reconstruction objective to executable Transformer structures rather than scalar weights. For a candidate structure S with column mask $\mathbf{M}_S$, the removal score is

$$\arg \min_S \sum_{i=1}^{d_{\text{row}}} \mathbf{W}_{i, \mathbf{M}_S} \left((\mathbf{H}^{-1})_{\mathbf{M}_S, \mathbf{M}_S} \right)^{-1} \mathbf{W}_{i, \mathbf{M}_S}^T, \quad (3.32)$$

where d_{row} is the output dimension of the weight matrix and the sum aggregates the reconstruction cost across all output rows. After removing a structure, the exact compensation and Hessian

update are applied through block Gaussian elimination, so that all subsequent saliency scores remain consistent with the already-pruned state.

Inference-Aware Search

ZipLM adds a device-level objective on top of the local second-order scoring. It benchmarks attention and FFN configurations on the target hardware and stores measured runtimes in a lookup table. The global search then minimizes layer-wise structural sensitivity subject to a real speed constraint $\sum_{\ell} \tau_{\ell, s_{\ell}} \leq T_{\text{target}}$.

Algorithm Overview

ZipLM operates in two phases (see Figure 3.10). In the global phase, attention-head and FFN configurations are benchmarked on the target environment and a layer-wise sparsity profile satisfying the speedup requirement is selected. In the local phase, each selected layer is compressed by iteratively scoring candidate structures with the structured saliency metric, applying the optimal weight compensation, and updating the inverse Hessian by block Gaussian elimination. Recovery follows a gradual prune-and-fine-tune schedule with combined task, logit, and token-level distillation losses, at a cost of 8–10 fine-tuning epochs per pruning step.

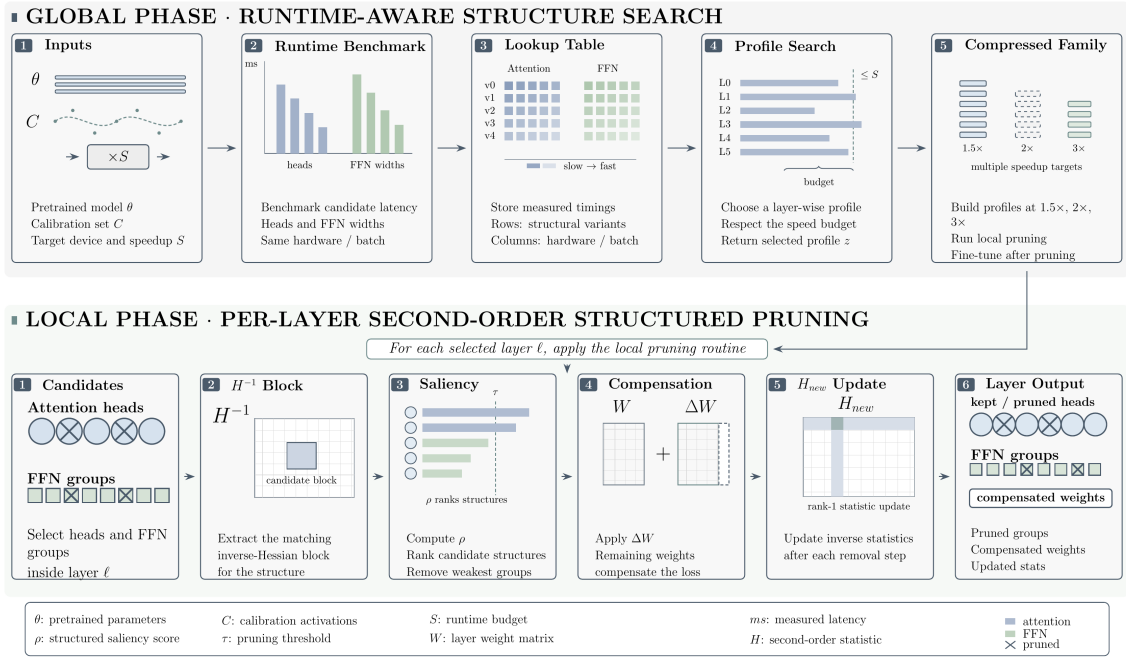


Figure 3.10: ZipLM pipeline. The global phase benchmarks attention-head and FFN configurations and searches for layer-wise sparsity profiles satisfying the required speedup. The local phase applies second-order structured pruning through inverse-Hessian scoring, compensation updates, and iterative Hessian refinement.

Second-order methods reduce local reconstruction error, but they do not fully resolve the allocation problem across the whole model. A method may know how to remove a head or

channel with limited local damage and still not know how many heads or FFN channels should remain in each layer under a global budget. This is the point at which search-based methods become relevant.

3.3.5 Search-Based Methods

Search-based methods operate over complete sparsity configurations rather than applying a single closed-form ranking rule to individual units. This is especially relevant when the quality of a pruning decision depends on how several removals interact across layers.

Constrained Evolutionary Frameworks

EvoPress. EvoPress `evopress2024` searches over discrete layer-wise compression profiles instead of assigning a single saliency score per layer. A candidate is a configuration $\mathbf{C} = \{c_1, \dots, c_L\}$ where $c_\ell \in \mathcal{S}_\ell$ is the compression level for layer ℓ . Budget is preserved by level-switch mutation: one layer becomes more compressed only if another becomes less compressed, keeping the global average sparsity S_{glob} fixed. Mutations are deliberately local, with most offspring differing from the parent by only one or two switches.

Selection proceeds in multiple rounds with increasing token budgets and decreasing survivor counts:

$$\mathcal{P}^{(r+1)} = \text{Top}_{n_{r+1}} \left(\{ \mathcal{F}_{T_r}(C) \mid C \in \mathcal{P}^{(r)} \} \right), \quad T_1 < T_2 < \dots, n_1 > n_2 > \dots, \quad (3.33)$$

where the fitness signal is KL divergence to the reference model. This progressive evaluation scheme is what makes EvoPress practically efficient: early rounds use few tokens to discard poor candidates quickly, reserving the full calibration budget for finalists. The method converges in approximately 5–6 GPU hours for a standard Llama-2-7B sparsity search.

Figure 3.11 summarizes this search loop. The important point is that the algorithm evaluates complete compression profiles, selects promising candidates with increasing evaluation budgets, and mutates the retained profile while preserving the global compression constraint.

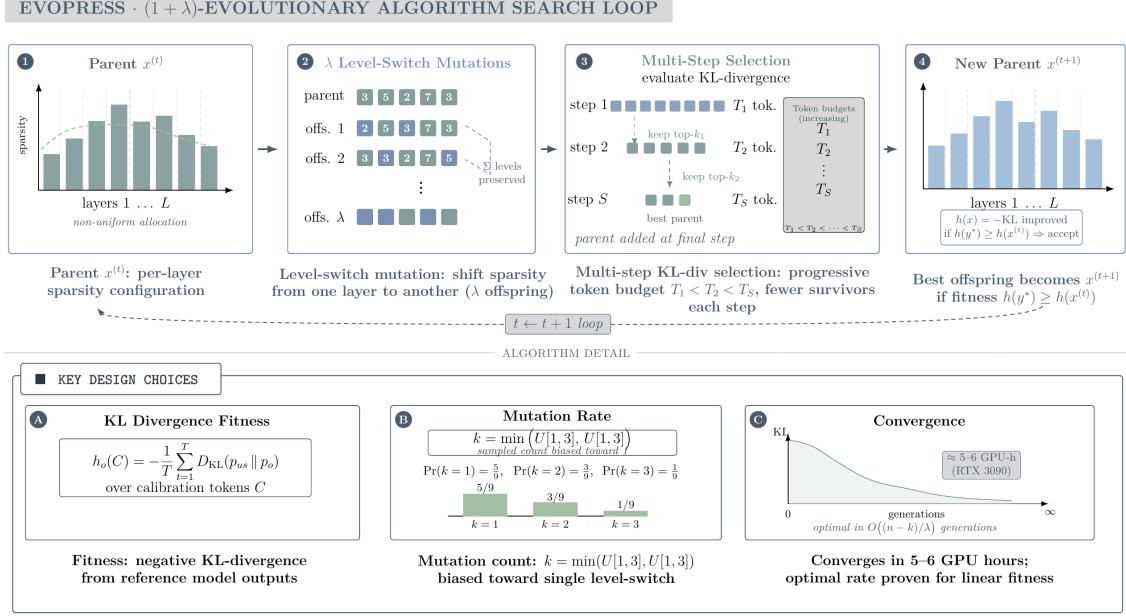


Figure 3.11: EvoPress $(1 + \lambda)$ evolutionary search loop for dynamic LLM compression (evopress2024). The method applies budget-preserving level-switch mutation and progressively evaluates candidates with larger calibration budgets.

DarwinLM. DarwinLM [darwinlm2025](#) extends evolutionary search to *structured* pruning by unifying OBS-style second-order pruning with a training-aware offspring selection strategy. It differs from EvoPress on three axes: structured granularity (heads and FFN channels), front-loaded Hessian computation stored in a sparsity-level database (SLD), and fitness measured after a short lightweight finetune rather than from the one-shot pruned model.

Sparsity Level Database

For each module, DarwinLM solves the structured OBS problem at N_l equally-spaced sparsity levels and stores the pruned-and-compensated weights. The optimal structured mask and compensation are:

$$M^* = \arg \min_M \sum_{i=1}^{d_{\text{row}}} \mathbf{w}_{i,M} (\hat{H}_{MM}^{-1})^{-1} \mathbf{w}_{i,M}^\top, \quad (3.34)$$

$$\delta^* = -\mathbf{w}_{:,M} (\hat{H}_{MM}^{-1})^{-1} \hat{H}_{M,:}^{-1}, \quad (3.35)$$

where d_{row} is the output dimension of the module weight matrix, M indexes the pruned columns, and $\hat{H}$ is the empirical Hessian computed from calibration activations. During search, candidate models are assembled by looking up stored entries, avoiding any Hessian recomputation.

Training-Aware Selection

Each candidate receives a short lightweight finetune before evaluation:

$$\mathcal{F}(C_k) = D_{\text{KL}}(\hat{M} \parallel \text{finetune}(C_k, T_f)), \quad (3.36)$$

where $\hat{M}$ is the output distribution of the dense reference model and T_f is the fine-tuning token budget. Selection uses three progressive rounds ($T_f \in \{10\text{K}, 50\text{K}, 200\text{K}\}$ tokens) with decreasing survivor counts. The SLD must be computed once per model and sparsity range, making the upfront cost proportional to $L \cdot N_l$ structured OBS operations. DarwinLM compresses Llama-2-7B to 2.7B parameters with higher downstream accuracy than ShearedLlama using $5\times$ fewer post-training tokens (10B vs. 50B).

Algorithm Overview

DarwinLM operates in two phases. In the offline phase, the SLD is built: for each module and each of N_l sparsity levels, the structured OBS problem is solved and the pruned-and-compensated weights are stored, so that any candidate configuration can be assembled by table lookup rather than recomputation. In the search phase, candidate models are stitched together by selecting one SLD entry per module, then evaluated through a lightweight finetune. Mutation follows the same budget-preserving logic as EvoPress, swapping one module’s sparsity level up only if another’s is swapped down. Selection proceeds over three rounds with token budgets of 10K, 50K, and 200K and shrinking survivor counts, so weaker candidates are eliminated cheaply before the full evaluation budget is spent. The upfront SLD cost is proportional to $L \cdot N_l$ structured OBS operations and is paid once per model.

Figure 3.12 shows the corresponding DarwinLM workflow. Compared with EvoPress, the key visual distinction is the front-loaded sparsity-level database: candidate models are assembled from stored structured-pruning choices, then filtered through training-aware selection.

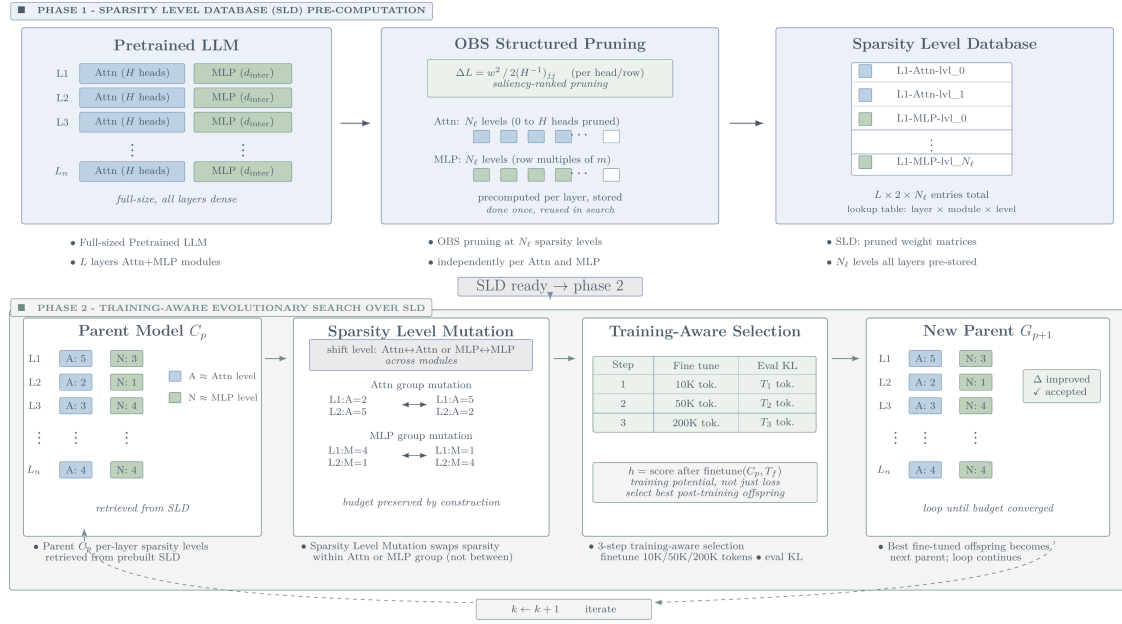


Figure 3.12: DarwinLM search loop (darwinlm2025). The method first builds a sparsity-level database with OBS-style structured pruning, then performs training-aware evolutionary selection over stitched candidate models.

Table 3.5 compares the practical preparation cost of the reviewed families. This table is included separately from the empirical results because a method can report strong accuracy while still requiring a substantially different calibration, recovery, benchmarking, or search budget.

Table 3.5: Preparation cost and evaluation requirements of representative pruning methods. The table separates low-cost post-training scoring, training-dependent structured pruning, curvature-based reconstruction, and population-based search.

Method	Dominant preprocessing cost	Additional requirement
Magnitude	Linear scan $O(W)$	None
Wanda	Activation collection and linear scoring	Small calibration set (≈ 128 samples)
CoFi	Multi-epoch joint mask and weight optimization	Task-specific teacher; ≈ 33 h for MNLI
LLM-Pruner	Forward/backward scoring over dependency groups	LoRA recovery (≈ 2.5 h on RTX 4090)
SparseGPT	Block-wise Hessian estimation and reconstruction	Calibration Hessian
MRP	Group-wise mask search and compensation	Block curvature updates for N:M groups
Fast PTPF	Diagonal or block Fisher estimation with mask tuning	Calibration set; optional latency lookup table
ZipLM	Layer-wise Hessian updates and runtime-table search	Calibration set plus hardware benchmarking pass
EvoPress / DarwinLM	Population search over compression profiles	1–6 GPU-hours; SLD pre-computation for DarwinLM

After the main method families have been described, the next step is to compare what they actually report. The empirical results below are therefore organized by pruning regime rather than by derivation, because perplexity, task accuracy, speedup, and recovery cost are not interchangeable measurements.

3.3.6 Empirical Comparison

The empirical evidence is organized by pruning regime. This avoids treating incompatible measurements as if they were directly comparable: unstructured and N:M methods mainly report perplexity under weight sparsity, structured methods report task accuracy or speedup after architectural removal, and search-based methods report the quality of full pruning configurations under budget constraints.

Unstructured and Semi-Structured Results

Table 3.6 keeps representative low, medium, and large model settings rather than listing every available row. The unstructured rows compare magnitude pruning, SparseGPT, and Wanda at 50% sparsity, while the 2:4 rows show the semi-structured regime that motivates group-aware compensation such as MRP.

Table 3.6: Representative unstructured and semi-structured pruning results on WikiText perplexity. PPL denotes perplexity; lower is better.

Model	Setting	Dense PPL	Magnitude PPL	SparseGPT PPL	Wanda PPL
LLaMA-7B	50% unstructured	5.68	17.29	7.22	7.26
LLaMA-30B	50% unstructured	4.77	7.54	5.31	5.24
LLaMA-65B	50% unstructured	3.56	5.90	4.57	4.57
OPT-13B	50% unstructured	10.13	1.2×10^4	11.17	–
OPT-175B	50% unstructured	8.35	4.3×10^4	8.21	–
OPT-13B	2:4 semi-structured	10.13	–	12.96	–
OPT-66B	2:4 semi-structured	9.34	–	10.09	–
OPT-175B	2:4 semi-structured	8.35	–	8.74	–

The table shows two separate points. First, pure magnitude pruning can fail badly at 50% sparsity, while activation-aware and second-order methods preserve perplexity much more effectively. Second, semi-structured 2:4 pruning remains more constrained than unconstrained unstructured pruning, so the compensation mechanism must respect the local group structure.

Structured Pruning Results

Table 3.7 summarizes representative structured-pruning results without listing every benchmark. The rows are selected to show low, medium, and high compression or speedup regimes where available, while keeping the evaluation suite explicit: CoFi is reported on GLUE tasks and SQuADv1, Fast PTPF reports GLUE/SQuAD results, and ZipLM reports GLUE-style classification tasks as well as SQuADv1.

Table 3.7: Representative structured Transformer pruning results across recovery regimes. GLUE denotes the General Language Understanding Evaluation benchmark, SQuADv1 denotes Stanford Question Answering Dataset version 1.1, acc. denotes accuracy, and F1 denotes question-answering F1 score.

Method	Model	Setting	Key result
LLM-Pruner ma2023llmpruner	LLaMA-7B	20% prune ratio	98.6% zero-shot accuracy retention after LoRA recovery
	LLaMA-7B	50% prune ratio	83.6% zero-shot accuracy retention after LoRA recovery
CoFi xia2022structured	BERT Base	GLUE/MNLI, $\approx 12\times$ speedup	80.6 acc. versus 78.8 for TinyBERT ₄ at similar speedup
	BERT Base	SQuADv1, $\approx 8.7\times$ speedup	82.6 F1, matching TinyBERT ₄ at comparable speedup
Fast PTPF kwon2022fast	BERT Base	GLUE and SQuADv1, FLOPs-constrained pruning	60–70% FLOPs with $\leq 1\%$ accuracy/F1 drop
	BERT Base	GLUE and SQuADv1, latency-constrained pruning on V100	$1.34\times$ – $1.47\times$ geometric-mean speedup depending on batch size
ZipLM kurtic2023ziplm	BERT Base	GLUE/QNLI and MNLI, $2\times$ speedup	91.4 / 84.8 acc.; essentially dense-level performance
	BERT Base	GLUE/QNLI and MNLI, $10\times$ speedup	88.6 / 82.7 acc.; approximately 5M parameters
	BERT Large	SQuADv1, $10\times$ speedup	89.1 F1 with 20.9M parameters

The structured results are closer to deployment because they remove executable units, but they are also harder to compare directly. GLUE classification tasks and SQuADv1 question answering measure different behavior, and the recovery protocols also differ: CoFi uses task-specific training, LLM-Pruner depends on LoRA recovery, Fast PTPF is largely post-training, and ZipLM adds device-specific runtime profiling.

Evolutionary and Search-Based Results

Table 3.8 keeps representative search-based results and includes both EvoPress and DarwinLM. EvoPress illustrates configuration search for depth pruning and sparsity allocation, while DarwinLM shows structured evolutionary pruning with training-aware offspring selection.

Table 3.8: Representative search-based pruning results. Wiki2 and C4 are perplexity values where lower is better; task average is accuracy where higher is better.

Setting	Model	Budget	Method	Wiki2↓	C4↓	Task Avg.↑
Depth pruning	Mistral-7B	25%	EvoPress	8.66	12.04	—
	Mistral-7B	25%	ShortGPT	43.26	40.16	—
	Mistral-7B	25%	Shortened LLaMA	14.94	19.30	—
Depth pruning	Llama-2-7B	37.5%	EvoPress	17.98	18.91	—
	Llama-2-7B	37.5%	ShortGPT	70.94	63.51	—
	Llama-2-7B	37.5%	Shortened LLaMA	35.37	26.07	—
Unstructured allocation	Mistral-7B	70%	EvoPress	14.42	16.46	53.8
	Mistral-7B	70%	OWL	17.22	21.66	51.9
	Mistral-7B	70%	Uniform	23.08	30.03	49.9
Structured search	Llama-2-7B	2.7B model	DarwinLM	—	—	62.8 after 10B-token post-training
	Llama-2-7B	2.7B model	ShearedLLaMA	—	—	62.6 after 50B-token post-training

The search results support the same conclusion from two directions. EvoPress improves over local or heuristic allocation when multiple removals interact, and DarwinLM shows that training-aware evolutionary selection can produce stronger structured models with less post-training data than a fixed pruning baseline.

Cross-Regime Synthesis

Table 3.9 aggregates the evidence at a higher level. It is not meant to replace the detailed result tables above; it summarizes what each regime establishes for the design of a pruning-search method.

Table 3.9: Aggregated interpretation of the empirical pruning evidence by regime.

Regime	Representative setting	Selected evidence	Reading
Weight-level unstructured	LLaMA-7B, 50% sparsity	Dense PPL 5.68; magnitude 17.29; SparseGPT 7.22; Wanda 7.26.	Better saliency preserves perplexity, but the resulting pattern remains unstructured and does not guarantee dense-kernel speedup.
Semi-structured N:M	OPT-13B, 2:4 sparsity	Dense PPL 10.13; SparseGPT-style compensation gives 12.96 under the 2:4 constraint.	N:M sparsity needs group-aware compensation, which motivates MRP-style joint treatment of simultaneously removed weights.
Structured LLM pruning	LLM-Pruner on LLaMA-7B	20% prune ratio retains 98.6% zero-shot accuracy after LoRA recovery; 50% retains 83.6%.	Dependency-aware pruning keeps the model executable, but the final quality depends strongly on recovery.
Structured encoder pruning	CoFi, Fast PTPF, and ZipLM on BERT	CoFi reaches 80.6 MNLI acc. near 12× speedup; Fast PTPF keeps 60–70% FLOPs with ≤1% drop; ZipLM reaches 88.6/82.7 QNLI/MNLI acc. at 10× speedup.	Structured pruning can produce executable acceleration, but the protocols differ in training cost, hardware profiling, and recovery budget.
Search-based pruning	EvoPress and DarwinLM	EvoPress improves depth-pruning and sparsity-allocation baselines; DarwinLM reports 62.8 average accuracy after 10B-token post-training versus 62.6 for ShearedLLaMA after 50B tokens.	Configuration search addresses global allocation, while training-aware selection improves structured candidates beyond one-shot scoring.

3.4 Discussion

3.4.1 Local scoring and its limits

The reviewed methods show a clear progression in what is being scored. Magnitude pruning assigns importance through the absolute weight value $|W_{i,j}|$ alone, a criterion that carries no information about how actively each weight contributes to the model output. Applied to large language models, this proves insufficient: Sun et al. report that magnitude-based selection fails severely even at relatively low sparsity levels **sun2023simple**, and Ma et al. observe that pruning ratios above roughly 20% trigger a rapid representational collapse under magnitude scoring **ma2023llmpruner**. The perplexity results in Table 3.6 confirm this directly: on LLaMA-7B at 50% unstructured sparsity, magnitude pruning raises perplexity from 5.68 to 17.29. Wanda addresses this by incorporating the activation norm of the coupled input channel into the saliency score, so that a weight receives higher importance when its input direction is consistently active across calibration samples **sun2023simple**. SparseGPT replaces the magnitude surrogate with a second-order compensation term derived from a block-wise Hessian approximation, applied locally after each weight removal to counteract the induced output error **frantar2023sparsegpt**. Both methods substantially narrow the gap: SparseGPT gives 7.22 and Wanda gives 7.26 on LLaMA-7B at 50% sparsity (Table 3.6), with a consistent ordering on LLaMA-13B where magnitude reaches 20.21 against SparseGPT at 6.21 and Wanda at 6.15 **frantar2023sparsegpt**; **sun2023simple**. Across both models, the evidence supports a specific conclusion: for weight-level pruning of large language models, a saliency criterion must account for activation statistics or output curvature rather than weight magnitude in isolation.

The semi-structured results add another constraint. SparseGPT at 2:4 sparsity gives 12.96 perplexity on OPT-13B, 10.90 on OPT-30B, 10.09 on OPT-66B, and 8.74 on OPT-175B, against dense baselines of 10.13, 9.56, 9.34, and 8.35 respectively **frantar2023sparsegpt**. The absolute penalty decreases with model scale, but the 2:4 constraint consistently exceeds the cost of unconstrained unstructured pruning at the same nominal sparsity (Table 3.6). This is why MRP treats the removed N:M group jointly rather than applying single-weight compensation repeatedly **zhao2024pruning**. The point is not that MRP replaces SparseGPT in every setting. It handles a case where the feasible mask itself couples the removed weights.

3.4.2 Structured pruning as architecture selection

Once pruning moves to heads, FFN channels, hidden dimensions, or layers, the decision is no longer only about individual weights. LLM-Pruner first builds dependency groups so that the removed structures remain consistent with the model graph **ma2023llmpruner**. CoFi learns masks over several Transformer structures at once, including attention blocks, heads, FFN blocks, intermediate channels, and hidden dimensions **xia2022cofi**. Fast post-training pruning uses Fisher-based scores under FLOPs or latency constraints **kwon2022fast**. ZipLM applies second-order scoring to executable Transformer structures and adds measured runtime information during search **kurtic2023ziplm**.

The reported structured results match this shift in object. LLM-Pruner keeps 98.6% zero-shot accuracy retention at a 20% prune ratio on LLaMA-7B, but drops to 83.6% retention at a 50% prune ratio. CoFi reaches 80.6 MNLI accuracy at about $12\times$ speedup, compared with 78.8 for TinyBERT₄ at a similar speedup, and reaches 82.6 F1 on SQuADv1 at about $8.7\times$ speedup. Fast post-training pruning reports 60–70% remaining FLOPs with at most a 1% accuracy or F1 drop on GLUE and SQuAD, together with measured latency speedups of $1.34\times$ at batch size 32 and $1.47\times$ at batch size 256 **kwon2022fast**. ZipLM reports 91.4/84.8 accuracy on QNLI/MNLI at $2\times$ speedup for BERT Base, then 88.6/82.7 at $10\times$ speedup with roughly 5M parameters. These results are not just better or worse versions of weight pruning. They are pruning results over executable model structure.

Structured pruning also makes the search space more coupled. Removing one head, layer, or intermediate dimension changes the role of nearby structures, and a uniform allocation across the network is rarely optimal. DarwinLM frames this directly: different model components exhibit different sensitivities to pruning, which motivates non-uniform structured compression and a training-aware search over candidate architectures **darwinlm2025**. The implication is that the best allocation is a property of the full configuration, not of any single removable unit scored in isolation.

3.4.3 Why global allocation remains unresolved

A constraint that is common to all these techniques is that of allocation. We notice that while techniques like Wanda and SparseGPT demonstrate how much weight level pruning can be augmented with the help of activation information, or second order compensation **sun2023simple**;

frantar2023sparsegpt, the issue here is that these scores still only represent locally the impact of removing individual weights, channels, heads or blocks before the whole architecture post compression is determined. A layer, for example, that is 'safe' to prune according to its score can still have a negative interaction with the decision to prune some other layer elsewhere, and a given layer can only be drastically compressed if capacity is conserved in the layers around it.

EvoPress formalises this observation by arguing that many dynamic compression methods rely on an assumption of error monotonicity, where the quality of a compressed model is expected to follow the sum of layer-wise compression errors **evopress2024**. The authors show that this assumption need not hold for large language models: two compression profiles can have similar or even inverted local error sums while producing substantially different end-to-end behavior. This weakens the justification for a purely greedy or layer-wise allocation rule and motivates evaluating the realized effect of a complete profile rather than accumulating per-unit scores.

The empirical results make the scale of this effect visible. In depth pruning on Mistral-7B at 25%, EvoPress gives 8.66 WikiText perplexity and 12.04 on C4, while ShortGPT gives 43.26 and 40.16, and Shortened LLaMA gives 14.94 and 19.30 **evopress2024**. On Llama-2-7B at 37.5% depth pruning, EvoPress gives 17.98/18.91 on WikiText/C4, compared with 70.94/63.51 for ShortGPT and 35.37/26.07 for Shortened LLaMA (Table 3.8). These gaps are too big to just chalk up to local scoring formula differences. They indicate that the chosen mix of removed layers plays a major role in the outcome. The unstructured allocation results seem to point in the same direction but in a more moderate way: at 70% sparsity on Mistral-7B, EvoPress achieves a 14.42 WikiText perplexity and a 53.8 task average, while uniform allocation hits 23.08 and 49.9 (Table 3.8). Once the budget is split among layers, the way it's allocated naturally impacts the quality of the candidates.

3.4.4 Need for evolutionary search

At high compression, the difficulty lies in finding the transformer's units allocation that keeps the remaining network usable, since a unit that appears safe under a local score may become damaging once other layers are also compressed around it, and since the quality of the compressed model is only observed after all layer decisions have been chosen together we begin to see the evolutionary search becomes as a more useful framework although being usually more costly. in our case for example EvoPress represents each candidate as a vector of layer-wise compression ratios, mutates that vector under a fixed global budget, and evaluates the resulting allocation through KL-divergence against the reference model across progressively larger token budgets **evopress2024**

The same candidate-level treatment appears in DarwinLM, where structured pruning is organized through a sparsity-level database built by OBS-style module pruning, then candidate models are assembled and are evaluated after a short fine-tuning stage, so the search acts on the full compressed architecture produced by the selected module configuration **darwinlm2025**, and the reported comparison gives a useful indication of this role, since DarwinLM compresses Llama-2-7B into a 2,7B model with 62,8 average accuracy after 10B post-training tokens, close

to the 62,6 reported for ShearedLLaMA after 50B tokens in Table 3.8, supports the use of search over complete candidate structures within the reported setting

This candidate-level formulation also makes the cost of the search explicit, since the number of possible profiles grows with the number of layers, removable structures, and admissible sparsity levels, and since each candidate has to be instantiated and measured, sometimes after partial training or fine-tuning, making fitness evaluation the dominant cost of the procedure, and making diversity harder to preserve when selection repeatedly amplifies structurally similar high-ranking candidates across generations **cantupaz2000efficient**; **luque2011parallel**

From this point, the island model follows naturally as an organization of the same evolutionary process, since the population is divided into semi-independent subpopulations that evolve separately and exchange candidates only through periodic migration, allowing different islands to explore different regions of the pruning space before useful candidates are shared **cantupaz2000efficient**; **luque2011parallel**; **meng2018dynamicisland**, and this structure also fits the computational shape of Transformer compression, since candidate evaluation carries most of the cost and can be distributed across workers, allowing the search to preserve architectural diversity and reduce wall-clock time while keeping evaluation centered on complete compressed models **harada2021distributedpga**.

3.5 Conclusion

Post-training quantization methods address the precision loss caused by activation outliers in large language models, differing in whether they handle that constraint through weight rotation, salience-aware protection, or second-order weight adjustment. Pruning methods remove parameters or structures from the base model directly. Unstructured approaches such as Wanda and SparseGPT can reach high sparsity ratios with limited accuracy loss, but the resulting irregular sparsity patterns rarely translate into wall-clock speedup without specialised sparse kernels. Structured approaches such as LLM-Pruner, CoFi, and ZipLM remove executable units and produce models that accelerate on standard hardware, though the reported gains depend heavily on the recovery budget invested after pruning.

Across the reviewed methods, a common characteristic emerges: core decisions are made locally, before the effect of the full set of changes on the compressed model is known. The discussion and limitations sections of this chapter make this explicit for pruning. The same principle carries over to the proposed framework, which treats the compression configuration as the primary optimisation object rather than aggregating per-unit scores.

Part II

Contribution

Chapter 4

Methodology

In this chapter we present our methodology to find compressed transformer architectures within a budget. The aim is to formalize the notion of allocation-aware compression into a pipeline that identifies a candidate compression, applies budget constraints, compares candidates before recovery, searches the feasible space, then finally finetunes the candidate.

The presentation uses encoder structured pruning as the main running example because it is the most detailed setting in the work. The same methodology also applies to decoder compression, where the candidate representation changes according to the compression family. In both cases, the central question is how the available budget should be distributed across the model so that the compressed architecture remains recoverable.

The chapter follows the order of the pipeline. It first defines the search object and the budget constraint, then introduces the pre-recovery ranking signal used to compare candidates. It then presents the evolutionary search procedure and the recovery stage applied after selection.

4.1 Candidate Parameterisation

This section defines the candidate representation used by the search. A candidate can be seen as a record for the compressed units selected for one architecture, and naturally, its own form depends on the compression family. In our case, the encoder structured pruning uses binary masks over attention heads and feed-forward neurons, which follows the unit granularity used in structured Transformer pruning (michel2022not; xia2022cofi; kurtic2023ziplm) and the decoder compression uses discrete level vectors for layer dropping, quantization, low-rank factorisation, and rank-based recovery.

The first encoding is the structured-pruning mask used for the encoder experiments in which we will elaborate on in the next point.

4.1.1 Encoder Structured-Pruning Encoding

A candidate architecture is a pair of binary masks

$$c = (\mathbf{M}^H, \mathbf{M}^N) \in \{0, 1\}^{L \times H} \times \{0, 1\}^{L \times N}. \quad (4.1)$$

For layer ℓ , the active attention heads and active feed forward neurons are

$$H_\ell(c) = \{h : \mathbf{M}^H[\ell, h] = 1\}, \quad N_\ell(c) = \{n : \mathbf{M}^N[\ell, n] = 1\}. \quad (4.2)$$

The head mask selects all head slices in the attention projections of the Transformer. The neuron mask selects the rows of the first feed forward projection and corresponding columns of the second feed forward projection. Indices for the masks are specified using teacher original coordinates.

For this encoder encoding, the remaining definitions are derived from the same mask pair: structural counts, layer status, and candidate identity.

4.1.2 Encoder Structural Counts

The first derived object is the count vector induced by the two masks, which summarizes the mask for budget control and reporting. At layer ℓ , summing the binary decisions in each row gives

$$a_\ell(c) = \sum_{h=1}^H \mathbf{M}^H[\ell, h], \quad (4.3)$$

$$n_\ell(c) = \sum_{n=1}^N \mathbf{M}^N[\ell, n]. \quad (4.4)$$

In this sense, $a_\ell(c)$ records how many attention heads remain in layer ℓ , whereas $n_\ell(c)$ records how many feed forward neurons remain in that same layer. The search keeps these local quantities because cost and behaviour depend on where units are kept.

The global head and neuron counts are then obtained by summing the local counts across depth:

$$A(c) = \sum_{\ell=1}^L a_{\ell}(c), \quad N(c) = \sum_{\ell=1}^L n_{\ell}(c). \quad (4.5)$$

These totals report candidate size. Since they do not show whether the remaining units are concentrated in a few layers or distributed across many layers, we also track the number of layers that contain at least one active sublayer:

$$L_{\text{act}}(c) = \sum_{\ell=1}^L \mathbf{1}\{a_{\ell}(c) + n_{\ell}(c) > 0\}. \quad (4.6)$$

Finally, the same per-layer counts are used to evaluate the compute budget. Each active head contributes one head cost, and each active feed forward neuron contributes one neuron cost:

$$\text{MAC}(c) = \sum_{\ell=1}^L (a_{\ell}(c)c_H + n_{\ell}(c)c_N), \quad (4.7)$$

where c_H and c_N are the per head and per neuron costs derived in Section 4.2.

The pair (a_{ℓ}, n_{ℓ}) also fixes the execution case of layer ℓ . Table 4.1 shows how these counts determine the masked forward pass. The search therefore selects both the budget and the computation that remains at each depth.

Condition	Status	Masked forward
$a_{\ell} > 0, n_{\ell} > 0$	both active	attention and feed forward execute
$a_{\ell} = 0, n_{\ell} > 0$	feed forward only	attention update is zero; feed forward executes
$a_{\ell} > 0, n_{\ell} = 0$	attention only	attention executes; feed forward update is zero
$a_{\ell} = 0, n_{\ell} = 0$	identity	the layer returns its input

Table 4.1: Layer statuses induced by the structural masks.

This status is utilized whenever a candidate is evaluated. If it has positive counts then it must keep the corresponding sub-layer active, whereas zero counts remove that sub-layer update. Naturally, two candidates with the same global size can behave differently if their active units create different layer statuses. This is why the implementation also needs two levels of candidate description: an exact identity for caching and a coarser signature for population tracking, which is the idea of the next point

4.1.3 Encoder Granularity and Candidate Identity

Neuron choices are grouped for search efficiency. The implementation allocates feed forward neurons in blocks of sixteen. As a direct consequence, $n_{\ell}(c)$ is restricted to multiples of sixteen, except for the zero case. This block size keeps mutation aligned with meaningful cost changes and reduces candidates that differ only by negligible single-neuron moves.

For exact caching, the identity of a candidate is the bit string obtained by concatenating $\mathbf{M}^H$ and $\mathbf{M}^N$. Two candidates are identical if and only if this bit string is identical. This key

is deliberately fine-grained because it preserves the exact placement of every active unit. For diversity tracking, the search also records the coarser macro signature

$$\sigma(c) = (A(c), N(c), L_{\text{act}}(c)). \quad (4.8)$$

The macro signature loses placement information. Two candidates can have the same total counts and active layer count while assigning compute to different depths, so exact deduplication still uses the full bit string. The macro signature is used only as a population-level descriptor in the search mechanisms described in Section 4.5.

4.1.4 Decoder Compression Encodings

The decoder-model searches utilize smaller discrete genomes that point to preset compression options. In this manner, the search space can remain small and the search simply selects feasible level of layer, bit width, rank or recovery.

Layer drop. A candidate is a pair of boolean vectors over the L layers,

$$c_{\text{drop}} = (d^{\text{attn}}, d^{\text{mlp}}) \in \{0, 1\}^L \times \{0, 1\}^L, \quad (4.9)$$

where $d_{\ell}^{\text{attn}} = 1$ marks the attention sub-block of layer ℓ as dropped and $d_{\ell}^{\text{mlp}} = 1$ the feed forward sub-block; a dropped sub-block is replaced by an identity map. The budget fixes the exact number of dropped sub-blocks to $\lfloor s(2L) \rfloor$ for a target drop fraction s , so every candidate removes the same amount of depth and differs only in placement.

Quantization. A candidate assigns a bit-width to each quantizable weight matrix,

$$c_{\text{quant}} = (b_1, \dots, b_K), \quad b_i \in \mathcal{B}, \quad (4.10)$$

where $\mathcal{B} = \{2, 2.5, 3, 4, 8\}$ in the experiments. Each value names a pre-quantized version of matrix i that is built offline and then loaded during evaluation. Writing $|W_i|$ for the number of parameters in matrix i , the budget caps the size-weighted average bit-width,

$$\sum_i b_i |W_i| \leq \bar{b} \sum_i |W_i|, \quad (4.11)$$

for a target average bit-width $\bar{b}$.

Low-rank factorisation. A candidate assigns a retained-rank fraction to each factorised matrix,

$$c_{\text{rank}} = (\alpha_1, \dots, \alpha_K), \quad \alpha_i \in \mathcal{R}, \quad (4.12)$$

where $\mathcal{R} = \{0.05, 0.10, 0.20, 0.35, 0.50, 0.75\}$ in the experiments. For a matrix of shape $m_i \times n_i$, the retained rank is

$$q_i = \lfloor \alpha_i \min(m_i, n_i) \rfloor. \quad (4.13)$$

The factorised matrix therefore costs $q_i(m_i + n_i)$ parameters instead of $m_i n_i$. The budget caps the total stored size relative to the dense model,

$$\sum_i q_i (m_i + n_i) \leq \rho \sum_i m_i n_i, \quad (4.14)$$

for a target compression ratio ρ .

Rank-based recovery. For recovery, a candidate assigns a residual rank to each layer,

$$c_{\text{rec}} = (r_1, \dots, r_L), \quad 0 \leq r_\ell \leq r_{\text{cap}}, \quad (4.15)$$

and the recovered weight of layer ℓ adds back the top r_ℓ singular components of the residual between the dense and the compressed weight,

$$W_\ell = W_\ell^{\text{base}} + \sum_{i=1}^{r_\ell} \sigma_i u_i v_i^\top, \quad (4.16)$$

where (σ_i, u_i, v_i) are the residual singular triplets, computed once and reused. The budget caps the average residual rank, $\sum_\ell r_\ell (m_\ell + n_\ell) \leq \bar{r} \sum_\ell (m_\ell + n_\ell)$, for a target average rank $\bar{r}$.

Table 4.2 summarises the encodings used across the encoder and decoder experiments. The rest of the methodology develops the detailed mechanics on the structured-pruning mask because it is the setting that requires explicit head and neuron accounting.

Family	Candidate encoding	Budget constraint
Encoder structured pruning	head mask + FFN-neuron mask	MAC budget (head and neuron cost)
Decoder layer drop	paired boolean vectors ($d^{\text{attn}}, d^{\text{mlp}}$)	exact dropped sub-block count
Decoder quantization	per-matrix bit-width b_i	size-weighted average bits $\bar{b}$
Decoder low-rank	per-matrix retained-rank fraction α_i	stored params $\leq \rho \sum_i m_i n_i$
Decoder rank recovery	per-layer residual rank r_ℓ	average residual rank $\bar{r}$

Table 4.2: Candidate encodings and budget constraints across the five compression families. The table records what a candidate stores before any search operator, evaluation rule, or recovery step is applied.

4.2 Efficiency Budget and Cost Model

The binary masks from Section 4.1 indicate which attention heads and which feed-forward neurons are used at each layer. Based on them, the search computes for each candidate a deterministic operation cost, on which feasibility checks, repair and reporting are based. The cost for each candidate is computed with the reference sequence length S .

For one active attention head with head dimension d_h , the mask-dependent projection cost is

$$c_H = 2Sdd_h. \quad (4.17)$$

For one active feed-forward neuron, the corresponding cost of the two linear projections is

$$c_N = 2Sd. \quad (4.18)$$

Let $a_\ell(c)$ and $n_\ell(c)$ denote the active head and neuron counts of layer ℓ . The layer and candidate costs are

$$\text{MAC}_\ell(c) = a_\ell(c)c_H + n_\ell(c)c_N, \quad (4.19)$$

$$\text{MAC}(c) = \sum_{\ell=1}^L \text{MAC}_\ell(c). \quad (4.20)$$

The dense reference cost is

$$\text{MAC}_{\text{dense}} = L(Hc_H + Nc_N),$$

and the keep ratio is $\rho(c) = \text{MAC}(c)/\text{MAC}_{\text{dense}}$.

The target budget B is discrete because head counts are integer and neuron counts are allocated in blocks. A candidate is feasible when its cost lies in the tolerance band

$$(1 - \varepsilon)B \leq \text{MAC}(c) \leq (1 + \varepsilon)B. \quad (4.21)$$

This constraint fixes the candidate scale before search. The remaining selection problem is the allocation of the allowed compute across depth, attention heads, and feed-forward neurons.

In all of the three other decoder-model search types described in the decoder compression encodings, the same feasibility role is played by a different measure. It is the precise number of dropped sub-blocks for layer drop, the size-weighted average bit-width for quantization, and the number of stored parameters or average residual rank, respectively, for the low-rank and recovery searches. This measure is a single global constraint that can be evaluated from the candidate vector in closed form and is kept constant throughout the search (the repair mechanism thus operates exactly as in Eq. 4.21). The per-layer costs are fetched from a small look-up table that is first precomputed using the level database from Section 4.4.1 as the bit-count times the matrix dimensions for quantization and as $r(m + n)$ for a rank r factor, and no forward-pass calculation is required to assess feasibility.

4.3 The Surrogate Distance Metric

After budget filtering, candidates are already close in operation count, so cost no longer decides which one is better. The metric must instead compare how the same budget is allocated across depth, attention heads, and feed-forward neurons. It is evaluated before recovery, because fully fine tuning every searched mask would make the search infeasible.

For the decoder searches, token-level KL on large web-text calibration sets gives a dense enough signal to rank candidates directly. The encoder tasks have fewer labelled examples and much flatter class outputs at high sparsity, so KL alone is less reliable before recovery. The surrogate below therefore ranks encoder candidates in two steps: first by residual-update preservation, then by a task-facing distance among the surviving candidates.

4.3.1 Equal-Cost Candidate Ranking

The budget constraint turns a large architecture space into a local comparison problem. The metric receives a cohort of feasible allocations near the target cost, so the candidates differ

mainly in where their remaining units are placed.

Let $\mathcal{C}_t$ be the cohort evaluated at generation t . Every $c \in \mathcal{C}_t$ satisfies the feasibility band

$$(1 - \varepsilon)B \leq \text{MAC}(c) \leq (1 + \varepsilon)B. \quad (4.22)$$

The band makes the members of $\mathcal{C}_t$ comparable in cost. Small residual MAC differences inside the band are treated as secondary; the main comparison is the placement of the retained heads and feed forward neurons. This is why the score is cohort-local: it decides which feasible allocation should survive this generation.

For each candidate, the teacher $\mathcal{T}$ and masked student $\mathcal{S}(c)$ are evaluated on a calibration pack

$$\mathcal{D}_{\text{cal}} = \{x_i\}_{i=1}^B.$$

Let $p_{i,s} \in \{0, 1\}$ indicate whether token position s of example i is non-padding. All hidden-state and update distances are computed only over positions where $p_{i,s} = 1$.

The output of the metric is a cohort-local ordering. Lower values are better:

$$\Phi(c_1) < \Phi(c_2) \implies c_1 \text{ is preferred to } c_2 \text{ inside the current search generation.}$$

This ordering drives search-time selection before recovery. The first quantity needed by the two-step rule is the structural rank, so the metric starts from the layer updates.

4.3.2 Residual Update Matching

The structural rank is based on a simple intuition: a retained architecture should reproduce the transformations made along the teacher’s depth. Comparing final hidden states is too coarse for this purpose. A hidden state contains the accumulated residual stream, so a weak layer can be hidden by computation that happened earlier. The layer update exposes the contribution made at one depth.

Let h_ℓ denote the hidden state after layer ℓ , with h_0 the embedding output. The layer update is

$$\Delta h_\ell = h_\ell - h_{\ell-1}. \quad (4.23)$$

For the teacher and student, these updates are written as $\Delta_\ell^{(\mathcal{T})}$ and $\Delta_\ell^{(\mathcal{S})}$.

The update Δh_ℓ isolates the computation performed at layer ℓ . If a masked layer contributes almost nothing, then $\Delta_\ell^{(\mathcal{S})}$ becomes small even when the absolute hidden state remains numerically close to the teacher stream. This makes update matching a better search signal for partial and fully dropped layers.

For tensors A and B shaped as batch, sequence, and hidden dimension, define the calibration inner product

$$\langle A, B \rangle_{\text{cal}} = \sum_{i=1}^B \sum_{s=1}^S p_{i,s} A_{i,s}^\top B_{i,s}, \quad \|A\|_{\text{cal}} = \sqrt{\langle A, A \rangle_{\text{cal}}}. \quad (4.24)$$

The inner product sums only real token positions, so padding does not change the distance. The per-layer trajectory distance then checks two properties of the student update. Direction agreement asks whether the student update points the same way as the teacher update. Projection agreement asks whether the student recovers the teacher update with the right scale along that direction:

$$d_{\ell, \text{cos}}^{\text{traj}}(c) = \frac{1}{\pi} \arccos \left(\frac{\langle \Delta_{\ell}^{(S)}, \Delta_{\ell}^{(T)} \rangle_{\text{cal}}}{\|\Delta_{\ell}^{(S)}\|_{\text{cal}} \|\Delta_{\ell}^{(T)}\|_{\text{cal}} + \delta} \right), \quad (4.25)$$

$$\alpha_{\ell}(c) = \frac{\langle \Delta_{\ell}^{(S)}, \Delta_{\ell}^{(T)} \rangle_{\text{cal}}}{\|\Delta_{\ell}^{(T)}\|_{\text{cal}}^2 + \delta}, \quad (4.26)$$

$$d_{\ell, \text{proj}}^{\text{traj}}(c) = |1 - \alpha_{\ell}(c)|, \quad (4.27)$$

$$d_{\ell}^{\text{traj}}(c) = (1 - \lambda)d_{\ell, \text{cos}}^{\text{traj}}(c) + \lambda d_{\ell, \text{proj}}^{\text{traj}}(c). \quad (4.28)$$

The constant δ is a numerical stabiliser. The cosine term measures direction agreement between the student and teacher updates. The projection coefficient α_{ℓ} measures how much of the teacher update is recovered by the student along the teacher direction. A value near one means the student update has the correct magnitude in the teacher direction. A value near zero means the update is missing or orthogonal. Values above one mean the student overshoots the teacher update.

Both parts are needed. A layer with the right norm and wrong direction fails the cosine term. A layer with the right direction and weak magnitude fails the projection term. A dropped layer gives $\Delta_{\ell}^{(S)} \approx 0$, so its projection coefficient is close to zero. The per-layer distance therefore penalises missing, misdirected, and over-scaled computation.

4.3.3 Aggregation Across Layers

A candidate is selected as a whole architecture, so the layer distances need a single structural score. The aggregation below turns the sequence of per-layer errors into one trajectory distance:

$$d^{\text{traj}}(c) = \frac{1}{Z} \sum_{\ell=1}^L w_{\ell} d_{\ell}^{\text{traj}}(c), \quad Z = \sum_{\ell=1}^L w_{\ell}. \quad (4.29)$$

The weights w_{ℓ} determine how depth contributes to the structural score. The simplest choice is $w_{\ell} = 1$, which gives each layer equal influence. A magnitude-weighted variant uses a function of $\|\Delta_{\ell}^{(T)}\|_{\text{cal}}$, so layers where the teacher update is larger receive more weight. A late-layer-weighted variant can emphasise depths closer to the classifier. These choices change the search pressure, so the experiments report them as metric ablations.

The trajectory score is the first-stage rejector in the metric. Candidates with many missing, misdirected, or scale-wrong updates receive high d^{traj} . Candidates that preserve a coherent computation path receive lower d^{traj} . At this point the metric has identified which masks still perform a plausible internal computation; the next stage can then use final-output evidence to order those masks.

4.3.4 Task-Facing Distance After Structural Filtering

The structural score does not inspect the final label decision. The second ranking stage fills that gap with task-facing evidence from the final logits. This signal is useful after the update gate, because the surviving candidates have already preserved enough internal computation to make their logits meaningful.

Let $z_i^{(T)}$ and $z_i^{(S)}(c)$ be the teacher and student logits for input x_i . The distillation distance is

$$d^{\text{kd}}(c) = T^2 \frac{1}{B} \sum_{i=1}^B \text{KL} \left(\text{softmax}(z_i^{(T)}/T) \parallel \text{softmax}(z_i^{(S)}(c)/T) \right), \quad (4.30)$$

where T is the distillation temperature. The hard task-facing term is

$$d^{\text{ce}}(c) = -\frac{1}{B} \sum_{i=1}^B \log \text{softmax}(z_i^{(S)}(c))_{y_i^*}, \quad (4.31)$$

where y_i^* is either the true label or the teacher argmax label, depending on the calibration mode.

At high sparsity, many unrecovered candidates produce logits that are nearly uniform, biased toward one class, or unstable under small mask changes. In that regime, d^{kd} and d^{ce} can be similar for masks with different internal computations. This is the reason the search does not use the task distance as the first gate. The task distance is written as

$$d^{\text{task}}(c) = \beta_{\text{kd}} d^{\text{kd}}(c) + \beta_{\text{ce}} d^{\text{ce}}(c), \quad (4.32)$$

with non-negative coefficients. This distance is computed for every candidate in the cohort, then converted to a percentile rank. The final selection rule uses this task rank after the update-rank gate has selected the structurally acceptable candidates.

A low task distance from a collapsed candidate can occur for accidental reasons on a small calibration pack. A low task distance from a candidate with good residual updates is more meaningful, since the candidate has preserved a computation path capable of supporting recovery.

4.3.5 Two-Step Cohort Ranking

The search implementation uses the double-rank gate in `updt_gate` mode. This is the point where the two distances become the actual selection score. Raw trajectory and task distances have different units, ranges, and noise levels, so the implementation compares candidates by percentile ranks within the current cohort.

For any component d , define the lower-is-better percentile rank

$$r_d(c) = \frac{1}{\max(|\mathcal{C}_t| - 1, 1)} \text{rank}_{\mathcal{C}_t}(d(c)), \quad (4.33)$$

where ties receive their averaged rank and lower values are better. The two ranks used by the search are

$$r^{\text{traj}}(c) = r_{d^{\text{traj}}}(c), \quad r^{\text{task}}(c) = r_{d^{\text{task}}}(c).$$

Here r^{traj} comes from the trajectory distance and r^{task} comes from the task-facing distance. The ranking rule then uses them in a fixed order.

The first ranking step keeps candidates whose update rank is inside the retained fraction ρ :

$$\mathcal{S}_t = \{c \in \mathcal{C}_t : r^{\text{traj}}(c) \leq \rho\}, \quad \rho = \text{UPDT_GATE_KEEP_FRAC}.$$

This step implements the structural filter: candidates outside $\mathcal{S}_t$ have weaker update preservation than the retained fraction of the cohort. The second step orders the survivors by task rank, with a small update-rank tie-break:

$$\Phi(c) = \begin{cases} r^{\text{task}}(c) + \epsilon r^{\text{traj}}(c), & c \in \mathcal{S}_t, \\ 1 + r^{\text{traj}}(c), & c \notin \mathcal{S}_t. \end{cases} \quad (4.34)$$

The first branch is the survivor ordering used for selection. The second branch assigns every rejected candidate a score above one, placing it behind the survivor set. Rejected candidates still keep their update-rank order, so the genetic search retains pressure toward better structure even among candidates that fail the current gate.

4.3.6 Calibration Noise and Resampling

The two-step ranking depends on measurements from a finite calibration pack. A candidate near the update gate can change rank if the pack changes. The implementation reduces this variance by averaging component distances over independent packs before computing ranks. Let $\mathcal{P}_1, \dots, \mathcal{P}_R$ be independently sampled calibration packs. A resampled component is computed as

$$\bar{d}(c) = \frac{1}{R} \sum_{r=1}^R d_{\mathcal{P}_r}(c). \quad (4.35)$$

The same averaging applies to d^{traj} , d^{kd} , and d^{ce} before cohort ranks are computed.

The search uses two fidelity levels. Cheap evaluation scores many candidates on a small pack and drives broad exploration. Full evaluation averages over larger or multiple packs for candidates that may survive, enter the Hall of Fame, or be compared across generations. The cheap score gives speed. The full score reduces rank flips near the structural gate.

Resampling is especially important around κ . A small calibration perturbation can move a candidate from survivor to non-survivor. Averaging reduces these boundary errors and makes the selected population less dependent on one calibration draw.

4.3.7 Limits of the Surrogate

The surrogate ranks unrecovered candidates. It cannot guarantee recovered validation accuracy. Recovery can repair a candidate with a mediocre initial surrogate score, and recovery can fail on a candidate whose initial updates look good. The metric is therefore evaluated empirically by rank correlation with post-recovery validation accuracy in the later diagnostic experiments.

4.4 The Genetic Search Algorithm

The search stage receives the fixed mask representation, the compute budget, and the ranking metric defined in the preceding sections. Its role is to select one feasible architecture for recovery. The search cannot use gradients with respect to the binary masks, and exhaustive enumeration is infeasible. The procedure therefore uses an evolutionary search that keeps multiple feasible candidates alive, evaluates only a bounded number of masks, and returns the best mask found under the surrogate ranking.

The detailed presentation is written for encoder pruning. The decoder searches follow the same overall procedure, and decoder-specific changes are stated where they are needed.

4.4.1 The Level Database

The three decoder searches, namely quantization, low-rank factorisation, and rank-based recovery, rely on a *level database* built once before the search begins. The quantization database stores one compressed matrix and its size cost for each available bit-width. The low-rank database stores one factorised matrix and its parameter cost for each retained-rank level. The recovery database stores the residual-rank additions and their costs. A candidate is then assembled by selecting entries from the relevant database, so evaluation only requires a forward pass and feasibility can be checked from the stored costs. This follows the level-database idea used in EvoPress (evopress2024) and the sparsity-level database of DarwinLM (darwinlm2025).

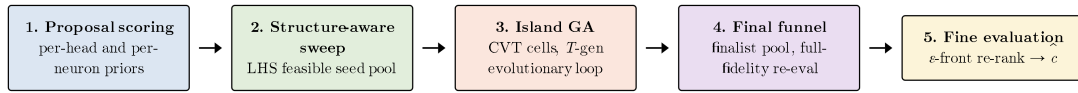


Figure 4.1: Five-phase overview of the genetic search. Phase 1 computes proposal scores from the teacher on the fixed evaluation subset. Phase 2 builds a feasible seed pool by a structure-aware sweep.

Phase 3 runs the island GA: a Centroidal Voronoi Tessellation (CVT) partition of $\mathcal{X}_B$ (du1999centroidal) and a T -generation evolutionary loop with archive updates. Phase 4 pools finalists and re-evaluates them at full fidelity. Phase 5 selects $\hat{c}$ from the pooled finalists.

4.4.2 Search Interface and Feasibility Objective

The previous sections define the pieces of the pruning problem in isolation. Of course, they need to be assembled into a concrete selection process, this is where the search stage comes in. It uses the representation, the budget model, and the ranking metric as fixed inputs to identify one feasible architecture for recovery.

Figure 4.2 summarizes the interface of the search stage. The search receives three fixed objects from the previous sections: the candidate representation, the target compute budget, and the surrogate metric. It returns a single mask $\hat{c}$ that satisfies the structural and computational constraints while minimizing the surrogate distance among the candidates explored by the search procedure.

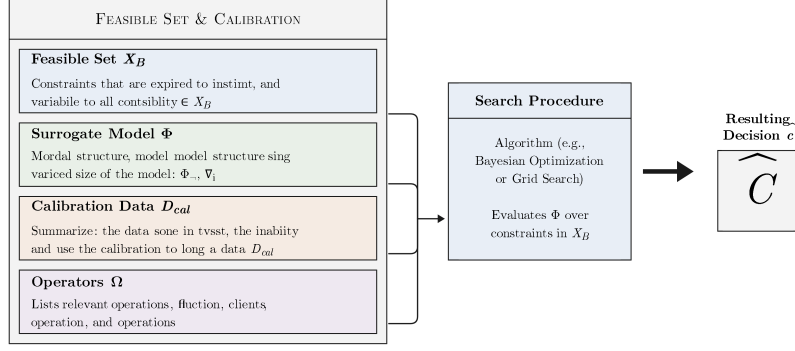


Figure 1: The main inputs are the feasible set X_B in λ . The Surrogate Model Φ is structured by model structure, model size, and relevant constraints in Ω .

Figure 2: Search Procedure: Algorithm c . Evaluates Φ over constraints in X_B .

Final Resulting Decision $\hat{c}$

Figure 4.2: Inputs and output of the search stage. The candidate representation, compute budget, and surrogate metric are fixed before search. The search procedure explores feasible candidates and returns one selected mask $\hat{c}$ for recovery.

For encoder pruning, feasibility is defined by two constraints: the operation cost must lie inside the target budget band, and the active depth must remain above a minimum value. The cost band admits the compact formulation below. A candidate

$$c = (M^H, \{n_l\}_{l=1}^L)$$

specifies a head mask and a per-layer neuron count, as defined in Section 4.1. Its operation cost $\text{MAC}(c)$ follows from these choices through the cost model of Section 4.2. The cost model restricts admissible candidates to the feasibility band

$$\mathcal{X}_B = \{c : (1 - \varepsilon)B \leq \text{MAC}(c) \leq (1 + \varepsilon)B\}.$$

Inside this band, the surrogate distance Φ of Section 4.3 ranks candidates against the teacher on the fixed metric subset. The active-depth floor

$$N_a(c) \geq L_{\min}$$

keeps at least $L_{\min}$ layers active. Equivalently, the complete feasible set can be written as

$$\mathcal{F}_B = \{c \in \mathcal{X}_B : N_a(c) \geq L_{\min}\}.$$

The search returns the mask that minimises the surrogate distance over this restricted feasible set:

$$\hat{c} \in \arg \min_{c \in \mathcal{F}_B} \Phi(c).$$

The objective above defines the mask selected by our method. To approximate this objective under the finite evaluation budget, we organize the search as a structured genetic procedure over the feasible band $\mathcal{F}_B$. The procedure is designed to maintain diversity across distinct regions of the budget-constrained space, enforce feasibility after candidate modifications, and reserve higher-fidelity evaluation for the most competitive masks. The next subsection details this procedure and describes how it produces the final selected mask $\hat{c}$.

4.4.3 The Five-Phase Genetic Procedure

The combinatorial size of $\mathcal{X}_B$ and the bounded evaluation budget shape the five-phase structure of the procedure. Phases 1 and 2 prepare the search state with per-head and per-neuron proposal scores and a feasible seed pool. Phase 3 runs the iterated evolutionary loop over regional populations, accumulating best candidates into three global archives. Phases 4 and 5 collapse the search state: a final funnel re-evaluates pooled finalists at full fidelity, and a fine-evaluation step selects $\hat{c}$ under an ϵ -front tie-break.

Phase 1: Proposal Scoring

Before any candidate is constructed, the procedure computes per-head and per-neuron Fisher-style local sensitivities of the teacher on the calibration set, denoted $s_H[\ell, h]$ and $s_N[\ell, n]$. The per-layer aggregate

$$\text{imp}(\ell) = \sum_h s_H[\ell, h] + \sum_n s_N[\ell, n]$$

is a layer importance prior. The proposal scores drive the mechanics of the encoder search. Since the search edits very fine-grained units, heads and neurons are sampled stochastically rather than by always deleting the lowest-scoring unit. The sampling probability is importance-weighted, so low-importance units still have a smaller chance of being selected, while the search keeps enough randomness to explore alternative masks. The same scores are also used when projecting a candidate back into the feasible band after an edit.

For the decoder searches, the architecture space is much larger and this head-and-neuron proposal stage is not used. Quantization, low-rank factorisation, and rank-based recovery instead use the prebuilt level databases of Section 4.4.1. Layer drop is the exception: it does not require a level database, because a candidate only selects which attention and feed-forward sub-blocks are kept or skipped.

Phase 2: Structure-Aware Sweep

The initial seed pool for encoder pruning is built by a Latin hypercube sweep (**mckay1979comparison**) over three axes: the active-layer count N_a , the fraction ϕ_H of the operation budget spent on attention heads, and a subset mode that decides how the active layers are drawn from $\text{imp}(\ell)$. Latin hypercube sampling is used here as a stratified design: each axis is divided into intervals, and the sweep draws combinations that cover the range of each axis without requiring a full grid. For each sampled cell, a candidate is constructed with the chosen N_a active layers and the chosen split of MAC between heads and neurons, then projected into $\mathcal{X}_B$. The sweep returns a pool of feasible candidates spread across coarse macro descriptors.

For the decoder searches, the seed pool is built directly from feasible level vectors, since the genomes are layer-wise bit, rank, recovery, or drop decisions rather than head-and-neuron masks. These feasible decoder candidates are then mapped to the same kind of low-dimensional descrip-

tor space used for the CVT partition. In both cases, the seed pool serves two roles in Phase 3: it initializes the regional populations and provides the sample from which $\mathcal{X}_B$ is partitioned into regions of similar candidates by a *Centroidal Voronoi Tessellation* (CVT) (du1999centroidal).

Phase 3: Island GA

In the third phase, multiple populations explore the feasible band $\mathcal{X}_B$ while each is rooted in a separate portion of the band. Thus local refinement and wider exploration may occur in parallel. Many feasible masks are close to each other: that is, they are only separated by minor budget-preserving edits such as moving active heads, relocating blocks of neurons in or between layers, or altering head–neuron balance such that the MAC cost remains almost the same. Such edits are consistent with preserving the candidate structure and are thus well-suited to local refinement.

Other masks which also fit within the MAC budget, might be constructed with capacity in different locations within the layers. For example, one mask could leave capacity mostly with early layers, one might favor later layers, or it might select entirely different layers to remain active. They correspond to points which may not be small perturbations of the same candidate but are instead independent exploration targets.

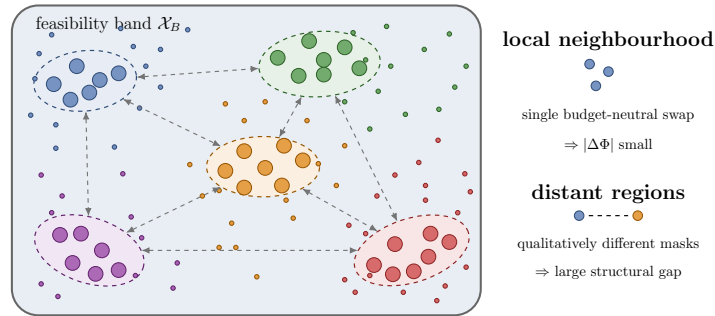


Figure 4.3: The feasibility band $\mathcal{X}_B$ as a populated space. Feasible candidates form clusters of architecturally similar masks; clusters in different regions of the band differ qualitatively.

The third phase encodes this two-scale structure through a *Centroidal Voronoi Tessellation* (CVT) of the feasible candidate space $\mathcal{X}_B$ (du1999centroidal). Each candidate is first mapped to a six-dimensional descriptor $z(c) \in [0, 1]^6$, and k -means is applied to the descriptor vectors from the seed pool. The descriptor is family-specific: encoder pruning uses head, neuron, active-depth, entropy, and depth-centre statistics, while the decoder searches use descriptors built from their level vectors. The full descriptor definitions are reported in Appendix .3. Each Voronoi cell defines an island’s basin; a population of size P is seeded inside each basin from the sweep pool of Phase 4.4.3, projected into the basin’s bounds. Local refinement is then carried out inside each island, while the partition itself enforces coverage across regions.

Within this partition, the loop runs for T generations (Figure 4.5). In each generation, per island: parents are chosen by a tournament rule whose score combines a fitness-shared rank (each rank divided by a structural-crowding count) with a novelty bonus equal to the minimum mask-distance to already-selected parents; offspring are produced by crossover and mutation

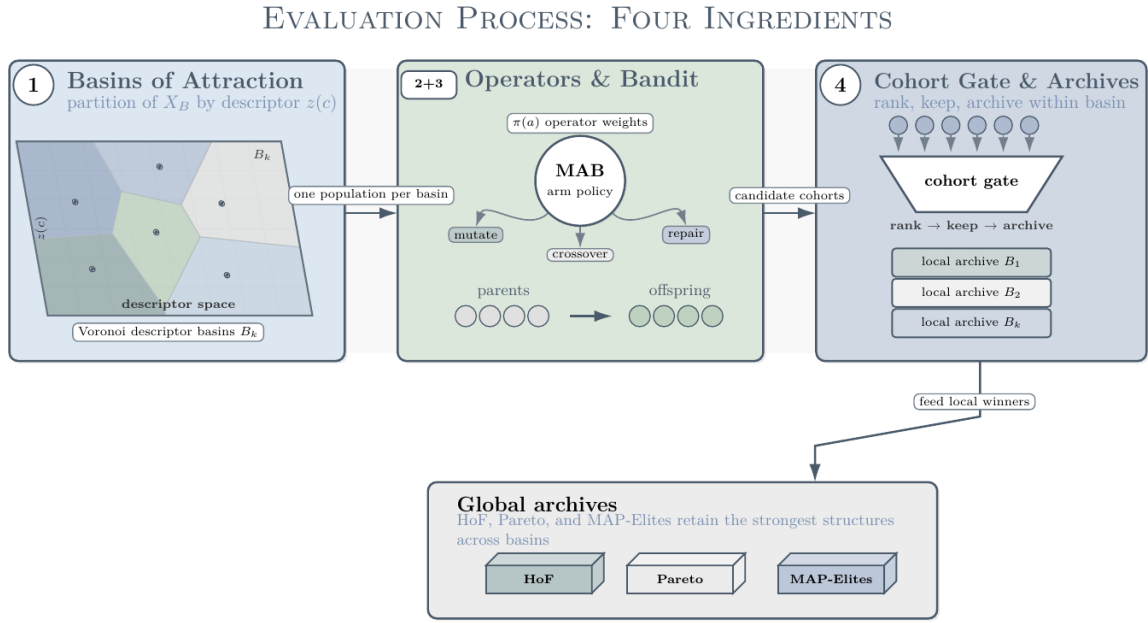


Figure 4.4: The four pieces of machinery inside the generational loop. Regions maintain separate populations, operators produce feasible offspring after repair, bandits adapt operator probabilities, and selection feeds both the next population and the archives.

operators drawn from an EXP3-IX bandit local to the island (Appendix .1); each offspring is projected back into the basin to enforce the budget band and the basin bounds; offspring are evaluated by Φ at cheap number of calibration tokens, with a scheduled subset re-evaluated at higher number of tokens; and the union of incumbents and offspring is ranked by the two-step cohort rule of Section 4.3.5. Survivors form the next-generation population. The best-scoring candidates of the cohort are inserted into three global archives: a Hall of Fame keyed by exact mask signature, a Pareto front on (Φ, MAC) , and a MAP-Elites grid keyed by (N_a, ϕ_H) .

Phase 4: Final Funnel

After T generations, the procedure assembles a finalist pool from four sources: the global best candidate observed across the run, the top candidates of each island’s final population, samples from the three archives (Hall of Fame, Pareto front, MAP-Elites), and the global best running incumbent (Figure 4.5). The pool is deduplicated by exact mask signature. Each finalist is then re-evaluated on the full-fidelity calibration pack with R independent resamples averaged into a single score.

Phase 5: Fine Evaluation

The averaged finalist scores define an ϵ -front: the candidates whose resampled Φ lies within ϵ of the lowest score in the finalist pool. The ϵ -front is sorted lexicographically by $(\Phi, -\text{MAC})$,

so among candidates within calibration noise of one another the procedure prefers the one that uses more of the allowed compute. The first element of the sort is returned as $\hat{c}$ (Figure 4.5).

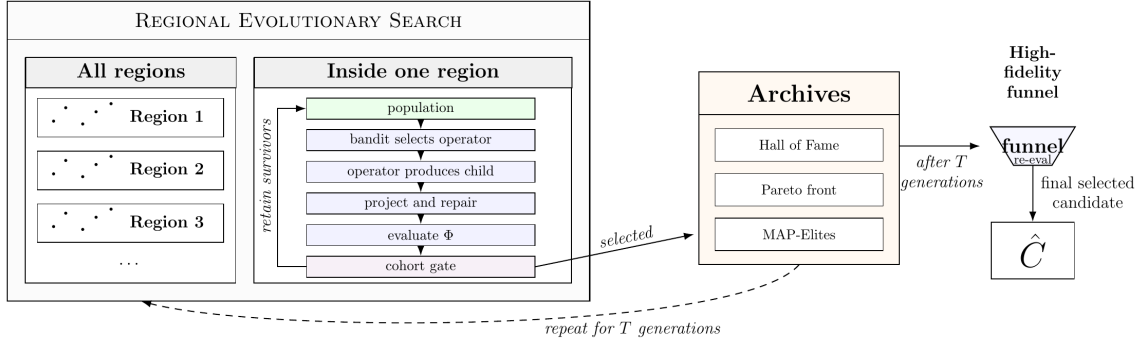


Figure 4.5: One iteration of the search and the final funnel. Each region’s population produces offspring with operators chosen by its bandit; offspring are projected back into the region, evaluated on the calibration packs, ranked by the cohort gate, and used to update the population and the archives. After T iterations, the archives and the per-region best candidates are re-evaluated at higher fidelity and one mask $\hat{c}$ is returned.

4.5 The Detailed Pipeline

4.5.1 Proposal Scoring



The first phase computes proposal scores before any candidate mask is sampled. These local saliency estimates make later discrete edits less arbitrary. The intuition follows classical pruning saliency: a unit is valuable when a small change to its gate would strongly affect the objective, or when removing it directly increases reconstruction error (**lecun1989optimal**; **hassibi1993second**; **molchanov2017variational**; **michel2022not**). Here the saliency is computed for heads and feed-forward neurons against the teacher’s per-layer residual update target on the fixed metric subset.

For head h of layer ℓ , let $g_{\ell,h}(x)$ denote the gradient of the teacher-forced per-layer update target with respect to a multiplicative gate placed at head (ℓ, h) for calibration example x . This gradient measures local sensitivity: if the squared gradient is large, changing the gate of that head is expected to disturb the layer update. The head proposal score combines this local sensitivity with an explicit zero-out reconstruction term:

$$s_H[\ell, h] = \mathbb{E}_{x \sim \mathcal{D}_{\text{cal}}} [g_{\ell,h}(x)^2] + \lambda \cdot \text{MSE} \left(\tilde{\Delta}_{\ell}^{(T, h=0)}, \Delta_{\ell}^{(T)} \right),$$

where λ is a fixed mixing coefficient. The MSE term measures how much the teacher’s per-layer update changes when head (ℓ, h) is zeroed. The neuron score $s_N[\ell, n]$ is defined analogously by placing the gate on feed-forward neuron (ℓ, n) . Both scores are row-normalised per layer, so they rank units within the same layer rather than comparing raw magnitudes across layers.

The search also needs a layer-level score because the sweep and the repair routine sometimes decide which layers should remain active before they decide which exact units to keep. The per-layer aggregate

$$\text{imp}(\ell) = \sum_h s_H[\ell, h] + \sum_n s_N[\ell, n]$$

gives this single importance number per layer. Phase 2 uses it when choosing active layers, and repair uses it to define the protected core.

Neuron mutation and repair operate on blocks rather than individual neurons. To make block addition and removal efficient, each layer receives a cumulative block table. For layer ℓ , let π_ℓ be the permutation that sorts $s_N[\ell, \cdot]$ in descending order. The cumulative score of the first b neuron blocks is

$$S_\ell[b] = \sum_{j=1}^{b \cdot b_n} s_N[\ell, \pi_\ell(j)], \quad b = 0, 1, \dots, \lfloor N/b_n \rfloor.$$

The marginal gain of adding the next block to layer ℓ is $S_\ell[b+1] - S_\ell[b]$, and the marginal loss of removing the current last block is $S_\ell[b] - S_\ell[b-1]$. After the one-time descending sort, both queries are constant-time. This matters during mutation and repair because many candidate edits require quick decisions about where to add or remove neuron capacity.

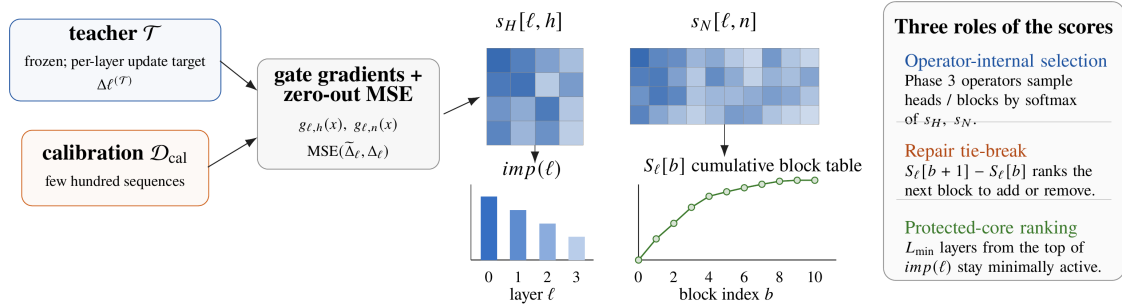


Figure 4.6: Proposal scoring. For each calibration batch, gradients of the teacher’s per-layer update target with respect to head and neuron gates yield squared sensitivities and a reconstruction MSE; their combination, row-normalised per layer, produces $s_H[\ell, h]$ and $s_N[\ell, n]$. Summing over each layer gives $\text{imp}(\ell)$. Sorting $s_N[\ell, \cdot]$ in descending order and accumulating block sums yields the table $S_\ell[b]$, from which marginal gain and loss queries take $O(1)$.

The scores then feed three parts of the search. The mutation operators of Phase 3 use s_H and s_N to define sampling probabilities when heads or neuron blocks are added, removed, or swapped. The repair routine uses S_ℓ to choose the best block to add or the least damaging block to remove when a candidate must be projected back into the feasible band. The guarded core

during repair is formed from the $L_{\min}$ most important layers, so that repair cannot deactivate every high-priority layer while satisfying the budget.

4.5.2 Initialisation: Structure-Aware Sweep



Proposal scores say which units look locally promising, but they do not yet give a diverse set of feasible architectures to search from. Phase 2 builds that set: a pool of masks inside $\mathcal{X}_B$ spread across qualitatively different allocations, since the island partition of Section 4.5.3 is fitted and seeded from it. Coverage comes from a stratified Latin hypercube design (**mckay1979comparison**) over three macro axes, with each sampled point turned into one candidate.

The axes are the active-layer count $N_a \in \{L_{\min}, \dots, L_{\max}\}$, the head-MAC fraction ϕ_H that splits a layer’s budget between attention and feed forward, and a subset mode that picks which N_a layers stay active from the importance vector $\text{imp}(\ell)$, ranging from strictly top-scoring to uniformly random with an optional single-branch variant. The depth bound is the most minimally complete layers the budget allows,

$$L_{\max} = \min \left(L, \left\lfloor \frac{B}{c_H + b_n c_N} \right\rfloor \right),$$

where $c_H + b_n c_N$ is the cost of one layer carrying a single attention head and a single block of feed-forward neurons.

Each design point becomes a mask in one pass, as summarised in Figure 4.7. The subset mode fixes the active set A ; the budget is divided across A in proportion to importance-weighted shares $\tilde{w}_\ell$, with a random perturbation on each weight so that repeated samples under the same prior still differ; and ϕ_H converts each layer’s share into an integer head count and a block-aligned neuron count,

$$a_\ell = \text{round} \left(\frac{\phi_H B \tilde{w}_\ell}{c_H} \right), \quad n_\ell = b_n \left\lfloor \frac{(1 - \phi_H) B \tilde{w}_\ell}{b_n c_N} \right\rfloor.$$

The retained head identities are sampled from $s_H[\ell, \cdot]$, and layers outside A stay empty. Rounding can push the cost just outside the band, so a budget-repair step adds or removes block-aligned structures by proposal score until the candidate lies in $\mathcal{X}_B$. Repeating this over all samples gives the feasible seed pool used by the partition fit of the next phase (**du1999centroidal**).

The Latin hypercube is used only on the encoder, whose genome exposes a few interpretable meta-axes that can be stratified directly. The decoder genomes instead carry one axis per layer

(Section 4.1.4), around ninety discrete dimensions for the bit and rank vectors, where stratified sampling would need exponentially many full forward passes. Those searches stratify in the six-dimensional descriptor space through the CVT partition of Section 4.5.3, which is cheap and adapts to where feasible genomes actually fall.

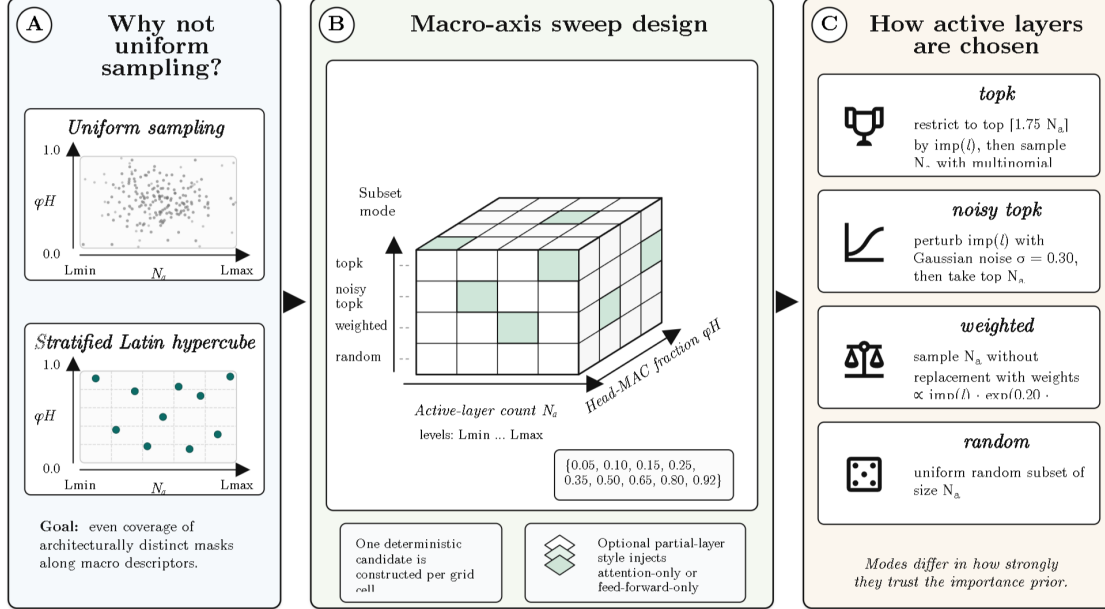
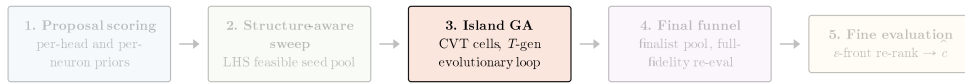


Figure 4.7: Structure-aware sweep. Each sampled design point $(N_a, \phi_H, \text{mode})$ yields one candidate. The subset mode chooses the active layers from the layer importance scores; the head–neuron split ϕ_H divides the budget; and the importance-weighted per-layer allocation distributes the split into integer head counts and block-aligned neuron counts.

4.5.3 CVT Partition and Island Construction



The sweep defined in Section 4.5.2 results in a scored feasible pool $\mathcal{S} = \{x_i\}_{i=1}^M$, where each x_i lies within the budget band and has a surrogate distance $\Phi(x_i)$ from the teacher (Section 4.3), with lower values being better. Sorting by quality alone is not sufficient to start the local search, because the top candidates can belong to different allocation regimes: some spend more budget on attention, some keep more feed-forward width, and some concentrate the budget into fewer active layers. A head swap or neuron transfer affects these regimes differently, so one mixed population can push edits that benefit one regime while harming another. Phase 3 therefore separates candidates by architecture and gives each region its own local population, or island

(cantupaz2000efficient). Each island carries its own best solution so far and its own record of which edits have been working.

Architectural Descriptor. To group candidates by shape, each candidate is mapped to a six-dimensional descriptor vector, $z(x) = (z_1(x), \dots, z_6(x)) \in [0, 1]^6$. For encoder pruning, Table 4.3 lists what each coordinate measures: how the budget splits between attention and feed forward, how many layers keep both branches active, how evenly the budget is spread across depth, and how high or low in the stack the attention and feed-forward capacity sits. For decoder searches, the same role is played by family-specific descriptors built from the layer-drop, bit-width, low-rank, or recovery level vectors. The closed-form expressions for the encoder and decoder descriptors are given in Appendix .3.

This descriptor projects each possible candidate to a point in the six-dimensional shape space described above. That point is not yet a search region. It only says where the candidate sits relative to other candidates in terms of allocation, active depth, budget spread, and depth placement of capacity. The next step is therefore to cluster the descriptor cloud into a small set of regions whose members are structurally close.

Coordinate	Quantity	Interpretation
z_1	attention MAC share	separates head-heavy from neuron-heavy allocations
z_2	surviving feed-forward width	measures how much dense feed-forward capacity remains
z_3	complete active-layer fraction	measures how much depth retains both branches
z_4	budget entropy across layers	distinguishes spread-out and concentrated allocations
z_5	head centre of mass	locates attention capacity along depth
z_6	neuron centre of mass	locates feed-forward capacity along depth

Table 4.3: Six encoder descriptor coordinates used to partition the feasible sweep pool.

Descriptor Standardisation and CVT Fitting. The descriptor now gives a common language for architecture shape. Before distance-based clustering is applied, the descriptor coordinates are standardised because their empirical spreads can differ across the sweep pool. For every candidate $x_i \in \mathcal{S}$,

$$\tilde{z}_i = \frac{z(x_i) - \bar{z}}{\sigma_z + \epsilon},$$

where $z(x_i)$ is the descriptor of sweep candidate x_i , $\bar{z}$ is the componentwise sample mean, σ_z is the componentwise sample standard deviation, ϵ avoids division by zero, and $\tilde{z}_i$ is the standardised descriptor used for clustering. Standardisation places all six coordinates on a common numerical scale.

The standardised descriptors are then partitioned by a *Centroidal Voronoi Tessellation* (CVT) (du1999centroidal). The fitting procedure uses farthest-point seeding followed by Lloyd iterations, as in k -means clustering (lloyd1982least; macqueen1967classification). The first centre $\tilde{\mu}_1$ is drawn uniformly from the standardised seed descriptors $\{\tilde{z}_i\}$. Each following centre is chosen from the part of the descriptor cloud farthest from the centres already selected:

$$\tilde{\mu}_{j+1} := \arg \max_{\tilde{z}_i} \min_{1 \leq q \leq j} \|\tilde{z}_i - \tilde{\mu}_q\|_2^2.$$

This farthest-point construction places each new centre in the part of descriptor space least covered by the centres already selected. Lloyd’s assignment step then maps each descriptor to its closest centre,

$$\text{label}(i) := \arg \min_{1 \leq j \leq K} \|\tilde{z}_i - \tilde{\mu}_j\|_2^2,$$

and the update step replaces each centre by the mean descriptor assigned to it:

$$\tilde{\mu}_j \leftarrow \frac{1}{|C_j|} \sum_{i \in C_j} \tilde{z}_i, \quad C_j = \{i : \text{label}(i) = j\}.$$

Assignment and update are repeated until the labels stop changing. The final centres are mapped back to the original descriptor coordinates by

$$\mu_j = (\sigma_z + \epsilon)\tilde{\mu}_j + \bar{z}.$$

The resulting cells are nearest-centre regions in descriptor space:

$$V_j := \{z \in [0, 1]^6 : \|z - \mu_j\|_2 \leq \|z - \mu_q\|_2 \text{ for all } q \neq j\}.$$

Each cell groups candidates with similar allocation shape. These cells provide the regions from which island populations are constructed.

Island Seeding. With the CVT cells fixed, each cell receives its own island population. The candidates assigned to cell j are sorted by the surrogate distance Φ defined in Section 4.3, where lower values are better. The strongest feasible candidates are copied into that island. If a cell contains too few candidates, additional candidates are taken from nearby cells to complete the initial population. Each island then carries the running state

$$\mathcal{I}_j = (\mathcal{P}_j, x_j^*, d_j^*, \mathcal{A}_j),$$

namely its population $\mathcal{P}_j$ of P feasible masks, the best mask x_j^* found in the cell so far with its distance d_j^* , and a selector $\mathcal{A}_j$ that learns which edit operator to try next in this cell from how much each operator improved candidates in the past (**auer2002nonstochastic**; **neu2015explore**). Phase 3 therefore begins with one population per cell, each one starting from a different way of spending the same budget (Figure 4.8).

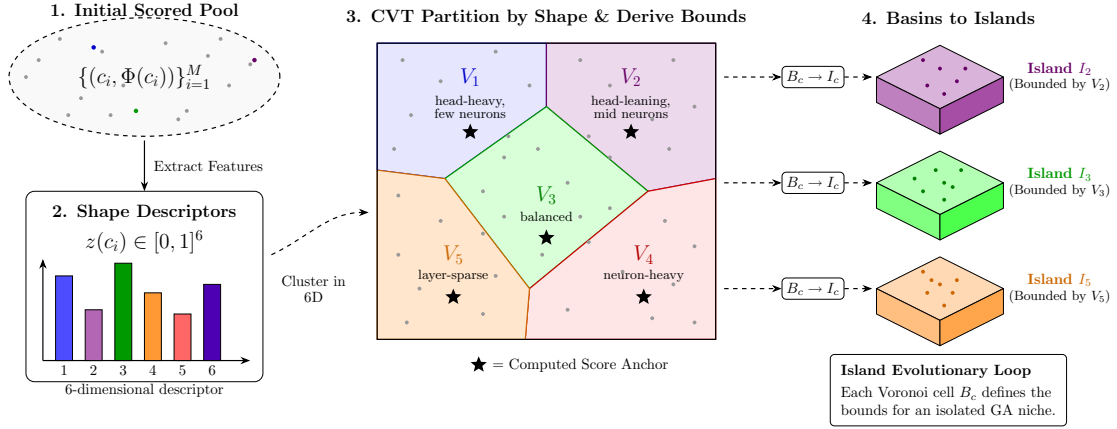
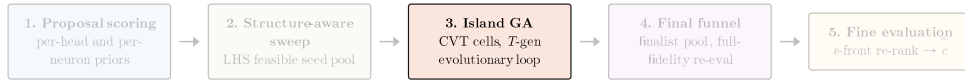


Figure 4.8: From scored sweep candidates to islands. Sweep candidates are projected into the (z_1, z_3) plane on the left and form a continuous cloud. The centre-based clustering step assigns each descriptor to its nearest centre and updates each centre to the mean of its assigned descriptors. This produces Voronoi cells. Each cell becomes a basin B_j on the right; the best scored candidates assigned to that basin seed the island I_j , and the island then evolves with its own population, best-so-far candidate, and operator-selection bandit.

4.5.4 Operator Library



Now that we have the cells and populations set up, we need to explore the controlled variation on each island. The search utilizes two mutation scales. First is a local, small edit which refines a candidate within the current basin of attraction. Local edits will tend to preserve the global architecture, swapping units of similar cost. The second mutation size is structural, which changes the pattern of allocation (for example changing the active depth or the distribution of computation). This way an island can refine a local mask but also explore broader architectures as local refinement yields no new off-spring.

A Micro mutation is a local exchange of one unit for another, for example a budget-neutral head swap in the structural pruning of the encoder takes out one active head and adds in another head of the same cost, of course as mentioned before the proposal scores of the encoder model s_H guides what head is added/ removed with higher chances for low importance units to be removed. Macro mutation involves a larger structural change, for example a layer-rewiring mutation switches between the set of active layers whilst remaining within the budget and active-depth constraint after projection.

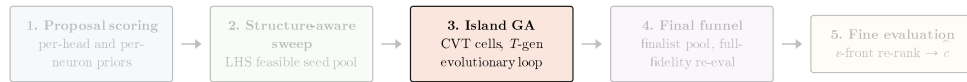
The decoder searches reuse this two-scale design, with each task carrying its own family of operators. As one example from each, the layer-drop family can swap a dropped sub-block for

a kept one, the quantization family can swap bit-width between two layers, and the low-rank family can swap rank between two layers. The full family for each task, encoder and decoder, is listed in Appendix .4.

Crossover is another operators used. Mutation changes one of the parents into another; on the contrary, crossover takes two parents to produce one offspring. The function of crossover is to recombine beneficial layer assignments discovered in two different contexts. One parent can provide the active layer topology while the other may provide branch widths or layer wise count and the produced offspring is then mapped back to the basin, since it can be mapped to a valid looking mask whose cost or basin descriptor might be out of range.

When generating an offspring from existing parents, the algorithm either uses mutation or crossover, according to the operator mode. If it is mutation, then the bandit described in Section 4.5.5 is used to select an operator from the corresponding family. If it is crossover, then two parents are merged and the child is passed through the same feasibility projection used for generated offspring before evaluation. In this sense, the operator set contains the set of local or structural modifications; crossover provides recombination between already existing candidates.

4.5.5 Bandit and Niche Conditioning



Once the operator families are defined, each island must decide which move to apply when producing offspring. This is treated as an operator-selection problem, with one EXP3-IX bandit per island ([auer2002nonstochastic](#); [neu2015explore](#)). For island i , the bandit stores a positive weight $w_i[o]$ for each operator o , and these weights define the base sampling distribution

$$p_i[o] = (1 - \gamma) \frac{w_i[o]}{\sum_{o'} w_i[o']} + \frac{\gamma}{K},$$

where K is the number of available operators and γ is the exploration coefficient. The learned term favours operators with larger accumulated weight; the uniform term keeps every operator selectable. After an offspring is evaluated, only the sampled operator receives reward, so the update uses the implicit-exploration estimate

$$\hat{r}_o = \frac{r_o}{\hat{p} + \gamma}.$$

Here, $\hat{p}$ is the probability used to sample operator o . The island then applies the multiplicative update

$$w_i[o] \leftarrow w_i[o] \exp\left(\frac{\gamma \hat{r}_o}{K}\right).$$

This is the EXP3-IX update. The extra γ in the denominator limits the size of reward estimates for rarely sampled operators, and weights are periodically rescaled for numerical stability.

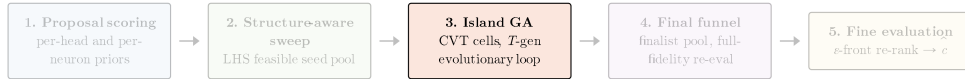
The initial weights encode the cell focus obtained in Section 4.5.3: depth-changing, attention-shifting, feed-forward-shifting, and balanced operators can start with different priors depending on the cell centroid. These priors only initialise the search; the multiplicative updates can override them once the observed rewards disagree.

For encoder pruning, the island-level bandit is further conditioned by a coarse parent niche because the head–neuron mask space can collapse toward repeated active-depth and allocation patterns. The niche is $\nu(c) = (N_a, \text{mac_bucket}, \text{shape_bucket})$: active depth, relative-cost bucket $\text{MAC}(c)/B$, and head-versus-neuron utilisation bucket. For each pair (ν, o) , the island maintains the exponentially weighted reward average $\text{ema}_i[\nu, o] \leftarrow (1 - \eta_{\text{ema}})\text{ema}_i[\nu, o] + \eta_{\text{ema}}r_o$. When a parent has niche ν , the base probability $p_i[o]$ is reweighted by this niche statistic and renormalised:

$$\tilde{p}_i[o; \nu] \propto p_i[o] (0.5 + \text{ema}_i[\nu, o]), \quad \text{ema}_i[\nu, o] \in [0, 1].$$

The resulting distribution $\tilde{p}_i[\cdot; \nu]$ combines island-level evidence with parent-specific evidence. The reward r_o is assigned after the offspring is compared with its parent by the two-step cohort rule of Section 4.3.5; a small extra bonus is added when the offspring also improves the island’s best-so-far candidate.

4.5.6 The Generation Loop



With the islands initialised and the operator distributions in place, the search enters the main generational loop of Phase 3. One generation updates every island once. The loop has a fixed order: score the current population, select parents, preserve elites, generate offspring, project them into the basin, evaluate them, apply the cohort rule, update the population, and pass strong candidates to the archives of Section 4.5.7.

Population Scoring. At the start of generation t , island i holds a population

$$\mathcal{P}_i^{(t)} = \{c_{i,1}, \dots, c_{i,P}\},$$

all of whose members satisfy the basin constraints of Section 4.5.3. The loop first evaluates every candidate at cheap fidelity on the calibration packs, producing a surrogate distance $\Phi(c_{i,j})$. Since lower distance is better, the distance is converted into a maximisation-oriented raw fitness

$$f_{i,j} = -\Phi(c_{i,j}).$$

This raw fitness ranks candidates by quality inside the current island. Before parent selection, it is corrected for crowding so that a dense group of near-identical masks does not dominate the parent pool.

The correction is a standard fitness-sharing rule. The distance D_{jk} is the normalised Hamming distance between the exact binary mask signatures of candidates $c_{i,j}$ and $c_{i,k}$ in the same island, and σ_s is the sharing radius. The shared fitness of candidate j is

$$\tilde{f}_{i,j} = \frac{f_{i,j}}{1 + \sum_{k \neq j} \max\left(0, 1 - \frac{D_{jk}}{\sigma_s}\right)}.$$

Candidates surrounded by many near-duplicates receive lower selection strength. More isolated candidates retain more of their raw fitness. This keeps the island from collapsing too quickly onto one local shape.

Parent Selection and Elites. Parent selection is then applied on top of the shared fitness. The candidate pool is formed from the highest shared-fitness members of the island. Parents are drawn by repeated tournament selection. If the algorithm has already chosen parents $p_1, \dots, p_{k-1}$ for the current offspring batch, then a tournament candidate $c_{i,j}$ receives the score

$$S_{i,j}^{\text{parent}} = \tilde{f}_{i,j} + \alpha_{\text{nov}} \min_{k' < k} D_{j,p_{k'}} + \alpha_{\text{tie}} u, \quad u \sim \text{Unif}[0, 1].$$

The second term is a novelty bonus weighted by α_{nov} ; among candidates with similar shared fitness, it favours candidates far from the parents already chosen for the same batch. The final term is a small random tie-break weighted by α_{tie} , used only to avoid deterministic cycles.

Before any offspring are created, the island preserves a small elite set unchanged. If the global elite budget is `elite_size` and the number of islands is I , then island i copies forward

$$e = \left\lceil \frac{\text{elite_size}}{I} \right\rceil$$

top candidates from $\mathcal{P}_i^{(t)}$ directly into the next population. The remaining slots are filled with offspring. The fraction of those offspring allocated to macro operators is controlled by a schedule $\beta_M(t)$. Early generations emphasise broader structural edits. Later generations shift probability mass toward micro operators once each island has established its local region.

Offspring Generation and Evaluation. Each remaining population slot is filled independently. The bandit of Section 4.5.5 samples an operator with the niche-conditioned distribution $\tilde{p}_i[o; \nu]$ of the selected parent. A crossover-or-mutation decision then determines whether the child is produced from one parent by the sampled mutation operator or from two parents by crossover. In both cases, the child is projected back into the basin so that it satisfies the budget band and the island bounds.

For the decoder searches, this projection edits only the discrete level vector: dropped sub-block flags, bit-width levels, rank levels, or residual-rank allocations. The candidate is then

materialised from the level database of Section 4.4.1, so evaluation loads precomputed weights or residual corrections rather than running quantization or SVD inside the search loop.

Before scoring, the offspring batch is filtered for repeated masks. Exact duplicates are removed by exact mask signature. A second pass removes structural near-duplicates using the same normalised mask distance and a deduplication radius σ_d . This prevents the offspring cohort from spending evaluations on masks that differ only by negligible local perturbations.

Every surviving offspring is evaluated at cheap fidelity. At scheduled generations, the strongest cheap-fidelity offspring are re-evaluated at full fidelity. If $\mathcal{D}_{\text{cal}}^{\text{full},r}$ denotes the r -th full calibration resample and R the resample count, then the full-fidelity score used in the loop is

$$\Phi^{\text{full}}(c) = \frac{1}{R} \sum_{r=1}^R \Phi(c; \mathcal{D}_{\text{cal}}^{\text{full},r}).$$

The cheap score remains the working ranking signal inside the generation. The full-fidelity pass provides a scheduled correction for candidates that already look competitive under the cheaper estimate.

Cohort Replacement and Rewards. The island then forms a cohort from incumbents and offspring and applies the two-step cohort rule of Section 4.3.5. Let

$$\mathcal{C}_i^{(t)} = \mathcal{P}_i^{(t)} \cup \mathcal{O}_i^{(t)}$$

denote this combined set. The gate first removes candidates whose structural metric is unacceptable, then ranks the survivors with the task-facing terms defined by the surrogate metric. The next population $\mathcal{P}_i^{(t+1)}$ is filled from this gated ranking after elite preservation. The same ranked cohort also supplies candidates for the archives described in Section 4.5.7.

The same gated score also defines the reward signal returned to the bandit. If operator o produced offspring c from parent c^{par} , then its base reward is

$$r_o = \mathbf{1}[\Phi^{\text{gate}}(c) < \Phi^{\text{gate}}(c^{\text{par}})].$$

A small additive bonus is applied when the offspring also improves the island best c_i^* . The operator-selection rule is therefore rewarded for producing children that survive the same gated ranking used for population replacement.

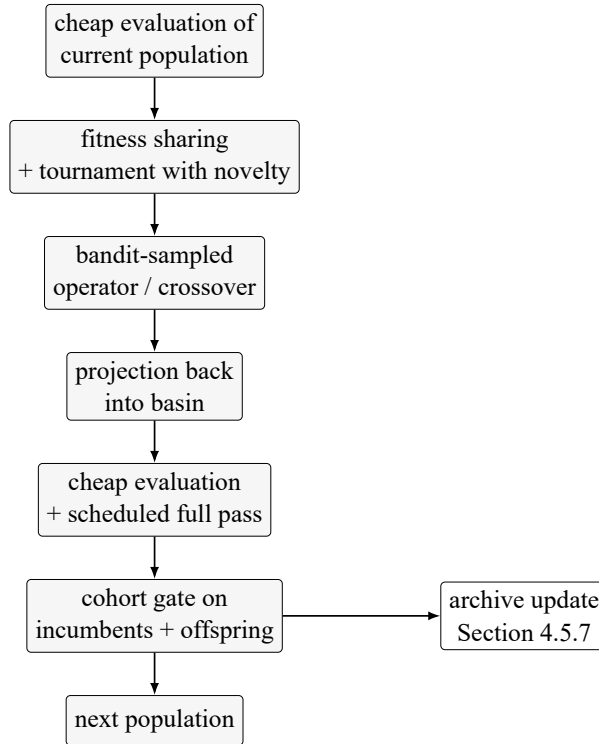
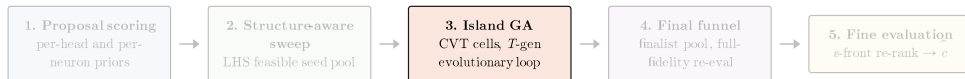


Figure 4.9: One generation of Phase 3 inside a single island. The current population is scored, parents are selected with crowding correction and novelty, offspring are produced and projected back into the basin, the combined cohort is ranked by the two-step cohort rule of Section 4.3.5, and the surviving candidates form the next population. Archive updates use the same ranked cohort and are detailed in Section 4.5.7.

4.5.7 Archives

The generation loop updates the current population, but the search also preserves strong candidates across generations in three global archives. These archives have different purposes: one stores exact elites, one stores the non-dominated cost–distance frontier, and one stores representatives spread over a coarse descriptor grid.



The Hall of Fame, denoted $\mathcal{H}$, is the strict elite memory. It has capacity 50 and is deduplicated by exact mask signature. To prevent the archive from being monopolised by one macro pattern, it also enforces a per-signature cap of three entries for each macro signature. Here the macro signature is the pair formed by the active-layer set and the rounded head-MAC fraction,

$$\text{macro}(c) = (\{\ell : a_\ell + n_\ell > 0\}, \text{round}(\phi_H(c), 0.1)).$$

Candidates enter $\mathcal{H}$ only if they improve on existing members under the gated search score and pass the exact-signature deduplication.

The Pareto archive, denoted $\mathcal{P}$, tracks the non-dominated frontier on the pair (Φ, MAC) . A candidate c_a dominates c_b if

$$\Phi(c_a) \leq \Phi(c_b), \quad \text{MAC}(c_a) \leq \text{MAC}(c_b),$$

and at least one of these inequalities is strict. The archive stores only non-dominated candidates and is capped at size 100. This archive preserves alternatives that are not globally best in distance but remain competitive at a different point inside the budget band.

The MAP-Elites archive, denoted $\mathcal{M}$, stores one best-distance candidate per cell of a coarse grid on

$$(N_a, \text{round}(\phi_H, 0.1)).$$

If a candidate lands in a grid cell already occupied by another candidate, only the lower-distance one is kept. This archive preserves coverage across coarse depth and head-allocation patterns even when those patterns are not currently winning inside the islands.

4.5.8 Final Funnel



Once the generation loop finishes, the search no longer needs to maintain island populations. The remaining task is to gather the strongest candidates produced by the run into one finalist set and re-evaluate them at higher fidelity.

The finalist pool is assembled from the final incumbent, the top candidates of each island, and samples from the three archives. Writing $\hat{c}^{(T)}$ for the best candidate observed by the end of generation T , the pool is

$$\mathcal{F} = \{\hat{c}^{(T)}\} \cup \bigcup_i \text{top}_3(\mathcal{P}_i) \cup \text{sample}_5(\mathcal{H}) \cup \text{sample}_5(\mathcal{P}) \cup \text{sample}_5(\mathcal{M}).$$

The union is then deduplicated by exact mask signature, so every finalist corresponds to a distinct binary architecture.

Each finalist is re-evaluated on the full calibration packs with $R_{\text{final}} = 4$ independent resamples. The final score used in the funnel is

$$\Phi^{\text{final}}(c) = \frac{1}{R_{\text{final}}} \sum_{r=1}^{R_{\text{final}}} \Phi(c; \mathcal{D}_{\text{cal}}^{\text{full}, r}).$$

This final pass is more expensive than the generation-time evaluations, but it is applied only to the small finalist set that survives the funnel.

4.5.9 Fine Evaluation and Selection



In the last pass, the distances are measured across many of the calibration tokens. Since the closest candidates can only be separated up to calibration noise, the winner is chosen among the ϵ front.

Let

$$d^* = \min_{c \in \mathcal{F}} \Phi^{\text{final}}(c).$$

The ϵ -front is

$$\mathcal{F}_\epsilon = \{c \in \mathcal{F} : \Phi^{\text{final}}(c) \leq d^* + \epsilon\}, \quad \epsilon = 0.005.$$

This keeps all finalists whose high-fidelity score is effectively tied with the best observed one.

The candidates in $\mathcal{F}_\epsilon$ are then sorted lexicographically by

$$(\Phi^{\text{final}}(c), -B_{\text{use}}(c)).$$

The first key prefers smaller final distance. The second key prefers the candidate that uses more of the allowed budget whenever two candidates are indistinguishable in final distance at the ϵ scale. Here $B_{\text{use}}(c)$ denotes the used budget: MAC cost for encoder pruning, and the corresponding budget measure of the active decoder compression family. The selected architecture $\hat{c}$ is the first element of this lexicographic order.

The procedure therefore ends with a single selected architecture $\hat{c}$ obtained from the budget-constrained search. This completes the search stage itself. The selected architecture records the final compression decisions, but it does not by itself adapt the surviving parameters to that compressed form. When recovery is used, it is applied after search under the fixed architecture returned by the search.

4.6 Recovery Process (Encoder)

For encoder pruning, the search returns a fixed pruned architecture $\hat{c} = (\mathbf{M}^H, \mathbf{M}^N)$. At high sparsity, this architecture usually requires parameter recovery before evaluation: the surviving heads and neurons inherit the teacher parameters, whereas the residual pathway is altered by the removed components. Recovery therefore trains the selected masked student while keeping the architecture fixed.

The recovery objective combines representation-level and prediction-level supervision. These objectives impose distinct constraints on the compressed model. Hidden-state matching aligns the internal trajectory with the teacher computation, whereas label and logit supervision align the final predictive distribution. Under severe compression, the available capacity

may be insufficient to optimise both objectives effectively from the initial step. The recovery procedure is therefore staged. The first stage uses intermediate representation matching to stabilise the surviving layers. The second stage shifts the optimisation target to prediction-level distillation and task supervision. This follows the general principle of knowledge distillation (hinton2015distilling) and the use of intermediate supervision in compact language models (jiao2020tinybert).

Figure 4.10 summarises this recovery logic. It places the fixed mask before all optimisation steps, then separates internal representation recovery from final prediction recovery.

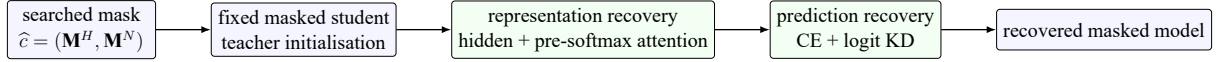


Figure 4.10: Recovery flow after architecture search. The mask is fixed throughout recovery; only the surviving parameters are trained.

The left-to-right order in Figure 4.10 fixes the implementation order used by the recovery process. The student parameters are initialised from the teacher. The head and neuron masks are hardened to binary tensors and are not optimised. Fully dropped layers are bypassed as identities during masked training. The classifier is frozen during the representation stage and unfrozen for prediction recovery. The procedure therefore separates architecture selection from parameter recovery: search chooses the structure, and the recovery process trains the selected structure.

4.6.1 Intermediate Representation Recovery

The first recovery stage trains the masked student before the classifier is allowed to adapt. Its purpose is to restore an internal trajectory that can support subsequent prediction training. If prediction loss is applied immediately, classifier gradients are propagated through representations that may already be poorly aligned with the teacher. Hidden-state supervision gives the surviving layers a direct target and supplies a denser signal than the task label alone.

Let the teacher hidden states be

$$h_0^{(\mathcal{T})}(x), h_1^{(\mathcal{T})}(x), \dots, h_L^{(\mathcal{T})}(x),$$

where $h_0^{(\mathcal{T})}$ is the embedding output. Let $\mathcal{S}(\hat{c})$ be the ordered set of active student layers under the selected mask. A fixed set of teacher anchor depths $\mathcal{A}$ is chosen along the teacher depth. The mapping

$$\pi : \mathcal{S}(\hat{c}) \rightarrow \mathcal{A}$$

assigns each active student layer to a teacher anchor according to its relative depth. This mapping is dynamic because the active student layers depend on the searched architecture. A shallow candidate and a deeper candidate therefore use different anchor assignments.

For a batch $\mathcal{B}$ and a padding mask $m_x(p)$, the hidden-state loss is

$$\mathcal{L}_{\text{hid}}(\theta) = \frac{1}{|\mathcal{M}|} \sum_{(s,a) \in \mathcal{M}} \mathbb{E}_{x \in \mathcal{B}} \left[\frac{\sum_p m_x(p) \left\| h_{s,p}^{(\mathcal{S})}(x; \theta) - h_{a,p}^{(\mathcal{T})}(x) \right\|_2^2}{d \sum_p m_x(p)} \right], \quad (4.36)$$

where $\mathcal{M} = \{(s, \pi(s)) : s \in \mathcal{S}(\hat{c})\}$. A useful interpretation of Equation 4.36 is that each active student layer is asked to reproduce the token representations of its assigned teacher depth. The padding mask removes padded tokens from the comparison, and the division by $d \sum_p m_x(p)$ makes the squared error comparable across sequence lengths.

Hidden-state matching gives each active layer a target representation after the layer has already combined its attention and feed-forward updates. This is useful, although it does not directly constrain how the surviving attention heads choose token interactions inside the layer. The recovery stage therefore adds pre-softmax attention supervision. This term compares the token-pair scores produced by surviving heads before those scores are converted into probabilities. Let $A_{a,r}^{(\mathcal{T})}(x)$ denote the pre-softmax attention score matrix of teacher anchor layer a and head r , before the row softmax is applied. Let $A_{s,r}^{(\mathcal{S})}(x; \theta)$ be the corresponding student score for a surviving head. With $\mathcal{H}_s$ denoting the active heads in student layer s , the attention-score loss is

$$\mathcal{L}_{\text{attn}}(\theta) = \frac{1}{|\mathcal{M}|} \sum_{(s,a) \in \mathcal{M}} \mathbb{E}_{x \in \mathcal{B}} \left[\frac{1}{|\mathcal{H}_s|} \sum_{r \in \mathcal{H}_s} \frac{\left\| P_x \left(A_{s,r}^{(\mathcal{S})}(x; \theta) - A_{a,\rho(r)}^{(\mathcal{T})}(x) \right) P_x \right\|_F^2}{\left(\sum_p m_x(p) \right)^2} \right], \quad (4.37)$$

where P_x masks padding positions and ρ maps each surviving student head to its teacher head index. A useful interpretation of Equation 4.37 is that it compares the student and teacher token-pair score tables before softmax. This is important because the softmax compresses score differences into a probability simplex. When one token already dominates a row, very different pre-softmax scores can produce similar post-softmax probabilities. Matching the scores therefore preserves more information about the attention decision than matching the probabilities alone. The denominator normalises by the number of valid token pairs, so longer sequences do not dominate the loss only because they contain more entries.

The representation-recovery objective is

$$\mathcal{L}_{\text{repr}}(\theta; t) = \lambda_h(t) \mathcal{L}_{\text{hid}}(\theta) + \lambda_a(t) \mathcal{L}_{\text{attn}}(\theta). \quad (4.38)$$

The hidden term is the primary recovery signal. The attention term is assigned a smaller weight because it constrains the mechanism that produces the layer update. The weights may be fixed or scheduled during the stage. The classifier is frozen, so validation checkpoints measure whether the recovered body supports the existing prediction head.

For checkpoint selection, the recovery process also maintains an exponential moving average of the student parameters:

$$\bar{\theta}_t = \mu \bar{\theta}_{t-1} + (1 - \mu) \theta_t. \quad (4.39)$$

Parameter averaging reduces short-term optimisation noise and often gives a more stable validation estimate than the instantaneous weights (**polyak1992acceleration**). The checkpoint selected at the end of this stage is therefore chosen from the raw or averaged parameter copy according to development accuracy.

4.6.2 Prediction-Level Distillation

After representation recovery, the masked student has been trained toward the teacher’s internal trajectory. The classifier is then unfrozen and the objective shifts to prediction behaviour. This stage uses two signals: the ground-truth task labels and the teacher’s pre-softmax logits. The labels anchor the model to the dataset, whereas the teacher logits provide soft class relations that are absent from one-hot labels.

Let $z^{(\mathcal{T})}(x)$ and $z^{(\mathcal{S})}(x; \theta)$ be the teacher and student logits. With label smoothing parameter ϵ , the smoothed target distribution for a class label y is

$$q_\epsilon(c \mid y) = (1 - \epsilon)\mathbf{1}[c = y] + \frac{\epsilon}{C}.$$

The task loss is

$$\mathcal{L}_{\text{ce}}(\theta) = -\mathbb{E}_{(x,y) \in \mathcal{D}_{\text{train}}} \sum_{c=1}^C q_\epsilon(c \mid y) \log \text{softmax}(z^{(\mathcal{S})}(x; \theta))_c. \quad (4.40)$$

The predictive distillation term uses temperature-scaled teacher logits:

$$\mathcal{L}_{\text{kd}}(\theta) = T^2 \mathbb{E}_{x \in \mathcal{D}_{\text{train}}} \text{KL} \left(\text{softmax} \left(\frac{z^{(\mathcal{T})}(x)}{T} \right) \parallel \text{softmax} \left(\frac{z^{(\mathcal{S})}(x; \theta)}{T} \right) \right). \quad (4.41)$$

The temperature exposes non-argmax class information, and the factor T^2 keeps the gradient scale comparable as T changes (**hinton2015distilling**). When confidence weighting is used, examples with lower teacher confidence receive a smaller KD weight.

The prediction-recovery objective is a dynamically weighted multi-objective loss:

$$\mathcal{L}_{\text{pred}}(\theta; t) = \alpha(t)\mathcal{L}_{\text{ce}}(\theta) + (1 - \alpha(t))\mathcal{L}_{\text{kd}}(\theta), \quad (4.42)$$

where $\alpha(t)$ increases slowly over the stage. Early optimisation assigns more weight to KD because the teacher distribution provides smoother gradients while the classifier adapts to the recovered representations. Later optimisation assigns more weight to CE because the final model is evaluated against task labels rather than teacher predictions.

The optimisation flow is therefore

$$\begin{aligned} \text{fixed mask} &\rightarrow \text{representation recovery} \\ &\rightarrow \text{classifier unfreeze} \\ &\rightarrow \text{prediction recovery} \\ &\rightarrow \text{best raw or EMA checkpoint.} \end{aligned} \quad (4.43)$$

The architecture remains fixed across the entire flow. Recovery trains the already masked architecture; it does not activate or deactivate heads or neurons. The recovered accuracy is therefore determined by the searched mask and the staged distillation procedure.

4.7 Technical Architecture of the Solution

The proposed solution follows a modular technical architecture: load a pretrained Transformer, prepare the task data, define the baseline fine-tuning setup, search for a compressed architecture, recover the selected candidate when required, and evaluate the final model. The implementation pattern is shared across compression families; only the candidate representation and budget measure change.

The search operates on the active compression family: structural masks for encoder pruning, kept or skipped sub-blocks for layer dropping, bit-width levels for quantization, and rank levels for low-rank methods. After selection, optional recovery fine-tuning adapts the chosen candidate, then evaluation saves the final checkpoints, metrics, logs, and figures. Figure 4.11 summarizes this end-to-end architecture.

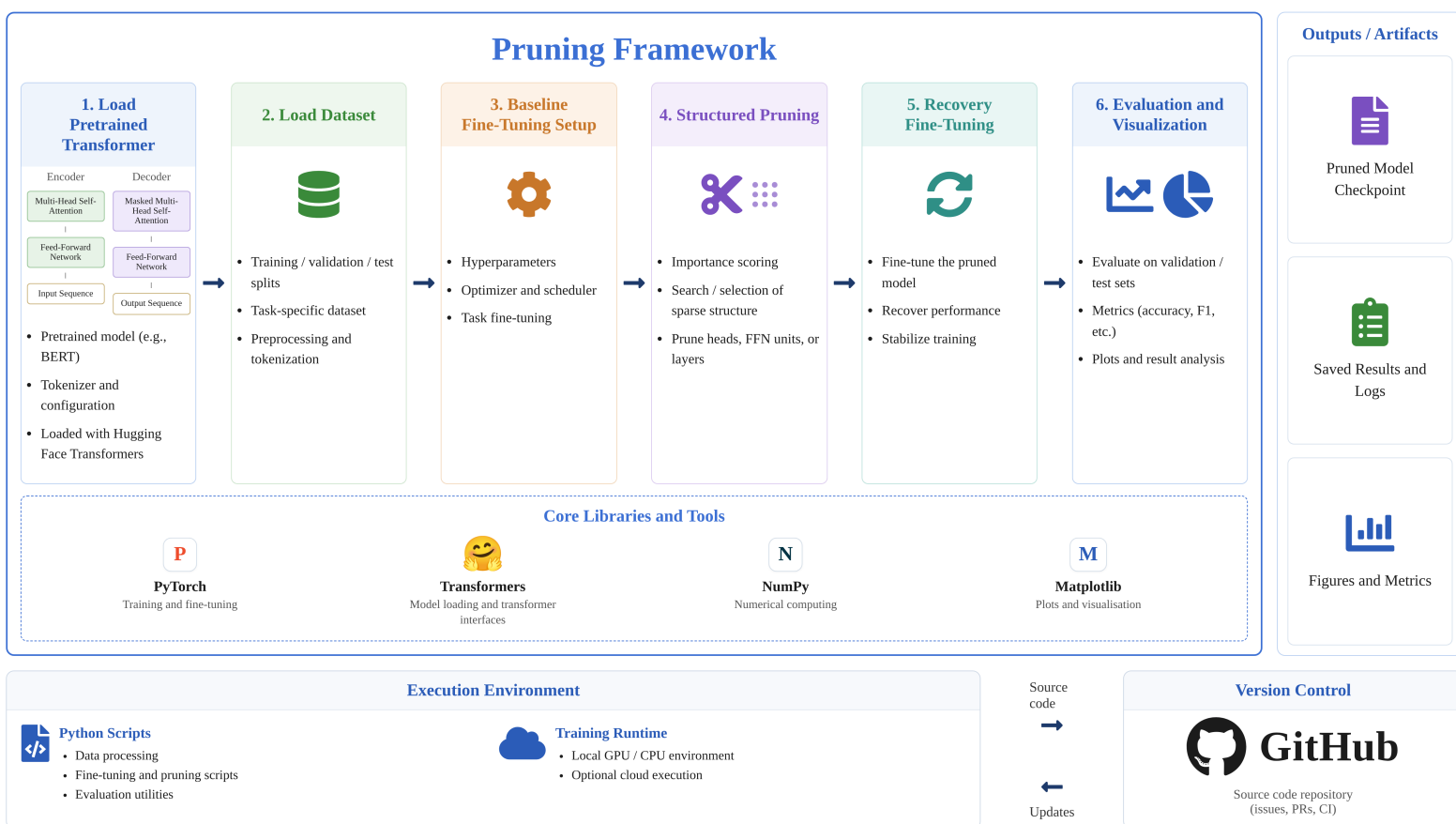


Figure 4.11: Technical pipeline of the implemented compression framework. A pretrained Transformer and task dataset are loaded first, a baseline setup fixes the training configuration, the search stage selects a compressed architecture under the active budget constraint, recovery fine-tuning adapts the selected candidate when needed, and the final stage evaluates the model and saves checkpoints, metrics, logs, and figures.

4.8 Technologies Used

This section presents the main technologies used during the design, implementation, experimentation, and management of the proposed solution. These tools provide the required environment for deep learning development, numerical computation, visualization, experiment execution, version control, and project organization.

4.8.1 Python

Python is an open-source, high-level programming language widely used in software development, scientific computing, data analysis, and artificial intelligence. Its ecosystem provides libraries for model implementation, numerical processing, data handling, and experiment automation. The pruning pipeline, training scripts, evaluation scripts, search procedures, and experiment utilities were implemented in Python `pythonAbout`; `pythonDocs`.



Figure 4.12: Python logo.

4.8.2 Transformers

Transformers is an open-source library developed by Hugging Face. It provides a unified interface for loading pretrained Transformer models, tokenizers, configuration files, and task-specific model heads `wolf2020transformers`; `huggingfaceTransformersDocs`.

The implementation uses this library to load the pretrained encoder model and access its internal components. These components include encoder layers, attention modules, feed-forward submodules, hidden states, attention outputs, configuration parameters, and classification heads. Structured pruning requires access to these components because the method removes complete computational units from the model.

Pruning an attention head requires identifying the corresponding projection dimensions inside the attention module. Pruning feed-forward neurons requires selecting intermediate dimensions inside the two-layer feed-forward network. The Transformers library provides a standardized model structure that supports these operations without requiring a complete reimplementation of the architecture.



Figure 4.13: Hugging Face Transformers logo.

4.8.3 PyTorch

PyTorch is an open-source deep learning framework that provides tensor computation, automatic differentiation, and GPU acceleration. It is commonly used for building, training, and evaluating neural networks. The implementation relies on PyTorch to execute forward passes, compute losses, perform backpropagation, apply masks, update model weights, and run recovery fine-tuning [pytorchDocs](#); [pytorchProject](#).

PyTorch is also used to represent pruning masks as tensors. An attention-head mask can be represented as a binary tensor of shape 12×12 for a model with twelve layers and twelve heads per layer. A feed-forward mask can be represented as a binary tensor of shape 12×3072 for twelve layers and 3072 intermediate neurons per layer.



Figure 4.14: PyTorch logo.

4.8.4 NumPy

NumPy is a fundamental Python library for numerical and scientific computing. It provides efficient multidimensional arrays, mathematical operations, random number generation, and linear algebra utilities. NumPy is used for candidate representation, statistical calculations, random sampling, sorting operations, and search-related utilities [numpyOfficial](#); [numpyPyPI](#).

Candidate architectures can be stored as arrays indicating the number of active heads and neurons per layer. These arrays are then used to calculate sparsity, compare candidates, and enforce computational constraints.



Figure 4.15: NumPy logo.

4.8.5 Matplotlib

Matplotlib is a Python library used to create static, animated, and interactive visualizations. It is used to present experimental results, search behavior, pruning statistics, training curves, and comparisons between candidate architectures [matplotlibDocs](#); [matplotlibOfficial](#).

Typical visualizations include validation accuracy after recovery, active heads per layer, pre-served feed-forward neurons per layer, and the relationship between computational cost and task performance.



Figure 4.16: Matplotlib logo.

4.8.6 Google Drive

Google Drive is a cloud storage service that allows files to be uploaded, stored, shared, and accessed from different devices. It is used to organize project files, store experimental outputs, keep backup copies of important resources, and transfer files between working environments [googleDriveOfficial](#).



Figure 4.17: Google Drive logo.

4.8.7 Git

Git is a free and open-source distributed version control system designed to track changes in source code and support collaborative software development. It is used to track modifications to the implementation, preserve experiment scripts, compare previous versions, and maintain a structured development history [gitOfficial](#); [gitBook](#).



Figure 4.18: Git logo.

4.8.8 GitHub

GitHub is a cloud-based platform built around Git. It allows developers to store, manage, share, and collaborate on code through repositories. It is used to host the project repository, synchronize code between machines, and maintain a remote copy of the implementation `githubDocs`; `githubOfficial`.



Figure 4.19: GitHub logo.

4.9 Datasets Used

The experimental evaluation uses supervised natural language understanding tasks. These tasks evaluate sentence classification and sentence-pair reasoning after structured pruning.

The selected benchmarks are SST-2, QNLI, and MNLI from the GLUE benchmark (`wang2018glue`). The dataset discussion is limited to the required information because the main focus of the project is structured model pruning.

4.9.1 Pre-Training Corpora

The pretrained encoder model was originally trained on BooksCorpus and English Wikipedia (`devlin2019bert`; `zhu2015aligning`). BooksCorpus provides long narrative text from unpublished books. English Wikipedia provides factual and encyclopaedic text. These corpora provide broad linguistic representations before supervised task adaptation.

The model uses WordPiece tokenisation (`wu2016googles`). Frequent words are represented as complete tokens, while rare words are decomposed into subword units. For example, a rare morphological form can be split into smaller units instead of being mapped to an unknown token. The same tokenizer is preserved during fine-tuning, pruning, and evaluation.

4.9.2 Fine-Tuning and Evaluation Benchmarks

Table 4.4 presents the selected evaluation tasks, including their input format, label space, and representative labelled examples.

Task	Example Input	Possible Labels	Correct Label
SST-2	Sentence: <i>“The movie is beautifully acted and emotionally powerful.”</i>	positive, negative	positive
SST-2	Sentence: <i>“The story is dull, slow, and completely forgettable.”</i>	positive, negative	negative
QNLI	Question: <i>“Where is the Eiffel Tower located?”</i> Sentence: <i>“The Eiffel Tower is located in Paris, France.”</i>	entailment, not_entailment	entailment
QNLI	Question: <i>“Where is the Eiffel Tower located?”</i> Sentence: <i>“The Eiffel Tower was completed in 1889.”</i>	entailment, not_entailment	not_entailment
MNLI	Premise: <i>“A man is playing a guitar on stage.”</i> Hypothesis: <i>“A person is performing music.”</i>	entailment, contradiction, neutral	entailment
MNLI	Premise: <i>“A man is playing a guitar on stage.”</i> Hypothesis: <i>“No one is playing an instrument.”</i>	entailment, contradiction, neutral	contradiction
MNLI	Premise: <i>“A man is playing a guitar on stage.”</i> Hypothesis: <i>“The man is performing in a crowded theatre.”</i>	entailment, contradiction, neutral	neutral

Table 4.4: Input formats and label spaces for the selected evaluation tasks.

SST-2

SST-2 is a binary sentiment classification task derived from the Stanford Sentiment Treebank (**socher2013recursive**). Each input is a single sentence, and the model predicts whether its sentiment is positive or negative.

This task provides a sentence-level evaluation of the compressed model. A model must distinguish positive expressions such as *“excellent and moving”* from negative expressions such as *“boring and weak”*. Accuracy is used as the evaluation metric.

QNLI

QNLI is derived from the Stanford Question Answering Dataset (**rajpurkar2016squad**). It is formulated as a sentence-pair classification task. The input contains a question and a candidate sentence. The model predicts one of two labels: entailment or not_entailment.

A label of entailment indicates that the sentence contains the answer to the question. A label of not_entailment indicates that the sentence does not provide the requested information. For example, if the question asks where the Eiffel Tower is located, the sentence *“The Eiffel Tower is located in Paris, France”* receives the label entailment. The sentence *“The Eiffel Tower was completed in 1889”* receives the label not_entailment.

This task evaluates the ability of the compressed model to compare two text segments. Accuracy is used as the evaluation metric.

MNLI

MNLI is a natural language inference task (**williams2018broad**). Each input contains a premise and a hypothesis. The model predicts one of three labels: entailment, contradiction, or

neutral.

A label of entailment indicates that the hypothesis follows from the premise. A label of contradiction indicates that the hypothesis is incompatible with the premise. A label of neutral indicates that the hypothesis may be possible without being guaranteed by the premise.

For example, given the premise “*A man is playing a guitar on stage*”, the hypothesis “*A person is performing music*” receives the label entailment. The hypothesis “*No one is playing an instrument*” receives the label contradiction. The hypothesis “*The man is performing in a crowded theatre*” receives the label neutral, since the premise does not specify the audience or the exact venue.

MNLI provides a stronger sentence-pair reasoning evaluation than single-sentence sentiment classification. Accuracy is used as the reported metric.

4.9.3 Language Model Evaluation Datasets

The language model pruning experiments use three standard perplexity benchmarks to evaluate the compressed models.

WikiText-2

WikiText-2 is a word-level language modelling benchmark derived from verified Good and Featured Wikipedia articles ([merity2017pointer](#)). It provides a clean, coherent corpus of encyclopaedic text. Perplexity on the test split is the standard evaluation metric; lower values indicate better language modelling quality.

C4

C4 (Colossal Clean Crawled Corpus) is a large-scale web text corpus produced by applying aggressive filtering and deduplication to Common Crawl snapshots ([raffel2020exploring](#)). It covers a broad range of domains and writing styles and is commonly used both as a pretraining source and as an out-of-domain perplexity benchmark for evaluating generalisation after compression.

FineWeb-Edu

FineWeb-Edu is a high-quality subset of the FineWeb web corpus filtered for educational content ([lozhkov2024fineweb_edu](#)). The filtering pipeline uses a classifier trained to identify instructional and informative text. It provides a more domain-specific evaluation signal than general web corpora and has been adopted as a benchmark for assessing language model quality on structured, factual text.

4.10 Model

4.10.1 BERT

BERT is an encoder-only Transformer architecture trained with masked language modelling and next-sentence prediction ([devlin2019bert](#); [vaswani2017attention](#)). Its bidirectional self-attention allows each token representation to incorporate information from both left and right context.

BERT_{BASE} contains twelve Transformer encoder layers. Each layer has twelve attention heads and a feed-forward block with intermediate width 3072. The hidden size is 768, and the full model contains approximately 110 million parameters. For classification tasks, the representation of the [CLS] token is passed to a task-specific classifier.

The pruning procedure targets structured units inside the encoder blocks. Attention heads and feed-forward neurons are removed as complete units. This pruning form supports practical acceleration because it can reduce matrix dimensions and remove complete computational paths.

For example, if layer 3 preserves only 2 attention heads out of 12, then the attention computation in that layer is reduced. If the same layer preserves only 192 feed-forward neurons out of 3072, then the intermediate feed-forward projection is also reduced. The final compressed architecture is therefore described by its total sparsity and by the distribution of preserved units across layers.

4.10.2 EleutherAI Pythia-1.4B

Pythia is a suite of autoregressive decoder-only language models released by EleutherAI and trained on The Pile, a diverse 825 GB English text corpus ([biderman2023pythia](#)). The suite spans eight model sizes from 70M to 12B parameters and was designed to support reproducible research on scaling laws, training dynamics, and model analysis.

Pythia-1.4B contains 24 Transformer decoder layers with a hidden dimension of 2048, 16 attention heads per layer, and a feed-forward intermediate width of 8192. The model has approximately 1.4 billion parameters. It uses rotary position embeddings and parallel attention and feed-forward computation within each layer.

This model is selected for the language model pruning experiments because its size is tractable for iterative search-based compression on a single GPU, while being large enough to exhibit realistic structured redundancy across layers. Perplexity on WikiText-2, C4, and FineWeb-Edu is used as the evaluation metric.

4.11 Experimental Environment

All experiments are conducted on a laptop equipped with an NVIDIA RTX 4090 Laptop GPU with 16 GB of VRAM. Model loading, tokenization, fine-tuning, pruning, checkpoint handling, and evaluation are implemented using the Hugging Face Transformers library

(wolf2020transformers).

Mixed precision training is used when numerically stable (micikevicius2018mixed). It reduces memory consumption and can improve throughput on compatible hardware. Numerical stability is monitored during recovery fine-tuning, especially for highly compressed architectures.

The hardware environment imposes practical constraints on the experimental protocol. Laptop GPUs are affected by thermal limits, power configuration, driver behavior, and cooling conditions. Therefore, wall-clock measurements are hardware-dependent. Small accuracy differences are interpreted cautiously because repeated runs may vary due to random initialization, data ordering, dropout, and GPU-level nondeterminism.

4.11.1 Baselines and Reporting

Included in the comparison is our search approach and random, beam search and structured pruning comparisons, drawing structured pruning refs from CoFi (xia2022cofi) and ZipLM (kurtic2023ziplm). Comparisons are made at the same constraints where possible, particularly the target computation budget, recovery process and measure.

For the decoder-model searches on Pythia-1.4B (layer drop, quant and low-rank factorisation), we compare against EvoPress (evopress2024), a search-based approach whose evolutionary algorithm and KL-divergence fitness are most similar to our approach. Both methods are run at the same sparsity or bit-width target and compared over the same perplexity datasets.

4.11.2 Level Database and Token Budgets

Both the quantization and the low-rank search are performed on a level database that has been generated beforehand. For each weight matrix, weights are pre-compressed to every possible level and persisted to disk: one GPTQ-quantized copy at each bit-width, and a truncated-SVD factor pair at each rank (plus the residue singular triplets required to recover weights). During search, each candidate is assembled by loading the required weights whose names are specified by its per-layer level vector. No decompression or on-the-fly compression happens and each query takes one forward pass.

Each of the searches tunes to approximately 250,000 held-out tokens. This is well below the budgets spent by the bigger baseline models, and necessary given the 16 GB of VRAM on the GPU. Candidates are scored using the multi-stage process described in Section 4.4 above: the early stages screen the population efficiently by spending a limited number of tokens per candidate, and the remaining candidates are then scored on more tokens to make the selection. Final perplexity is reported over the entire test sets.

4.12 Conclusion

In this chapter we have defined the entire implementation pipeline that is utilized for this work. Initially, we parameterize the compressed architecture in binary structural parameters, after which we constrain the candidate space using a cost budget based model for both the encoder and the decoder. We then compare candidate masks prior to recovery, using a calibration based surrogate metric; we finally find the mask using a genetic search, with efforts to keep variety within the different architectural regions of the candidate space, and recover the final architecture using cascaded distillation, using the already found mask.

The primary design decision here is the distinction between architecture selection and parameter recovery. The search stage determines where the rest of the computation is assigned; and the recovery stage would fine tune only the selected architectures' surviving parameters.

The next chapter reports the observed performance of the selected models and compares the proposed search procedure with other search baselines and well know pruning methods from the literature.

Chapter 5

Results

This Chapter reports two groups of results under the experimental setting of Section 4.11. The first group evaluates encoder-side structured pruning on GLUE classification tasks, where BERT architectures are selected by search and then recovered with the distillation procedure of Section 4.6. The second group evaluates decoder-side compression on Pythia-1.4B, where perplexity replaces accuracy and the same search machinery is applied to layer sparsity, low-rank factorization, and quantization.

5.1 Result Tables and Discussion

Table 5.1 summarizes the main empirical takeaways before the detailed comparisons. The table is intentionally selective: it highlights the strongest size-matched or near-size results without replacing the full task tables.

Table 5.1: Selected headline results. Accuracy is higher-is-better for GLUE; perplexity is lower-is-better for Pythia-1.4B.

Setting	Result
SST-2, 7.0M	Ours 93.35 vs ZipLM 93.0 at 5.7M and CoFi 91.51
SST-2, 4.0M	Ours 92.43 vs ZipLM 91.7 at 4.2M
QNLI, 3.0M	Ours 88.2 vs ZipLM 87.6 at 3.2M
Low-rank FT, WikiText-2	Ours 24.91 vs EvoPress 28.656
Quantization, 2.5 bits	EvoPress 33.06 vs ours 33.19 on WikiText-2

In addition to accuracy, the procedure has a measurable runtime cost. Under the hardware setting of Section 4.11, end-to-end runs on QNLI and SST-2 required nearly six hours on average, whereas MNLI required about thirty hours on average. This difference is consistent with the larger training split and heavier recovery cost of MNLI.

5.1.1 Encoder Classification Results

Search Baselines

The first question is whether the evolutionary search improves on simpler budget-constrained selection rules. Table 5.2 compares the proposed method against random search and beam search under matched recovery and matched budget targets. The comparison is organised by the final recovered model size, since the search problem changes materially between the tiny and moderate regimes.

Table 5.2: Development accuracy under matched recovery for three search strategies. Random search and beam search use the same teacher, budget band, and recovery procedure as the proposed method.

Task	Final model	Params	Random	Beam	Ours
QNLI	tiny	3.0M	84.72	85.36	88.20
QNLI	moderate	34.1M	90.74	91.10	91.90
MNLI	tiny	3.1M	80.18	80.76	81.34
MNLI	moderate	34.0M	84.92	85.19	85.60
SST-2	tiny	3.3M	91.21	91.76	92.43
SST-2	moderate	39.0M	93.08	93.31	93.70

The margin is maximal at the very small end. At 3.0 M on QNLI, our search returns 88.2 while beam search gets 85.36 and random search is lower still. This is repeated on MNLI and SST-2; in the very small regime where budget constrains the search to that architecture size, recoverability is directly correlated with the quality of search. The range decreases as we move into the moderate regime where our search still remains ahead. This is in line with our design choice; in the cases where many viable candidates share equal costs, random search and beam search still find viable masks, but do not maintain the structural diversity that island-based evolution preserves.

Comparison to Structured Pruning Baselines

The second question is whether the recovered models remain competitive against established structured pruning methods at comparable sizes. Table 5.3 compares the recovered models against representative CoFi and ZipLM results, together with the task-fine-tuned uncompressed baseline used in this study.

Table 5.3: Comparison against structured pruning baselines and the uncompressed task baseline. Scores are development accuracy.

Task	Model	Params	Score	Note
QNLI	CoFi, 95%	5.0M	86.1	high compression
	ZipLM, near-size	3.2M	87.6	13× speedup
	Ours	3.0M	88.2	best tiny model
	CoFi, 60%	34.0M	91.8	moderate compression
	ZipLM, 2×	38.0M	91.4	near-size reference
	Ours	34.1M	91.9	above baseline
	task baseline	full	91.4	uncompressed run
MNLI	CoFi, 95%	5.0M	80.6	high compression
	ZipLM, near-size	3.5M	81.3	13× speedup
	Ours	3.1M	81.34	above near-size baseline
	CoFi, 60%	34.0M	85.3	moderate compression
	ZipLM, 2×	38.5M	84.8	close size
	Ours	34.0M	85.6	above both
	task baseline	full	84.7	uncompressed run
SST-2	CoFi, compact	4.4M	91.1	near tiny-size comparison
	ZipLM, compact	4.2M	91.7	near-size comparison
	Ours	4.0M	92.43	above near-size ZipLM
	ZipLM, compact	3.6M	91.7	same size as ours
	Ours	3.6M	91.6	above CoFi, near ZipLM
	CoFi, 90% sparse	≈8.5M	91.51	0.1 × 85M encoder
	ZipLM, compact	5.7M	93.0	compact reference
	Ours	7.0M	93.35	above CoFi and compact ZipLM
	CoFi, 95%	5.0M	90.4	high compression
	ZipLM, near-size	3.6M	91.7	14× speedup
	Ours	3.3M	92.43	strongest earlier tiny run
	CoFi, 60%	34.0M	93.0	moderate compression
	ZipLM, 2×	38.7M	93.4	close size
	Ours	39.0M	93.7	best moderate model
	task baseline	full	93.22	uncompressed run

We make three general observations from these results. First, our technique is most promising in the small-model regime. The gaps with CoFi and ZipLM are greatest between 3M and 7M parameters; at this point, the other part of the architecture has little room for capacity. Specifically, our 7.0M model for SST-2 performs at 93.35 while the best compact result for CoFi is 91.51 and ZipLM performs at 93.0 at 5.7M parameters. Our 4.0M model is at 92.43 compared to 91.7 for ZipLM at 4.2M, and our 3.6M model is at 91.6, greater than the 4.4M CoFi model and similar to the 3.6M ZipLM model. For the 90% sparse CoFi result, the estimated size of the encoder remaining is approximately 8.5M parameters, because the encoder that was reported had approximately 85M parameters. Second, the moderate-compression results also perform competitively. The QNLI model of 34.1M parameters achieves 91.9, our MNLI model of 34M parameters achieves 85.6, and our SST-2 model of 39M parameters achieves 93.7. Third, the recovered models do approach the baseline performance at the corresponding task. The differences are minimal at QNLI and SST-2 and compare favorably to the stated results at similar sized networks. This means our search has value beyond finding the desired compression ratio, in that we can search for compressible architecture that may perform well even after pruning.

5.1.2 Decoder Compression Results (Pythia-1.4B)

Layer-Sparsity Search

The decoder experiments probe if the same genetic search machinery generalize to things outside of encoder pruning so it transcends into a generic framework. In these the candidate is a decoder compression scheme applied to Pythia-1.4B, and the candidates are evaluated by their perplexity on WikiText-2, C4, and FineWeb-Edu. EvoPress [evopress2024] is the closest baseline because it searches with evolutionary algorithms and it scores candidates using KL divergence similar to ours. The baselines are tested with identical sparsity values and identical datasets to the ones in this paper.

Table 5.4 reports results at 25% sparsity. The proposed method reaches lower perplexity than EvoPress on all three datasets after only 11 generations, whereas EvoPress requires 72 generations to reach its reported values. This earlier convergence reflects the efficiency of the search procedure in identifying well-structured sparse configurations under tight generation budgets.

Table 5.4: Perplexity ($\downarrow$) on Pythia-1.4B at 25% sparsity. EvoPress results are reported at generation 72; ours at generation 11.

Dataset	EvoPress (gen 72)	Ours (gen 11)
FineWeb-Edu	25.69	23.58
WikiText-2	28.38	25.00
C4	31.36	28.66

Table 5.5 reports results at 50% sparsity, a significantly more aggressive compression target, under the larger 2M-token calibration setting for both methods. At this regime, EvoPress reaches perplexity values of 152.50, 208.50, and 184.00 on FineWeb-Edu, WikiText-2, and C4 respectively after 90 generations. The proposed method achieves lower perplexity on all three datasets after only 30 generations, demonstrating that the search is effective even under severe compression where maintaining language model coherence is substantially harder.

Table 5.5: Perplexity ($\downarrow$) on Pythia-1.4B at 50% sparsity. EvoPress results are reported at generation 90; ours at generation 30.

Dataset	EvoPress (gen 90)	Ours (gen 30)
FineWeb-Edu	152.50	131.00
WikiText-2	208.50	155.50
C4	184.00	159.88

We also evaluated a separate 50% layer-sparsity setting using a smaller calibration budget of approximately 131k tokens for both methods. This lower-calibration run is therefore kept separate from the headline 2M-token comparison, but it is still informative: under the smaller calibration budget, the proposed configuration remains below EvoPress on all three evaluation datasets.

Table 5.6: Additional 50% layer-sparsity result under a lower calibration budget for both methods. Perplexity is lower-is-better.

Dataset	EvoPress (131k tokens)	Ours (131k tokens)
FineWeb-Edu	154.88	132.500
WikiText-2	202.12	195.875
C4	189.88	168.875

Proposed method achieves better perplexity compared to EvoPress with less generation counts in both sparsity levels. At 50% sparsity, the perplexity difference is above 50 for WikiText-2, and over 20 for FineWeb-Edu and C4. We can conclude that evolutionary search also transfer well to generative language models as measured by perplexity and it is not just the performance gained for encoder-based classification models.

Low-Rank Factorization

Table 5.7 reports perplexity results when the search is applied to low-rank factorization on Pythia-1.4B. The baseline is EvoPress. The proposed method reaches lower perplexity on all three evaluation datasets.

Table 5.7: Perplexity ($\downarrow$) on Pythia-1.4B under low-rank factorization. EvoPress is the baseline.

Dataset	EvoPress	Ours
FineWeb-Edu	63.34	59.97
WikiText-2	33.50	29.65
C4	83.88	80.94

The second low-rank table evaluates the recovery fine-tuning stage applied after search. This stage is part of the recovery pipeline rather than a new search objective: the architecture is fixed, and the remaining step adapts the selected low-rank model. It improves both methods, but the proposed search still retains a clear advantage after fine-tuning.

Table 5.8: Perplexity ($\downarrow$) on Pythia-1.4B under low-rank factorization, after recovery fine-tuning.

Dataset	EvoPress	Ours
FineWeb-Edu	53.219	46.25
WikiText-2	28.656	24.91
C4	68.500	64.06

The fine-tuned low-rank results show the effect of the selected rank allocation after recovery. EvoPress uses the same recovery stage, so both methods benefit from fine-tuning. After the stage, the proposed model remains lower by 6.969 perplexity on FineWeb-Edu, 3.746 on WikiText-2, and 4.440 on C4.

Quantization

Tables 5.9 and 5.10 report perplexity results under post-training quantization at two average bit-widths. The baseline is EvoPress.

Table 5.9: Perplexity ($\downarrow$) on Pythia-1.4B at 2.25 bits average quantization.

Dataset	EvoPress	Ours
FineWeb-Edu	45.97	45.63
WikiText-2	81.00	76.69
C4	65.06	63.84

At 2.25 bits, the proposed method reaches lower perplexity than EvoPress on all three datasets, with the most visible gap on WikiText-2.

Table 5.10: Perplexity ($\downarrow$) on Pythia-1.4B at 2.5 bits average quantization.

Dataset	EvoPress	Ours
FineWeb-Edu	25.14	25.80
WikiText-2	33.06	33.19
C4	32.84	33.69

At 2.5 bits, EvoPress holds a small edge on all three datasets, with gaps below one perplexity point. Results at this bit-width are competitive across both methods. This is a useful boundary case: the search is not presented as uniformly dominating EvoPress, but as competitive when the compression level changes.

5.2 Conclusion

This chapter tested the proposed search framework on encoder-side structured pruning and decoder-side compression. As for the encoder experiment, we can find that the search is most meaningful when the reduced architecture is extremely small, where the assignment of heads and feed-forward capacity is critical for recovery. In the decoder experiments, the same search principle holds outside of pruning. We demonstrate notable improvement for layer sparsity and low-rank factorization, and competitive performance in the quantization scenario.

From this follows our findings provide empirical support for the core hypothesis of the paper: rather than individual local decisions, it is better to view compressed Transformers as holistic architectures optimized within a given budget. The primary challenge remains the overhead from search and recovery; however, as illustrated, this cost can often be justified by the search for aggressively compressed but recoverable architectures.

Conclusion and Perspectives

General Conclusion

This thesis examined the problem of making Transformer models more practical to deploy under limited computational resources. It first presented the foundations of neural networks, Transformers, optimization, and evolutionary search, then reviewed the main families of model compression and the limitations that appear when compression decisions are made only from local importance signals. From this perspective, the work framed compression as a question of how a fixed budget should be distributed across the architecture so that the final model can still be recovered after compression.

The contribution is organized around three levels. First, the work defines a generic compression-search framework in which complete candidate architectures are optimized under an explicit budget before recovery. Second, it introduces the search machinery needed to keep this optimization diverse and tractable, including structural descriptors, CVT-based islands, and island-level operator adaptation. Third, it instantiates the framework for both encoder and decoder settings: the encoder search uses a task-aware surrogate metric to avoid relying only on saturated KL-divergence signals, while the decoder searches define candidate forms and descriptors for layer sparsity, low-rank factorization, and quantization. The results show that this framework can produce competitive compressed Transformers in both settings. The main limitation remains the cost of search and recovery, which constrains how widely the framework can be explored.

Perspectives

Future work should focus on reducing search cost, replacing MAC-only budgets with device-aware latency and memory objectives, and studying the recovery stage more systematically. A more exploratory direction would be to test joint decoder search over pruning, low-rank factorization, and quantization. This possibility is interesting, although it may be expensive to evaluate because the joint search space would grow quickly.

Appendices

Appendix: The EXP3-IX Algorithm

A conceptual overview of Implicit Exploration for Adversarial Bandits.

A Introduction: The Bandit Problem

Imagine you are in a casino with K different slot machines, which we call **arms**. You will play for a total of T rounds (the **horizon**). At each round t , you must pick exactly one arm. Instead of random payouts, imagine a malicious adversary assigns a hidden loss $y_{t,i} \in [0, 1]$ to each machine i right before you pull. Crucially, **you only observe the loss of the machine you actually played**. You do not get to see what the other machines would have cost you. This missing information setting is known as the *Adversarial Multi-Armed Bandit* problem.

The same view applies beyond casinos: any problem with several possible actions can be written in bandit form when the method chooses one action and receives feedback only for that chosen action. In our case, an island has many mutation operators, but it can choose only one operator at a time.

Our goal is to minimize **regret**: the mathematical difference between the total loss we accumulated and the total loss we would have suffered if we had magically known the single best machine in hindsight [1].

B The Flaw in Standard EXP3

A classical algorithm to solve this is **EXP3**. Because we only see the loss for the pulled arm A_t , EXP3 estimates the hidden losses of all arms using a trick called *importance sampling*. It estimates the cumulative loss $\hat{L}_{t,i}$ as:

$$\hat{L}_{t,i} = \hat{L}_{t-1,i} + \frac{\mathbb{I}(A_t = i) \cdot y_{t,A_t}}{P_{t,i}} \quad (1)$$

Here, $P_{t,i}$ is the probability that the algorithm chose arm i , and $\mathbb{I}$ is an *indicator function* (1 if we pulled arm i , and 0 otherwise).

The problem: If we Look at the denominator We notice that if the probability $P_{t,i}$ of picking a specific arm becomes very small (e.g., 0.0001), the estimated loss shoots to infinity! This creates *massive variance*. While EXP3 guarantees good performance *on average*, in practice, this variance means any single run can perform spectacularly poorly [1].

C The Solution: EXP3-IX

To fix this variance issue, the **EXP3-IX** algorithm introduces a brilliant, tiny modification. It adds a small regularization parameter, $\gamma > 0$, to the denominator of the estimate:

$$\hat{L}_{t,i} = \hat{L}_{t-1,i} + \frac{\mathbb{I}(A_t = i) \cdot y_{t,A_t}}{P_{t,i} + \gamma} \quad (2)$$

By adding γ , the fraction is forever bounded by $1/\gamma$, entirely eliminating the variance explosion, regardless of how small $P_{t,i}$ becomes.

Why “Implicit Exploration”? As we introduced γ into the denominator we get an algorithm that underestimates the actual loss of the unpulled arms. That is, by making the unseen arms look just slightly more “cheap”, the algorithm is implicitly persuaded to explore them. So we can obtain safe exploration without the necessity of having an explicit random exploration phase!

D Algorithm Flow

The architecture of EXP3-IX forms an elegant, self-correcting feedback loop (Figure 1).

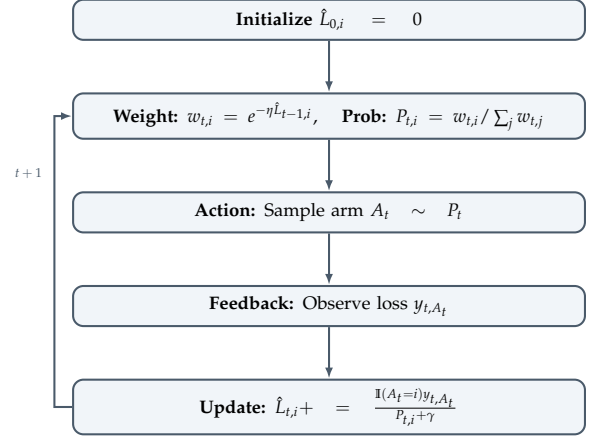


Figure 1: The continuous loop of EXP3-IX.

The step-by-step logic is detailed in Algorithm 1. The variable η (the learning rate) controls how aggressively we punish arms with high observed losses.

Algorithm 1 EXP3-IX

- 1: **Input:** Horizon T , arms K , learning rate $\eta > 0$, implicit exploration factor $\gamma > 0$.
- 2: **Initialize:** $\hat{L}_{0,i} = 0$ for all arms $i \in \{1, \dots, K\}$.
- 3: **for** $t = 1, 2, \dots, T$ **do**
- 4: $P_{t,i} = \frac{\exp(-\eta \hat{L}_{t-1,i})}{\sum_{j=1}^K \exp(-\eta \hat{L}_{t-1,j})}$.
- 5: Sample arm A_t according to P_t and observe actual loss $y_{t,A_t} \in [0, 1]$.
- 6: $\hat{L}_{t,i} = \hat{L}_{t-1,i} + \frac{\mathbb{I}(A_t = i) \cdot y_{t,A_t}}{P_{t,i} + \gamma}$ for all arms i .
- 7: **end for**

E Performance Guarantee

Because the variance is strictly controlled, EXP3-IX provides highly desirable **high-probability bounds** rather than mere expectations.

By setting the learning rate to $\eta = \sqrt{\frac{2 \log(K+1)}{TK}}$ and regularization to $\gamma = \frac{\eta}{2}$, the total regret R_T satisfies the following bound with probability exceeding $1 - \delta$ [1, 2]:

$$R_T \leq \sqrt{8TK \log(K+1)} + \mathcal{O}\left(\log\left(\frac{1}{\delta}\right)\right)$$

In plain terms: this guarantee says that, except with probability δ , the regret remains controlled even in adversarial settings. The era of catastrophic single-run failures is eliminated.

References

- [1] J. Jiao, E. Xia, and E. Liu. *Theory of Multi-Armed Bandits and RL: Lecture 12 - Exp3-Implicit Exploration*. UC Berkeley, 2020.
- [2] T. Lattimore and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.

Appendix B: Centroidal Voronoi Tessellations

A Representative Placement

Imagine placing k hospitals to cover a city. A natural assignment rule sends each neighbourhood to the nearest hospital, while a natural placement rule asks each hospital to lie near the centre of the area it serves. When these two requirements are satisfied simultaneously, the resulting structure is a **Centroidal Voronoi Tessellation (CVT)**.

This concept, formalised by Du, Faber, and Gunzburger [1], appears in image compression, mesh generation, clustering, and optimal quantisation. Here, CVTs divide the compression configuration space into balanced and well-separated search regions, each maintained as an independent population in the genetic search. The construction rests on two notions: Voronoi cells and centroids.

B Voronoi Assignment

Let Ω be a bounded region and let $\{z_i\}_{i=1}^k$ be a set of chosen points inside it. These points are the **generators**. The Voronoi cell V_i associated with generator z_i contains exactly the points of Ω for which z_i is the nearest generator [2]:

$$V_i = \{x \in \Omega \mid \|x - z_i\| < \|x - z_j\|, \forall j \neq i\}.$$

The cells $\{V_i\}$ tile Ω without gaps or overlaps. Their boundaries are perpendicular bisectors between neighbouring generator pairs. In the genetic search, this gives each compression configuration a unique region assignment and avoids ambiguity between populations.

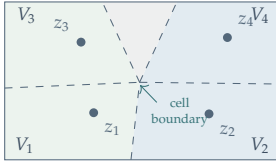


Figure 1: Voronoi diagram with four generators. Each region V_i contains the points closest to its generator.

C Density-Weighted Centroids

The **centroid** of a region is its density-weighted average position. For a region V with density function ρ , it is defined as

$$z^* = \frac{\int_V y \rho(y) dy}{\int_V \rho(y) dy}.$$

For a uniform rectangle, the centroid is the geometric centre. For an irregular region or non-uniform density, it shifts toward the side carrying more mass.

A generator z_i placed arbitrarily in Ω does not generally coincide with the centroid z_i^* of its Voronoi cell. The generator defines the cell, whereas the centroid is computed after the assignment has been made.

D Centroidal Consistency

A **Centroidal Voronoi Tessellation** is a Voronoi diagram in which every generator is also the centroid of its associated cell [1]:

$$z_i = z_i^* \quad \forall i, \quad \text{i.e., generator = centroid of its cell.}$$

This condition couples assignment and placement. Moving a generator changes the Voronoi boundaries, which changes the centroids and may require further generator updates. A CVT is a fixed point of this process. In the genetic search, it gives stabilised regions whose representatives lie at the centres of their assigned territories. Figure 2 illustrates this condition.

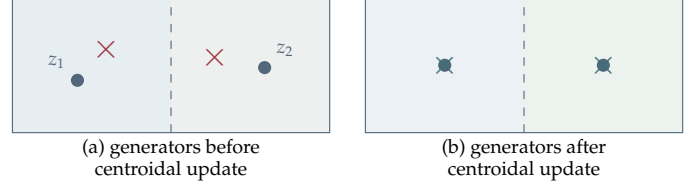


Figure 2: The centroidal condition aligns each generator with the centroid of its assigned Voronoi cell.

E Lloyd Iteration

A standard construction procedure is **Lloyd's algorithm** [1], which alternates between nearest-generator assignment and centroidal relocation:

1. Initialise k generators at random positions in Ω .
2. Build the Voronoi diagram by assigning each point or sample to its nearest generator.
3. Move every generator to the centroid of its current cell.
4. Repeat the assignment and relocation steps until the generators no longer move appreciably.

F Energy Formulation of CVTs

The CVT condition also has an energy formulation. For a partition $\{V_i\}_{i=1}^k$ and generators $\{z_i\}_{i=1}^k$, define the **quantisation error**, also called total inertia, as [1]

$$\mathcal{E} = \sum_{i=1}^k \int_{V_i} \rho(y) \|y - z_i\|^2 dy.$$

This objective measures the density-weighted squared distance from points in Ω to their assigned generators. Minimising $\mathcal{E}$ over the partition gives the Voronoi assignment rule; minimising it over the generator locations places each generator at the centroid of its cell. The stationary structure therefore satisfies the CVT condition.

Energy interpretation. Lloyd's algorithm can be viewed as alternating minimisation of $\mathcal{E}$. With fixed generators, the nearest-generator Voronoi assignment minimises the distance term. With fixed cells, replacing each generator by the corresponding centroid minimises the same objective inside that cell. The resulting compact regions help the genetic search maintain distinct populations over different parts of the compression configuration space.

References

- [1] Q. Du, V. Faber, and M. Gunzburger. *Centroidal Voronoi Tessellations: Applications and Algorithms*. SIAM Review, 41(4):637–676, 1999.
- [2] D. M. Mount. *CMSC 754 Computational Geometry Lecture Notes*. University of Maryland, 2002.

Appendix C: Search Descriptors

A Why a Descriptor Space

The genetic search never partitions candidates in their raw form. A candidate is a long per-layer vector: a bit-width for every weight matrix in the quantization search, a rank level for every matrix in the low-rank search, or a drop flag for every sub-block in the layer-drop search. For a model with dozens of layers this is on the order of a hundred dimensions, and clustering or stratifying directly in that space is hopeless, since covering each axis even once would take an impractical number of candidates and every candidate costs a full forward pass to evaluate.

Each candidate g is therefore mapped to a short, fixed-length **descriptor**: a six-number summary that captures the *style* of the allocation rather than its exact entries,

$$\Phi(g) = (\phi_1(g), \dots, \phi_6(g)) \in [0, 1]^6.$$

Two candidates that spend their budget the same way (early layers, attention-heavy, clustered, and so on) land near each other in Φ -space even if their individual entries differ. Every ϕ_k is normalised to $[0, 1]$ and is read straight from the candidate vector, with no extra forward pass.

Role in the search. The Centroidal Voronoi Tessellation of Appendix B is built in this descriptor space. Clustering there is cheap and meaningful, and it gives each island ownership of a structurally distinct region of allocation styles, so the populations stay diverse instead of collapsing onto one strategy.

The three searches use different descriptors because their candidates mean different things, but all six axes in each case follow the same intent: locate the budget along depth, split it between attention and feed forward, and measure how evenly or unevenly it is spread.

B Layer-Drop Descriptors

A layer-drop candidate marks attention and feed-forward sub-blocks for removal. Its descriptor records where the removals sit and how they are shaped.

- **drop_com**: the average depth of the dropped sub-blocks, scaled so that 0 means all drops are early and 1 means all are late. Writing $\mathcal{P}$ for the set of dropped depths,

$$\phi_{\text{com}} = \frac{1}{|\mathcal{P}| (L-1)} \sum_{p \in \mathcal{P}} p.$$

- **block_drop_share**: fraction of drops that remove a whole layer (both its attention and its feed forward), as opposed to only one side.
- **attn_only_share**: fraction of drops that remove only the attention sub-block.
- **mlp_only_share**: fraction of drops that remove only the feed-forward sub-block.
- **drop_clustering**: how bunched the drops are along depth, computed from the spread of the gaps between consecutive drops (1 = one contiguous run, 0 = evenly scattered).

- **total_drops_norm**: the number of dropped sub-blocks divided by the total available ($2L$ for L layers).

C Quantization Descriptors

A quantization candidate assigns a bit-width to each weight matrix. Its descriptor measures where the bits are concentrated and how precision is shared inside a block.

- **early_quarter_share**: fraction of the total bit budget spent in the first quarter of the layers.
- **late_quarter_share**: fraction spent in the last quarter of the layers.
- **qkv_share**: fraction of the bits spent on the query, key, and value input projections.
- **down_proj_share**: within the feed-forward block, the share of bits given to the down-projection rather than the up-projection.
- **longest_high_bit_run**: the longest run of consecutive blocks whose bit-width is above the candidate's own average, divided by the number of blocks.
- **numel_weighted_avg**: the parameter-weighted mean bit-width, normalised by the largest available level b_{max} ,

$$\phi_{\text{avg}} = \frac{\sum_i b_i |W_i|}{b_{\text{max}} \sum_i |W_i|},$$

a smooth stand-in for overall precision.

D Low-Rank and Recovery Descriptors

A low-rank candidate assigns a rank level to each matrix; the parameter cost of a layer grows with its rank. The descriptor reuses the depth and attention axes and adds two that come for free from the singular value spectrum.

- **early_rank_share**: fraction of the parameter budget placed in the first quarter of the layers.
- **late_rank_share**: fraction placed in the last quarter.
- **attn_rank_share**: fraction placed on the attention matrices rather than the feed-forward ones.
- **energy_mean**: the average fraction of each matrix's spectral energy that the chosen rank retains, read directly from the singular values σ_{ij} ,

$$\phi_{\text{energy}} = \frac{1}{K} \sum_{i=1}^K \frac{\sum_{j=1}^{r_i} \sigma_{ij}^2}{\sum_j \sigma_{ij}^2}.$$

- **energy_spread**: the standard deviation of that retained energy across layers, measuring how uneven the retention is.
- **longest_high_run**: the longest run of consecutive blocks whose rank level is above the candidate's average, divided by the number of blocks.

The rank-based recovery search reuses this same descriptor family, computed on the per-layer residual ranks and on the energy of the residual singular values, since a recovery candidate is itself a rank-allocation vector.

Appendix D: Search Operators

A Shared Design

Every operator is a local move that turns a parent candidate into one offspring. The single rule they all obey is that the offspring must stay within the budget: a move that would break the constraint is either compensated within the same move or repaired afterwards. The operators differ in how large a step they take and in which part of the candidate they touch, and a per-island bandit learns which ones are worth using in each region of the search.

Two scales of move. Small budget-preserving swaps refine a candidate without changing its overall shape, while larger moves rearrange the allocation across depth or across the attention and feed-forward axes. Keeping both lets each island polish a good candidate and also escape a stale one.

B Encoder (Pruning) Operators

The pruning search edits two masks, one over attention heads and one over feed-forward neurons.

Budget-neutral swaps.

- **Head swap:** remove one head from one layer and add one head to another, leaving the head count and the compute unchanged.
- **Neuron swap:** remove one neuron block from one layer and add one block to another, leaving the neuron count and the compute unchanged.
- **Head for neurons:** remove heads from one layer and add the equivalent compute as neuron blocks to another, shifting the split between attention and feed forward.
- **Neurons for head:** remove neuron blocks from one layer and add the equivalent compute as heads to another, the reverse trade.

Growth and shrinkage.

- **Grow heads:** add a head to a chosen layer and remove neuron blocks from the least important layers until the budget is restored.
- **Grow neurons:** add a neuron block to a chosen layer and remove heads from the least important layers until the budget is restored.
- **Shrink heads:** remove one head from a layer, leaving the candidate under budget so a later move can place the freed compute.
- **Shrink neurons:** remove one neuron block from a layer, leaving the candidate under budget so a later move can place the freed compute.

Layer-level.

- **Drop layer:** remove every surviving head and neuron in a layer, turning it into a skipped layer.
- **Activate layer:** re-activate a dropped layer by placing a minimum block of heads or neurons, paid for from the lowest-value structures elsewhere.
- **Activate attention only:** re-activate a dropped layer with attention heads only and no neurons.

- **Activate feed-forward only:** re-activate a dropped layer with feed-forward neurons only and no heads.

Macroscale.

- **Macro budget shift:** redistribute the compute of a contiguous band of layers among them according to the layer importance scores.
- **Neuron rebalance:** even out the neuron counts across the active layers so the feed-forward width is roughly uniform.
- **Layer rewire:** permute the per-layer allocations, keeping the same multiset of layer sizes but changing which layer is large or small.
- **Group reallocate:** pick two non-contiguous groups of layers and redistribute their combined compute across them by importance.
- **Sparsity shift:** increase or decrease the candidate's overall head fraction by a small amount, with neurons compensating.
- **Jitter:** randomly increment or decrement the head and neuron counts in a few layers to produce close neighbours of the parent.

C Decoder Operators

The decoder searches edit a per-layer level vector, namely a drop flag, a bit-width, or a rank level.

Layer drop.

- **Single swap:** move one drop from a dropped sub-block to a kept one, keeping the drop count fixed.
- **Double swap:** apply two single swaps in sequence for a slightly larger move.
- **Triple swap:** apply three single swaps in sequence for a coarser move.
- **Convert to block:** turn a partial attention-only or feed-forward-only drop into a full-layer drop, and free a drop elsewhere to keep the count.
- **Split block:** split a full-layer drop into a separate attention-only drop and feed-forward-only drop placed at different layers.
- **Crossover:** combine the drop patterns of two parents and repair the child back to the exact drop count.

Quantization.

- **Bit swap:** raise one layer's bit-width by one level and lower another's by one level, keeping the average-bit budget.
- **Bit walk (2-step):** apply two bit swaps in sequence for a slightly larger move.
- **Bit walk (3-step):** apply three bit swaps in sequence for a coarser move.
- **Bit rebalance:** take one bit level from the highest-bit layer and give it to the lowest-bit layer.

- **Block coordinate:** raise or lower the bit-width of all attention layers, or all feed-forward layers, of one transformer block together, then compensate elsewhere to stay in budget.
- **Crossover:** combine the bit vectors of two parents and lower over-budget layers until the child fits the budget.
- **Hill climb:** try every single bit swap around the current best and keep the one that most lowers the loss.

Low-rank.

- **Rank swap:** raise one layer's rank by one level and lower another's by one level, keeping the parameter budget.
- **Rank walk (2-step):** apply two rank swaps in sequence for a slightly larger move.
- **Rank walk (3-step):** apply three rank swaps in sequence for a coarser move.
- **Rank rebalance:** take one rank level from the highest-rank layer and give it to the lowest-rank layer.
- **Block coordinate:** raise or lower the rank of all attention layers, or all feed-forward layers, of one transformer block together, then compensate elsewhere to stay in budget.
- **Energy-guided swap:** raise the layer with the largest marginal gain in retained spectral energy and lower the one with the smallest marginal loss, read from the stored singular values so the move needs no forward pass.
- **Crossover:** combine the rank vectors of two parents and lower over-budget layers until the child fits the budget.
- **Hill climb:** try every single rank swap around the current best and keep the one that most lowers the loss.

Appendix E: BERT Search Hyperparameters

A Core Algorithm

These are the default high-level settings used by the encoder structured-pruning search.

Parameter	Value
Population size	120
Number of generations	80
Elite size	15
Number of islands	1

B Archives

The archives preserve high-quality candidates and prevent repeated macro signatures from dominating the stored final-ists.

Parameter	Value
Hall of Fame capacity	50
Hall of Fame max per macro signature	3
Pareto archive size	100

C Elites and Offspring

Elitism preserves the strongest candidates, while the offspring schedule shifts toward larger edits as stagnation increases.

Parameter	Value
Elite size	15
Per-island elites	15
Micro/macro split	0.35 initial; 0.80 maximum; cap reached at stagnation 15
Crossover rate	0.25 micro, 0.65 macro, 0.40 default

D Stagnation

Stagnation increases the probability of broader structural edits and can trigger additional full evaluations.

Parameter	Value
Restart threshold	stagnation ≥ 20
Extra full evals	24

E EXP3-IX Bandit

Each operator starts from a positive weight. The sampling probability mixes the learned distribution with uniform exploration.

Parameter	Value
Exploration coefficient γ	0.07
Initial weights	$w[o] = 1.0$ for all operators
Probability	$(1 - \gamma) w[o] / \sum_j w[j] + \gamma / K$
Update	$w[o] \leftarrow w[o] \exp\left(\gamma \frac{r}{p + \gamma} \frac{1}{K}\right)$
Overflow clip	normalise when $\max_o w[o] > 10^6$

F Island Operator Priors

Each island corresponds to a broad allocation style. The table below each label lists only the operators whose initial weights differ from the default value 1.0.

Head-heavy, low-neuron. This island favours moving capacity toward attention heads while keeping the feed-forward allocation small.

Operator	Weight
budget_neutral_swap_neurons_for_head	3.0
budget_neutral_swap_head	2.0
head_neuron_trade	1.5
layer_rewire	1.0
jitter	1.0
equilibrium_mutate	0.8

Head-heavy, mid-neuron. This island still favours attention, but keeps enough neuron capacity for mixed head-neuron refinements.

Operator	Weight
budget_neutral_swap_neurons_for_head	2.0
budget_neutral_swap_head	1.8
budget_neutral_swap_neurons	1.4
equilibrium_mutate	1.4
head_neuron_trade	1.2
jitter	1.0

Balanced. This island starts near an even attention and feed-forward allocation, so its priors favour local swaps and equilibrium-preserving moves.

Operator	Weight
budget_neutral_swap_head	1.8
budget_neutral_swap_neurons	1.8
equilibrium_mutate	1.6
group_reallocate	1.2
macro_budget_shift	1.2
jitter	1.0

Neuron-heavy. This island favours keeping more feed-forward capacity and uses operators that move budget from heads into neuron blocks.

Operator	Weight
budget_neutral_swap_head_for_neurons	3.0
budget_neutral_swap_neurons	2.2
neuron_rebalance	1.6
group_reallocate	1.2
jitter	1.0

Layer-sparse. This island explores architectures with fewer active layers and stronger depth-level edits.

Operator	Weight
drop_layer	3.0
drop_multi	2.5
layer_rewire	2.0
macro_budget_shift	1.8
budget_neutral_swap_head	1.2
budget_neutral_swap_neurons_for_head	1.2
jitter	1.0

Appendix F: BERT Selected Configurations

A SST-2 Final Architectures

The table reports two selected BERT-base encoder configurations found for SST-2. Each row is one encoder layer. Active heads are reported out of 12 attention heads, and active feed-forward neurons are reported out of 3072 intermediate neurons.

Reading guide. Darker cells retain more structure. The 3.7M-parameter model reaches 91.60 accuracy, while the 4.0M-parameter model reaches 92.43 accuracy.

L	3.7M, 91.60		4.0M, 92.43	
	Heads	Neurons	Heads	Neurons
0	4/12	112/3072	1/12	85/3072
1	2/12	144/3072	1/12	89/3072
2	2/12	64/3072	1/12	89/3072
3	1/12	112/3072	1/12	89/3072
4	0/12	96/3072	1/12	89/3072
5	0/12	48/3072	1/12	89/3072
6	0/12	64/3072	1/12	89/3072
7	1/12	16/3072	1/12	89/3072
8	1/12	32/3072	1/12	89/3072
9	0/12	48/3072	1/12	89/3072
10	0/12	80/3072	1/12	89/3072
11	1/12	64/3072	1/12	89/3072

B QNLI and MNLI Final Architectures

The table reports the selected tiny encoder configurations for QNLI and MNLI. The same layer-wise convention is used: active heads are reported out of 12, and active feed-forward neurons are reported out of 3072.

L	QNLI 3.2M, 88.20		MNLI 3.1M, 81.36	
	Heads	Neurons	Heads	Neurons
0	0/12	0/3072	2/12	112/3072
1	1/12	32/3072	3/12	96/3072
2	1/12	64/3072	2/12	64/3072
3	1/12	48/3072	2/12	64/3072
4	3/12	48/3072	0/12	16/3072
5	1/12	64/3072	0/12	16/3072
6	1/12	48/3072	0/12	0/3072
7	1/12	48/3072	1/12	32/3072
8	0/12	0/3072	0/12	32/3072
9	1/12	48/3072	1/12	32/3072
10	0/12	32/3072	0/12	16/3072
11	3/12	32/3072	1/12	16/3072

C Moderate QNLI and MNLI Architectures

The table reports the selected moderate-compression encoder configurations for QNLI and MNLI.

L	QNLI 34.1M, 91.90		MNLI 34.0M, 85.60	
	Heads	Neurons	Heads	Neurons
0	3/12	1408/3072	5/12	1168/3072
1	1/12	1552/3072	5/12	1056/3072
2	1/12	2800/3072	6/12	1408/3072
3	0/12	2128/3072	6/12	1504/3072
4	0/12	1504/3072	5/12	1504/3072
5	3/12	1760/3072	6/12	1280/3072
6	1/12	1152/3072	5/12	992/3072
7	3/12	2176/3072	6/12	1840/3072
8	2/12	1472/3072	6/12	1664/3072
9	0/12	1280/3072	6/12	752/3072
10	3/12	1520/3072	5/12	208/3072
11	0/12	1248/3072	5/12	192/3072

D Aggregate Structure

Configuration	Heads	Neurons	Zero-head layers
SST-2 3.7M, 91.60	12/144	880/36864	5
SST-2 4.0M, 92.43	12/144	1064/36864	0
QNLI 3.2M, 88.20	13/144	464/36864	3
MNLI 3.1M, 81.36	12/144	496/36864	5
QNLI 34.1M, 91.90	17/144	20000/36864	4
MNLI 34.0M, 85.60	66/144	13568/36864	0

Appendix G: Decoder Selected Configurations

A Reading the Tables

The tables report selected decoder-side configurations for Pythia-1.4B. Rows are transformer layers, indexed from 0 to 23. The layer-sparsity table reports which sub-blocks are kept in the verified 50% configuration evaluated with approximately 131k calibration tokens. The quantization and low-rank tables group the four repeated matrix families per layer: attention output, query-key-value projection, MLP down projection, and MLP up projection.

Layer-sparsity entries show the kept sub-blocks: attention plus MLP, attention only, MLP only, or full layer drop. **Quantization levels** are shown directly as bit-widths: 2, 2.5, 3, 4, or 8 bits. **Low-rank levels** are shown as the retained fraction of singular values: 5%, 10%, 20%, 35%, 50%, or 75%. Darker cells indicate larger retained precision or rank.

B Layer Sparsity: 50% Target, 131k Calibration Tokens

Layer	Kept	Layer	Kept
0	Attn.+MLP	12	Full drop
1	Attn. only	13	MLP only
2	Attn.+MLP	14	Attn. only
3	Attn.+MLP	15	MLP only
4	Attn.+MLP	16	MLP only
5	Attn.+MLP	17	Full drop
6	Attn.+MLP	18	Full drop
7	Attn.+MLP	19	Full drop
8	Full drop	20	MLP only
9	Full drop	21	Full drop
10	Attn.+MLP	22	Full drop
11	Attn.+MLP	23	Full drop

C Quantization: 2.25-Bit Target

L	Attn. out	QKV	MLP down	MLP up
0	3	3	3	3
1	4	4	3	3
2	3	3	3	4
3	4	3	3	3
4	3	3	3	4
5	4	3	3	4
6	3	3	3	4
7	3	4	3	4
8	4	4	3	4
9	4	4	3	4
10	4	3	3	3
11	3	4	3	3
12	3	4	3	3
13	4	4	3	3
14	3	4	3	3
15	3	8	3	3
16	4	3	3	3
17	3	4	3	3
18	3	3	3	3
19	4	3	3	3
20	4	3	3	3
21	3	3	3	3
22	4	3	3	3
23	3	3	3	3

D Quantization: 2.5-Bit Target

L	Attn. out	QKV	MLP down	MLP up
0	4	3	3	4
1	4	4	3	3
2	4	3	3	4
3	4	4	3	4
4	4	3	3	4
5	4	3	3	4
6	4	4	3	4
7	4	4	3	4
8	4	4	3	4
9	4	4	3	4
10	4	3	3	4
11	4	4	3	4
12	4	3	4	3
13	4	4	3	3
14	3	4	3	4
15	4	4	3	3
16	3	4	3	3
17	3	4	3	3
18	4	4	4	3
19	4	3	4	3
20	3	4	3	3
21	8	4	3	4
22	4	4	4	3
23	4	4	4	3

E Low-Rank: 0.5 Target

L	Attn. out	QKV	MLP down	MLP up
0	35%	20%	75%	75%
1	35%	35%	35%	50%
2	10%	5%	35%	35%
3	35%	35%	50%	50%
4	35%	35%	50%	50%
5	50%	50%	35%	50%
6	20%	35%	35%	50%
7	35%	50%	35%	50%
8	20%	35%	35%	50%
9	35%	35%	35%	50%
10	20%	35%	50%	75%
11	35%	75%	35%	50%
12	50%	50%	35%	35%
13	20%	20%	35%	35%
14	50%	50%	50%	50%
15	35%	35%	35%	35%
16	35%	20%	35%	50%
17	10%	20%	35%	35%
18	10%	35%	35%	35%
19	35%	35%	20%	20%
20	5%	5%	35%	35%
21	5%	5%	35%	35%
22	10%	5%	35%	50%
23	20%	10%	35%	50%